

VECTOR

eXtreme APL ...

• Helsingør Øksekaſt	50
• APL2000 User's Meeting	64
• The APL2000 Grid Control	76
• Reiter on Hyperbolic Frisbee	91
• McDonnell on Instant Insanity	113
• Translating your Dyalog Gui	131



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433
www.vector.org.uk
Vol.17 No.3 January 2001

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2000–2001

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: apl385@compuserve.com.

Contents

		Page
Editorial	Stefano Lanzavecchia	3
APL NEWS		
Quick Reference Diary		5
British APL Association News		
Notice of Annual General Meeting		6
News from Sustaining Members		8
APL Product Guide	Gill Smith	12
The Education Vector		
Zark Newsletter Extracts	edited by Jon Sandles	27
Crossword		37
J-ottings 27: Obverse to Adverse :::: a Trip along the Brailleyway	Norman Thomson	39
REVIEW		
The Cogwheel Brain by Doron Swade	Anthony Camacho	45
RECENT MEETINGS		
Viking APL in Denmark		50
Dragging and Dropping (workshop notes)	Adrian Smith	56
APL2000 User's Meeting in Orlando	Adrian Smith	64
A Grid Object for APL+Win	Eric Lescasse	76
GENERAL ARTICLES		
Hyperbolic Symmetry is a Breeze	Cliff Reiter	91
Working with Existing Ascii Files Using J Memory Mapping	Donald B. Pittenger	103
TECHNICAL SECTION		
Technical Correspondence		112
At Play with J: Four Cubes Redux	Gene McDonnell	113
Reflexive Data Domains	Phil Chastney	121
Translate your Dyalog GULs	Kimmo Kekäläinen	131
APL Unicode Font - Extended	Phil Chastney	139
Index to Advertisers		143

Check out our new site and download J for free!



DOWNLOAD



Home

Search the site

Contact Us

About Us

Online Books

Order
Jsoftware
Here

Welcome to Jsoftware

J is a modern, array oriented, high level, high performance, general purpose programming language. J is used in a variety of industries including banking, investment analysis, insurance, derivative trading, scientific research, and education.

Jsoftware is available on a number of platforms including Windows, Macintosh, Unix and Linux. A free downloadable edition is also available.

- J4 06 Beta Announced
- Statistical Calculations with Ease
- Eric Iverson: President of J Development Speaks to Local New York Users
- Actuarial Analysis is J (pdf)
- Frisbees Given Away @ MAA
- Check Out the Script Library

What is J?

(pdf)

Download Adobe Acrobat [here](#).

www.jsoftware.com



Jsoftware Inc.
Powerful Programming Language

Editorial

by Stefano Lanzavecchia

*'Oh, you can't help that,' said the Cat:
'we're all mad here. I'm mad. You're mad.'
'How do you know I'm mad?' said Alice.
'You must be,' said the Cat, 'or you
wouldn't have come here.'*

L. Carroll - "Alice's Adventures in Wonderland"

What is APL for you?

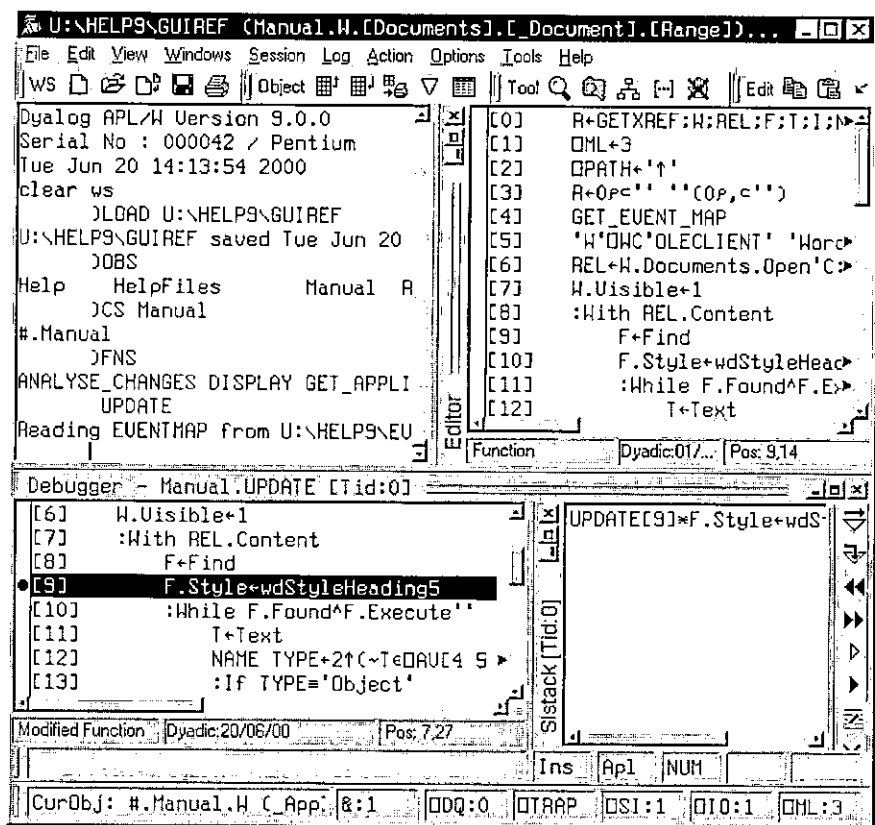
It's A Programming Language, but not just any programming language otherwise you would not be reading Vector. What is it that makes APL special for you? It's an interpreter, hosted in, and in the case of the Windows and the new Linux implementations completely based on, an old-fashioned interactive development environment, one of the few left in fact. It's a productive language, capable of quickly producing prototypes for the solution of complex problems, easily beating many of the fashionable Rapid Application Development tools. It's the nightmare of the maintenance of legacy systems, thrown together in a sleepless night of beers mixed with coffee, hastily put online and never to be taken off production because users loved them so much that the system became irreplaceable. It's the failure of all the dedicated task forces created to get rid of the last APL's symbol from the mainframes before the doom of the Y2K bug. It's the elegance of its terse syntax and the power unleashed by its array processing capabilities. It's the pain of its acceptance in the official IT circles. It's the old refrain from all the IBM dropouts "Does it still exist?". It's the offsprings, A, J, K, born from the original concept to develop their own view of high performance array processing. It's the long list of functions with three-letter names, where none of the variables need more than one keystroke to type. It's the feeling that it's cool to work out something by oneself even if it has already been done by somebody else. It's the tricks using the dyadic transpose combined with an outer product and a reshape to fool your puzzled colleagues. It's the I-can-do-it-in-one-line philosophy. It's also a lot of nice memories and friends scattered all over earth, from Australia, to Japan, from Russia, to England, from France to Canada, the annual conference, the Scandinavian meetings.

What is APL for you and what will it be for you in the new millennium?

I hear that the people behind A+, an APL dialect developed at Morgan Stanley, intend to release their interpreter in the public domain. Ask me for contact information if you want to take part in the alpha testing, which by the time you read this might have started already.

dyalog APL

The Definitive APL for Windows™



Version 9 - an even better IDE(A)

<http://www.dyadic.com>

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
Hants. RG24 7AL, United Kingdom.

Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

Quick Reference Diary

Date	Venue	Event
8th-9th Feb 2001	Helsinki, Finland	The 6th Annual Forest Seminar
18th May 2001	RSS, London	BAA Spring meeting and AGM
25th-28th June	Yale, USA	APL2001 Conference
TBA, 2002	Madrid, Spain	APL2002 Conference

Dates for Future Issues of VECTOR

	Vol.17 No.4	Vol.18 No.1	Vol.18 No.2
Copy date	9th March	8th June	7th September
Ad booking	16th March	15th June	14th September
Ad Copy	23rd March	22nd June	21st September
Distribution	April 2001	July 2001	October 2001

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

British APL Association News

Notice of Annual General Meeting

*2.00 p.m. Friday 18th May 2001
Royal Statistical Society, Errol Street, London EC1Y 8LX*

Agenda

- 1 Minutes of Annual General Meeting 2000
- 2 Chairman's Report
- 3 Proposal to increase subscriptions
- 4 Audited Statement of Accounts - Treasurer's report
- 5 Questions to Committee
- 6 Appointment of Auditor
- 7 Other business

Other business cannot be discussed without giving notice and may require that a notice is sent to members. If you wish to raise a matter of other business, please let the secretary know by 31st March 2001 at the latest.

Anthony Camacho, Hon Sec, December 2000

Nominations for the Committee for 2001-2002

Nominations are invited for next year's committee, to hold office from 1st May 2001 to 31st April 2002. Please send the name of the nominee (who must have consented to stand), the proposer and the seconder (all must be members of the Association) to the secretary, Anthony Camacho, 11 Auburn Road, Redland, Bristol BS6 6LS or by email to acam@tesco.net. Nominations must be in the secretary's hands before the 31st March 2001. If there is more than one nomination for any post there will be a postal election.

Here is a table showing the members of recent committees. If you haven't ever been a member do consider it.

Recent committees of the BAA

Post	1996-1997	1997-1998	1998-1999	1999-2000	2000-2001
Chairman	Alan Mayer	Anthony Camacho	Anthony Camacho	Anthony Camacho	Adrian Smith
Secretary	Sylvia Camacho	Ajay Askoolum	Ajay Askoolum	Ajay Askoolum	Anthony Camacho
Treasurer	Nicholas Small	Nicholas Small	Nicholas Small (co-opted)	Nicholas Small	Nicholas Small
Editor	Duncan Pearson	Vacant	Stefano Lanzavecchia (from Oct 98)	Stefano Lanzavecchia	Stefano Lanzavecchia
Projects & publicity	Anthony Camacho	Alan Mayer	Alan Mayer	Alan Mayer	Alan Mayer
Webmaster		Ray Cannon	Ray Cannon	Ray Cannon	Ray Cannon (co-opted)
Activities	Jon Sandles	Jon Sandles	Jon Sandles	Jon Sandles (co-opted)	Jon Sandles
Education	Ian Clark	Ian Clark	Ian Clark	Ian Clark	
Administration	Rowena Small	Rowena Small	Rowena Small	Rowena Small	Rowena Small

Anthony Camacho, Hon Sec

Sustaining Members' News

Dyadic Systems Ltd

One of the major new features introduced in Dyalog APL Version 9 is the concept of a *Namespace Reference*. This allows you to assign namespaces, to pass namespaces as arguments to functions, and to have arrays of namespaces.

Just before Christmas, Dyadic released further extensions namely *Namespace Reference Array Expansion* and *Distributed Assignment*. These enhancements provide additional syntax to reference and assign variables in arrays of namespaces.

For example, if you represent a personnel database by having a separate namespace for each person; where each namespace contains variables that specify details for that person, you can now pick out (and assign) those variables for some or all of the namespaces in a *single expression*. For example:

```
JOHN←[NS]'
JOHN.(FirstName LastName Age)←'John' 'Smith' 50
PAUL←[NS]'
PAUL.(FirstName LastName Age)←'Paul' 'Brown' 44
MEN←JOHN PAUL
```

MEN is an array of Namespace References.

The *Namespace Reference Array Expansion* enhancement allows you to reference variables in each of the namespaces as follows:

```
MEN.FirstName
John Paul
MEN.(LastName Age)
Smith 50 Brown 44
```

You can also assign values to these variables using *Distributed Assignment*.

```
MEN.Age←+1
MEN.Age
51 44
```

Dyadic continues to work with Microsoft to integrate Dyalog APL into the .NET Framework and has begun to demonstrate its capabilities to APL audiences around the world. Among these new features is the ability to use Dyalog APL as

a scripting language for ASP.NET web pages and to write and call Web Services from Dyalog APL.

Dyadic is also pleased to announce the availability of Dyalog APL/M Version 9 BETA for LINUX. It is expected that the production version will be available during first quarter 2001.

Soliton Associates Ltd.

SAX 6.0 Announcement

Soliton is pleased to announce the availability of SAX 6.0. An Enterprise Edition is available for IBM AIX, Linux and Sun OS, and a Personal Edition for Linux.

SAX 6.0 is supported for:

- IBM AIX 4.3
- RedHat Linux 6.2 and 7.0 for Intel
- Sun OS 6 and 7

The SAX Personal Edition is royalty-free for personal use, and can be downloaded from the Soliton FTP site.

The SAX Personal Edition is functionally identical to the SAX Enterprise Edition, except in areas such as:

- IBM AIX and Sun OS support;
- IBM OS/390 connectivity;
- Oracle, Sybase and DB2 connectivity;
- CORBA, SOAP and XML utilities.

In addition to more extensive functionality, the SAX Enterprise Edition provides the services of an Account Manager, and responsive telephone and e-mail support.

SAX 6.0 is the first fully POSIX-compliant version of SAX. The same compiler and source code are now used for all supported platforms, with resulting benefits for maintainability and reliability. Moreover, Soliton has successfully performed experimental ports of SAX 6.0 to other platforms, for example HP/UX,

demonstrating the advantages of POSIX. Soliton is following the MAC OS/X and FreeBSD platforms closely.

Enhancements in SAX 6.0 include:

- A new SVP (Shared Variable Processor) with no practical limit on the number of shares or the amount of memory used;
- A factor of up to 5 times increase in the speed of floating-point arithmetic;
- A factor of up to 10 times increase in the speed of sort (grade up and down);
- A new algorithm that speeds up "less", "epsilon" and "iota" by 25 percent in most cases;
- A set of interpreter shortcuts that speed up the most common uses of "take" and "drop";
- A new operator, "ply", that applies a function n times;
- A new operator, "tessallation", that divides an array into sub-arrays;
- An extension to "assignment" that allows "indirect" or "multiple" assignments.

SAX 6.0 introduces new glyphs: "@" and "&", for the "from" and "merge" primitives currently denoted by "{" and "}". The new glyphs are a first step towards reclaiming "{" and "}" for reuse in control structure and direct definition (dynamic function) facilities that are planned for future releases of SAX.

In SAX 6.0, the "old" glyphs for "from" and "merge" continue to be supported. To assist in their location and removal, a SAX start-up parameter has been added that enables tracing of their use.

Documentation

The SAX 6.0 Documentation Set is approximately 700 pages in size, and consists of a Language Guide, a System Guide, and a Handbook.

The Documentation Set can be downloaded from the Soliton FTP site, or ordered in printed form. The printed Documentation Set includes a CD containing SAX 6.0 for Linux and a PDF version of the documentation. The price of a printed Documentation Set is US \$120, including shipment.

SAX Java Interface (SJI)

The SAX Java Interface (SJI) can be downloaded from the Soliton FTP site for use with the SAX Personal Edition.

Java provides a large and growing range of platform-independent interfaces and utilities. One function of the SJI is to provide SAX with access to Java interfaces, such those for ODBC/JDBC, CORBA and SOAP; and to Java utilities, such as those for sound, compression, encryption and drawing.

Another function of the SJI is the support of Java-based client applications for SAX, for example graphical user interfaces.

SAX Integrated Development Environment (IDE)

Soliton continues to develop the SAX IDE (Integrated Development Environment) that was presented at APL Berlin 2000. The IDE is implemented in Java, because it is intended for multi-platform use, but the APL developer does not require Java knowledge to use it. The IDE will operate on any Java platform, for example Linux or Windows, simply requiring a network connection to a SAX server.

The first release of the SAX IDE will ease printing and editing, offer an APL explorer, a view over all objects in workspaces and files, and have contextual highlighting.

Because the IDE, as an application, is separate from SAX, the SAX S-task interface has been enhanced with transactions for transfer of objects to and from active APL workspaces. For use with the IDE, these objects are simply transferred to and from a set of Java objects, as required. The benefit of this simple architecture is that the IDE can fully utilize the facilities of Java, for example drag and drop, and nested trees, without extensive coding effort.

Contacts

Please visit the Soliton web site at www.soliton.com for information about the SAX Personal Edition.

For information about the SAX Enterprise Edition, please contact:

Andrew MacLeod (+1 416 364 9355) for North America
Laurie Howard (+31 20 646 4475) for Europe

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.

Please help us keep it up to date!

All contributions and updates to the Vector Product Guide should be sent to: Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
The Bloomsbury Software Company	APL+PC Version 11	250	Upgrade to version 11 gives free runtime (£120 from any version)
	APL+Win v3.0	1350	A 32-bit Windows-hosted interpreter that runs under all Windows platforms including Windows 95. Note: Upgrade for £350 from version 1.8 or 2, £920 from version 1.0
	Migration to APL+Win	1080	from APL*PLUS/PC APL*PLUS/DOS any version. from earlier versions of APL*PLUS II
	APL+DOS	1250	APL*PLUS II DOS is renamed to APL+DOS.
	Migration to APL+DOS	760	from APL*PLUS/PC
	APL Link	200	Database Access
	APL Link Pro	500	
	APL*PLUS II for UNIX	poa	APL2000's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.

	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL
	APL2 PC (European Version)	£348	Product No. 5604-280. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APIs; TIME facility
	APL2 for Windows version 1.0	\$1500	Product No. 5639-446, part number 4229558.
	APL2 Runtime environment	\$250	Part No. 39L8419 - Packager and runtime CDs. One additional runtime install \$200, 5 additional \$900, 10 \$1,500.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyadic Systems Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
Iverson Software Inc.	J on the Web online registration ...		
	J Educational Edition	\$95	
	J Standard Edition	\$295	
	J Professional	\$895	
	Books and accessories (discounts for reg users)		
	J Dictionary	\$50	
	J User Manual	\$50	
	J Phrases	\$50	

	J Primer	\$50	
	Concrete Math	\$40	
	Exploring Math	\$50	
	J User Conference Proceedings	\$35	
	Mugs, T-shirts, Mousepads	\$10 each	
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also
	APL+Unix	poa	distributes in Switzerland and Belgium. Call for price quotes.
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APLW	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL.68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL.68000 Level II Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
Oasis	Dyalog APL	poa	Dyalog Systems
	APL*PLUS	poa	Manugistics
	APL.68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APLW	995	Fully fledged Windows development environment.
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Sollton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyalog and ISI products	poa	
	USA		
	Dyalog and ISI products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.
APL Group (see Eventra)			
APL Software Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick.hott@juno.com.
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
The Bloomsbury Software Company (for VSAPL)	Enhancements & Sharefile	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	Compiler	poa	The First APL compiler!
	Sharefile/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	ReinPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win.
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.

CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0. At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/98/NT. Future plans include an APL+WIN version and later a LINUX version.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLD, IC/E and IC/I users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Eventra	Qualedi	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL+PLUS/PC
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, AIX, Solaris), Product Number 6764-082
		\$2500	for CMS, Product Number 5764-011
INFOSTROY	APL+PLUS/Xbase Interface (11/386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark
	<i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>		
JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version, \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W

	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	AP127/PC	poa	ODBC interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	ISAP	poa	MS Windows DLL for calling Dyalog APL/W from web pages via MS Internet Information Server. Visit www.lingo.com for a demo.
	AP747	poa	ODBC Interface for Dyalog APL/W
	QWIN	poa	Legacy DOS Windowing support for APL+Win
RE Time Tracker Oy	UITAW	poa	Comprehensive high-level Windows User Interface library for APL+Win and +II v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via ODE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL+PLUS PC & APL+PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) Including 2-disk set of utility functions (APL+PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Ajay Askoolum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.

Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
AUSCAN Software	Consultancy	poa	APL software development, training
Bloomsbury Software	Consultancy	300-750+ VAT	
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems Ltd	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know

where to find someone who is an expert in your version of APL and your application area, on your continent.

JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
George MacLeod	Consultancy	poa	Design and programming of new APL applications. Enhancing and maintaining existing APL applications. Porting existing APL applications from one APL system to another. Supporting users of APL applications. Experienced on both mainframe, UNIX and PC APL interpreters.
Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Object Oriented Ltd	Consultancy	poa	General APL consulting, code recycling - mainframe to PC, performance tuning.
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to APL-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.

Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Welghahead Systems	Consultancy	poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
Bloomsbury Software	Training	poa	Contact the company for details.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL FF350 (private) FF2800 (Company)		
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join

NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of City SI-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Marienhoehe 86, 25451 Quickborn, Germany. Tel: +49 4106 60977 Fax: +49 4106 67869 Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
Adaytum Limited	Douglas Rowley	Castlegate, Tower Hill, BRISTOL BS12 2JA. Tel: 0117 921 5555 Fax: 0117 922 7749. Email: sales@adaytum.co.uk
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnetlibm.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5456063-17
APL Club Germany	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461
APL Software\Services	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cais.com
Ajay Askoolum	Ajay Askoolum	42 Harworth Road, Redhill, Surrey RH1 5HT Tel: 01737-771643 Email: ajay@askoolum.freemove.co.uk
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@interlog.com
BACUS	Joseph De Kerf	Rooienberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19048, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888 Email: BeautifulSystems@gofis.net
Bloomsbury Software	Peter Day	Bloomsbury House, 74-77 Great Russell St., London WC1B 3DA, UK. Tel: +44(0)20 7436 9481 mobile +44(0)836 254660 Fax: +44(0)20 7436 0524 mobile +44(0)385 850629 Email: pd@bloomsbury-software.co.uk
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@tesco.net
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com

Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
Ian Clark	Ian Clark	1, Heifer Mill Cottages, Mosterton, Beaminstor, Dorset DT8 3HG, England. Tel: +44 (0)7931 370304. Email: earthspot@aol.com
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 865168 Fax: +39 11 812 2652 Email: codework@codework-it.com
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
GPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 7002820 Fax: +358 9 7002824 Email: dinosoft@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dp.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, UK. Tel: 01256-811125 Fax: 01256-811130
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-762-6648 Fax: +1 (408)-762-6627 Email: info@DynArray.com
Eventra	Stuart Sawabini	Merritt Crossing, 440 Wheelers Farms Road, Milford, CT 06460, USA. Tel: +1 (203) 882-9988. Fax: +1 (203) 882-9948 Email: ssawabini@eventra.com; eshaw@eventra.com
Evestic AB	Ole Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel/Fax: +46 778 4410 Email: ole.evero@mailbox.swipnet.se
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakawata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakawata@compuserve.com
Generat Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749328 Fax: 01622 749355 E-mail: martin@gsoft.freemove.co.uk
Godin London Incorporated	Gaétan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
H.M.W Trading Systems	Chris Hogan	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email: HMW@4tra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK Tel: 01483-771028 Email: bob.hoekstra@khamlin.demon.co.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: 100612.1057@compuserve.com
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept REN/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOSTROY	Alexei Miroshnikov	3 S. Tulen Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: aalm@infostroy.spb.su
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (416) 925-6096, Fax: +1 (416) 488-7559 Email: info@software.com
JAD Software	David Crossley	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net

Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buld.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022. Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KYU00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	Flat 3, 42 Queen Annes Road, Bootham, YORK YO30 7AF Tel: 01904-651544 Email: johc@jaustria.freemove.co.uk
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56,00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.htv.fi
Phil Last Ltd	Phil Last	148 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.ntl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.78 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Waiter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo_allegro@mail.com
Lucas Solutions	Jlm Lucas	Stubbedamsvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: akinloch@globalnet.co.uk
George MacLeod	George MacLeod	37 Newhouse Rd, Bovingdon, Herts, HP3 0EJ, UK. Tel: 01442-831385 Fax: 01442-831388 Email: george.macleod@simcorp.com
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E. Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	4215 St-Andre, Montreal, CANADA H2J 2Z3. Tel: +1 (514) 529-1434 Email: danb@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: Ellis@mrtfirm.demon.co.uk
NY Sig	David Siegel	PO Box 2697, New York, NY10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Theo Zwart, Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: info@oasis.nl or rijkse@oasis.nl
Object Oriented Ltd	Walter G. Fil	Am Grendel 2, CH-6004 Luzern, Switzerland. Tel: 41 41 418 70 70 Fax: 41 41 418 70 77 Email: info@object-oriented.com
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	1st Floor, 10 Raleigh Court, Priestley Way, Crawley, West Sussex, RH10 2PD, UK. Tel: 01293 562700 Fax: 01293 562699 Email: mailbox@optima-systems.co.uk
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 313, Newtown, CT 06470, USA. Tel: +1 (203) 270-9729 Email: rendata@apibooks.cnchost.com or orders@apibooks.cnchost.com
Resources & Results	Frank Rhodes	2438 W. Northgate Dr., Irving, TX 75062, USA. Tel: +1 972 523 5117 Email: frank@decision-support.net
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com

SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Laurie Howard	Soliton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email: sales@soliton.com
SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)31463 Fax: +1 (530) 504 8194 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (612) 470-7345 Email: sales@jsoftware.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulfhielm	Novator Consulting Group AB, Svärtdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CServe: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: sl@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
Weighahead Systems	Philip Bulmer	Camberley House, 1 Portesbery Road, Camberley, GU15 3RB, UK. Tel +44 1276 20789 Email: sales@weighahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ftp://software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
Adatym Limited	www.adatym.com/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/rb/apljourn.htm
AUSCAN	www.interlog.com/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Bloomsbury	www.bloomsbury.co.uk/software
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
COSY (Bob Armstrong)	www.cosy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de/ ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyadic Systems Ltd	www.dyadic.com/
DynArray	www.dynarray.com/

Eventra	www.eventra.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Hoekstra Systems	www.khamsin.demon.co.uk/about/hsl/noframe.html
IBM APL2	www.ibm.com/software/ad/apl
Infostroy	www.insight.dk/infostroy/
Insight Systems ApS	www.insight.dk/
Iverson Software Inc	www.jssoftware.com/
Japan APL Association	www.naska.co.jp/JAPLA/
KJK-tieto Oy	www.kjk-tieto.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	www.mackaykinlochltd.co.uk/~admk/
MicroAPL Ltd	www.microapl.co.uk/
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk
Renaissance Data	www.aplbooks.cnchost.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Snake Island Research Inc.	www.snakeisland.com
Strand Software Inc.	www.jssoftware.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

Zark Newsletter Extracts

introduced by Jon Sandles

I am pleased to report that we have some interesting entries from the last Zark Newsletter challenge. These can be found at the end of the following Zark extract. Once again I would like to encourage you all to enter responses as we can often shed new light on old problems, or even submit solutions in the more recent APL derivatives: J and K. In this case thanks to Morten Kromberg for submitting some interesting APL solutions, and to Stefano Lanzavecchia for the first K submission we have seen.

Corrections and Clarifications

Production department would like to apologise for the following howler in Vector 17.2. We received two entries for *CMIOTA*, one from Steve Jaffe and one from Carlo Spinicci (APL Italiana). Perceptive readers may have been puzzled as to why Steve had signed his code with an APLIT timestamp! Well, now you know. Here is Steve's code – the previous version really did belong to Carlo!

```

V I+X CMIOTA Y;J;K;Q;D
[1]  +S1×11=Y
[2]  +((2>ρρX)∨1<ρρY)/[LC+1 o Y+(-2+1 1,ρY)ρY  A Patch for 1 Y value
[3]  X+(2+(ρX),1 1)ρX  A Library of values
[4]  D=1+ρX+ρX  A-----flip to simulate 1 behavior on repeats
[5]  Y+(2+(ρY),1 1)ρY  A Values to look up in Library
[6]  X+(Q+X[;[IO]εY[;[IO]])×X  A Eliminate impossible X values
[7]  +(0=1+ρρX)/NUMERIC
[8]  K←[AVX  A Sort Library of values
[9]  I+I-[[I+(-1+ρY)+[AVX[K;],[[IO]Y  A LOOKUP for Characters
[10] +CONTINUE
[11] NUMERIC;
[12] K←X  A Sort Library of values
[13] I+I-[[I+(-1+ρY)+[X[K;],[[IO]Y  A LOOKUP for Numerics
[14] CONTINUE;
[15] J+~1=I+(-1,K)[[IO+I]  A Find Closest X value for each Y
[16] J+J\X[X[J/I;]=J×Y  A Eliminate all but exact matches
[17] I[(~J)/1ρJ]+[IO+(ρX)[[IO]  A Insert 1> ρX for all Y not found
[18] I+((Q/1ρQ),[IO+ρQ)[I]  A Expand to all X values (see [6])
[19] I+(1+D)-I  A-----correct for flip
[20] I[(I=0)/1ρI]+1+D
[21] →0
[22] S1:I+((+X)1+Y
V

```

Utility Corner: Comma-Delimited Data

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In last issue's LIMBERING UP column, we asked you to write a function named *COMMAΔMAT*. The function converts a character vector whose records are delimited by trailing newline characters (carriage returns), and whose fields within the records are delimited by commas, into a simple character matrix whose value are aligned by field.

Here's a sample:

```
26,154.25,Fred@
722,14.95,Jane@
17,0,Bob@
```

(We've used the @ symbol to represent the newline character, so you can "see" it. The @ symbol wasn't included last issue, and we received a few solutions that assumed the initial array was a matrix. Not so. It's a vector with embedded newline characters.)

Each of these records contain three values or "fields" of information. For this vector, here's the desired result:

```
26 154.25Fred
72214.95 Jane
17 0      Bob
```

The *COMMAΔMAT* function should also return a global vector named *<fieldwidths>*, whose elements are the respective widths of the fields in the matrix. In this case,

```
Fieldwidths
3 6 4
```

This vector allows you to readily extract a field from the matrix result. For example, if the matrix is named *CM*, you can get the *I*th field via:

```
CM[(0,+\fieldwidths)[I]+:\fieldwidths[I]]
```

When writing *COMMAΔMAT*, you need a way to squeeze out the unwanted commas and newlines, and to insert some well-placed blanks for proper re-alignment.

Bruce Frost sent a solution that uses compress (/) to do the squeezing and expand (\) to do the inserting:

```

▽ CM←COMMAΔMAT1 TEXT;D;E;I;L;NL;R;□IO
[1]  □IO←1  a Origin 1 works fine
[2]  NL←□TCNL  a APL+PLUS newline char
[3]  a NL←□TC[2]  a APL2
[4]  R←+/TEXTεNL  a Number of records
[5]  a Bits and indices of delimiters:
[6]  I←(D←TEXTε',',NL)/:ρTEXT
[7]  TEXT←(∼D)/TEXT  a Squeeze them out
[8]  L←I-1,1+~1+I  a Lengths of segments
[9]  L←(R,(ρL)÷R)ρL  a ... as a matrix
[10] fieldwidths←⌈L  a Biggest per field
[11] a Build expansion vector:
[12] I←1⌈/fieldwidths
[13] E←(,((ρL)ρfieldwidths)°.≥I)/,L°.≥I
[14] a Expand as needed, and reshape:
[15] CM←(R,+/fieldwidths)ρE\TEXT
▽

```

The logic is relatively straightforward up to line 13. On that line, two outer products are used to generate Boolean arrays that combine to generate the desired expansion vector. To illustrate the logic, consider the string 'Bob' in the sample above. Note *<fieldwidths>* is 3 6 4 and the value of *L* is 3 for 'Bob' (its length).

```

      ⌈/fieldwidths
6
      □←I+1⌈/fieldwidths
1 2 3 4 5 6
      4≥I      a 4=width of last field
1 1 1 1 0 0
      3≥I      a 3=width of 'Bob'
1 1 1 0 0 0
      ───6─── ───6───
      ───4─── ───3───
      1 1 1 1 0 0/1 1 1 0 0 0
1 1 1 0

```

The result, 1 1 1 0, is the segment of the expansion vector that expands 'Bob' into 'Bo**B**'.

Louis Zucchi's solution is similar to Bruce's but uses replicate (/) in place of compress and expand:

```

▽ CM←COMMAΔMAT2 TEXT;B;I;L;N;NL;R;⎕IO
[1]  ⎕IO+1  a Origin 1 works fine
[2]  a NL←⎕TCNL a APL+PLUS newline char
[3]  NL←⎕TC[2] a APL2
[4]  a Indices of delimiters:
[5]  I←(TEXTε','NL)/⍐TEXT
[6]  N←I;TEXT;NL a Number of fields
[7]  R←(⍐I)÷N a Number of records
[8]  L←1+I-1÷0,I a Lengths of segments
[9]  fieldwidths←[÷(R,N)⍐L a User biggest
[10] TEXT[I]+ ' ' a Blank out delimiters
[11] B←(⍐TEXT)⍐1 a Replication vector
[12] a Number of blanks needed per field:
[13] B[I]←((R×N)⍐fieldwidths)-L
[14] a Compress/expand, and reshape:
[15] CM←(R,+/fieldwidths)⍐B/TEXT
▽

```

The key to this function is the replacement of the commas and newlines by blanks so they can be subsequently compressed (replicated 0 times) or expanded (replicated 1 or more times).

Gary Logan's solution uses a nested APL2 approach:

```

▽ CM←COMMAΔMAT3 TEXT
[1]  a Use the newline to partition into
[2]  a a vector of char vec records:
[3]  CM←(TEXT*+1+⎕TC)⊖TEXT
[4]  a Use commas to partition into a vec
[5]  a (recs) of vecs (flds) of char vecs:
[6]  CM←(CM*',' )⊖CM
[7]  a Disclose to a mat (recs by flds)
[8]  a of char vecs:
[9]  CM←⊖CM
[10] a Enclose 1st dimension to get a vec
[11] a (flds) of vecs (recs) of char vecs:
[12] CM←⊖[⎕IO]CM
[13] a Disclose each item to get a vec
[14] a (flds) of mats (one row per rec):
[15] CM←⊖"CM
[16] a Disclose gives each mat item the
[17] a minimum no. of cols. How many?
[18] fieldwidths←+"-1+"⍐CM
[19] a OR:- fieldswidtgths←(⊖⍐"CM)[;1+⎕IO]
[20] a Cat the mats together via reduction
[21] a and disclose the scalar result:
[22] CM←⊖,/CM
▽

```

If you're not a daily user of nested arrays, this solution may be a bit perplexing. To help you understand the code, let's illustrate the steps on the sample problem above.

Parentheses are included to give a better view of nesting.

```
TEXT←      26,154.25,Fred@
           722,14.95,Jane@
           17,0,Bob@
```

```
[3]  (26,154.25,Fred) (722,14.95,Jane) (17,0,Bob)
```

```
[6]  ((26)(154.25)(Fred)) ((722)(14.95)(Jane)) ((17)(0)(Bob))
```

```
[9]  (26)      (154.25)  (Fred)
      (722)    (14.95)   (Jane)
      (17)     (0)       (Bob)
```

```
[12]  ((26)(722)(17)) ((154.25)(14.95)(0)) ((Fred)(Jane)(Bob))
```

```
[15]  (26      (154.25  (Fred
       722      14.95   Jane
       17 )     0       ) Bob )
```

```
[18]  3          6          4
```

```
[22]  26 154.25Fred
       72214.95 Jane
       17 0      Bob
```

Clearly, the approach of this function is entirely different from the approach of the first two functions. Which do you think is fastest?

We timed all three functions on an APL2 mainframe system. We used a character vector of length 63000 that defined 1000 records, each having ten fields. The fastest was *COMMAΔMAT2* (replicate), followed by *COMMAΔMAT1* (expand), which was 30% slower, and *COMMAΔMAT3* (nesting), which was 80% slower than *COMMAΔMAT2*.

We then time *COMMAΔMAT1* and *COMMAΔMAT2* on an APL*PLUS II system. The difference there was more dramatic. *COMMAΔMAT2* (replicate) ran four times faster than *COMMAΔMAT1* (expand).

On the APL*PLUS PC system, the character vector had to be shortened to 200 records (12600 characters) to avoid WS FULL from the outer products. At that size, *COMMAΔMAT1* (expand) ran five times slower than *COMMAΔMAT2* (replicate).

The winner: *COMMAΔMAT2*.

Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA

Vector Update

In Dyalog APL version 8.2, running under Windows 2000 in a 32 Mb workspace, the relative times were 174%, 100% and 161% for versions 1, 2 and 3 respectively. To get a reasonable number of seconds to make the timing comparison each function had to be run 2000 times on a character vector of length 63000.

Here are three modern day entries from Morten Kromberg:

```

      v←cam v;x;row;col;nl;cm;p;cw;f;b;s;o;fw;i;z
[1]  ⍝ Comma-separated to Char MX.
[2]  ⍝ Global SEP[1] is line delimiter, SEP[2] field delimiter
[3]
[4]  x←SEP[1],v,SEP[1]                ⍝ Put elephants in Cairo & Cape Town
[5]  row←+⍋nl×x=SEP[1]                ⍝ Which row is each char in?
[6]  col←col-⍋nl×col+⍋cm×x=SEP[2]    ⍝ Which field is each char in?
[7]  cw←⍋1+1+p-⍋1+0,p+(b+cmvnl)/⍋nl ⍝ Widths of each segment
[8]  f←1+col[⍋1+1+p]                 ⍝ Column number of each piece
[9]  fw←(×/s+(⍋1+⍋1+row),⍋/f)p0      ⍝ Ravelled field width matrix
[10] fw[(s[2]×⍋1+⍋1+b/row)+f]+cw     ⍝ Insert field widths
[11] fw←⍋spfw                        ⍝ Find greatest width of each field
[12] r←(×/s+s[1],+⍋fw)p' '          ⍝ Now we know how big r must be
[13] o←o-⍋b×o+⍋pb                   ⍝ Offset within each field of each char
[14] i←o+(⍋⍋1+0,fw)[1+col]]+s[2]×⍋1+row ⍝ Where in r does each char go?
[15] r[z/i]←(z+~b)/x                ⍝ Place them in r (except separator
                                   ⍝ chars)
[16]
[17] r←spr ◊ fieldwidths+fw
      v

```

```

v r←camΔapl2 x;sss
[1]  A APL2-style solution, thanks to Alexei for inspiration
[2]
[3]  r←(SEP[1]=r)←r+SEP[1],x      A Cut into rows
[4]  r←SEP[2],1+~r                A Replace line separator by field
                                A separator
[5]  r←(c0 1)+~>c[1]>(r=SEP[2])←~r  A Cut rows into fields, mix, split,
                                A drop leading separators
[6]  fieldwidths←2>~p~r          A One field in each cell now
[7]  r←+,/r                      A Extract the result
v
v r←camΔdya v;x;row;col;n1;cm;p;cw;f;b;s;o;fw;i;z
[1]  A Comma-separated to Char MX.
[2]  A Global SEP[1] is line delimiter, SEP[2] field delimiter
[3]
[4]  x←SEP[1],v,SEP[1]            A Put elephants in Cairo & Cape Town
[5]  row←+~n1+x=SEP[1]           A Which row is each char in?
[6]  col←col-[\n1×col++\cm-x=SEP[2]  A Which col/field is each char in?
[7]  cw←~1+i+p-~1+0,p+(b+cm~n1)/\pn1  A Widths of each segment
[8]  f←1+col[~1+i+p]             A Column number of each piece
[9]  fw←(1/f)p0 ◊ fw[f]{+cw      A [+ in Dyalog APL only, sorry: search
                                A FinnAPL idiom library for PMAxREDUCE
[10] r←(*s/+(~1+~1+row),+/fw)p' '  A Now we know how big r must be
[11] o←o-[\b×o+1pb              A Offset within each field of each char
[12] i←o+((+~1+0,fw)[1+col])+s[2]×~1+row  A Where in r does each char go?
[13] r[z/i]+(z←~b)/x            A Place them in r (except separator
                                A chars)
[14]
[15] r←spr ◊ fieldwidths+fw
v

```

Notes by Morten Kromberg

Here is my entry. It does not conform 100% with the spec, because I have made the separators configurable, you need to initialise the global variable *SEP* to contain the line and field delimited chars. In the workspace, they are set to *'/,'*, so you must replace *SEP[1]* with *newline*.

The comment on line 4 about elephants is a reference to an old joke about the PL/1 programmer who was asked how he would go about finding an African elephant. His explanation began: before starting in Cairo, I will send one elephant to Cape Town to make sure that the search will end.

cam is my actual entry. No enclosed arrays, just lots of scans and indexing - pure "ISO APL". *camΔdya* is a DyalogAPL only version, which is a bit shorter because it uses a *[+]* on line [9].

camΔapl2 is a MUCH shorter/simpler version using partitioned enclose (thanks to Alexey Miroshnikov for providing inspiration for this solution).

Depending on the arguments, the two functions take turns at being the fastest (it seems the APL2 version is actually significantly faster on quite a variety of arguments though, I would have sworn that the plain vanilla APL version would do better than it actually does).

Note `⎕ML` is 2 in the workspace, so that the "APL2" solution is APL2-like.

Sadly, they do no better than the three original Zark solutions:

```
COMMAΔMAT1 172
COMMAΔMAT2 100
COMMAΔMAT2 160
cΔm        298
cΔmΔapl2   210
cΔmΔdya    353
```

(Timings as percentages of the fastest)

I do not have the facilities to compare Stefano's K entry. However, here it is:

What about a solution in K? *(by Stefano Lanzavecchia)*

I won't claim I did it all by myself, not because I couldn't but simply because I remembered before starting that in the mail-list dedicated to K developers a solution to a similar problem had appeared in the past.

I will use this message (<http://www.kx.com/listbox/k/msg00544.html>) as base and will adapt it to the problem as described in Vector 17.2. I won't claim anyway that my solution is an example of good coding in K nor that it is an example of solving the problem with a fast K algorithm.

For a similar tutorial have a look at: <http://www.kx.com/a/k/examples/tab.k>

First of all: in K files are not read but mapped to a variable name. If the mapping is done using the operators that operate on text, the variable will be seen as a list of strings properly split at the end-of-line markers. Handy. Therefore, if I take for my example a file with 14 lines the following will apply:

```
data: 0:"file.csv"
#data
14
```

Now, since in comma-separated files, a text qualifier is used to allow fields to contain the separator, let's define two variables with the text qualifier and the separator

```
tq: "\"
sep: ","
```

This will allow us to interpret properly lines such as

```
"Fred, Jane", 10, UK
```

which is composed by 3 fields, the first one being

```
Fred, Jane
```

without the quotes. This makes the problem slightly more interesting and realistic, since most Spreadsheet and Database programs will export CSV files in this format.

Let's define a verb to mark the section of each line not enclosed in quotes (notice that in K the backslash is used to escape a quote inside a string):

```
uq: {~b[0]||':b:(~=)\y=x} \ dyadic verb: first argument is
                        the text qualifier
uq[tq; "\"Fred, Jane\", 10, UK"]
0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1
```

We also need a verb to remove quotes:

```
nq: dv[; tq]
nq "\"Fred, Jane\", 10, UK"
"Fred, Jane, 10, UK"
```

Please notice that in K strings are always printed with quotes surrounding them, but they are not part of the string itself.

We are almost there. We now need a function to cut a line at every separator taking into account the text qualifiers and removing unnecessary quotes.

```
cut: {(nq 1_) '(&uq[x[1]; d] &d=x[0]) _ d: (x[0]), y}
```

Once again the first argument is now the separator, followed by the text qualifier:

```
cut[sep, tq; "\"Fred, Jane\", 10, UK"]
("Fred, Jane"
 "10"
 "UK")
```

All we need to do is to repeat the cut on each line and calculate the max width of each column:

```
csv: {cut[x;]'y}
fieldw: {[/#:'csv[x;y]}
fieldw[sep,tq;data]
6 4
```

Now to generate a matrix as requested in the problem description we can use:

```
,/'(-fieldw[sep,tq;a])$/:csv[sep,tq;data]
```

Since `fieldw` calls `csv`, we are actually wasting quite a lot of work. We could factor the expression as follows:

```
COMMAdeltaMAT: {,/'{#[#x;,-|/#:'x];x)}csv[x;y]}
```

Actually, it doesn't make a lot of sense to repack the result of `csv` (already nicely cut field per field) into a matrix, but since this was part of the original problem statement, I decided to include this last step.

Exercises for the reader:

- extend the verb `cut` so that a quote inside a quoted string can be expressed as a double quote;
- make `fieldw` work in the case when in the original file not all the lines contain the same number of fields.

Since `K` is geared more and more towards supporting `Kdb`, one would expect a built-in operator to handle CSV files directly. Sure enough, with a little bit of patience, scavenging the terse documentation on the web site, I found that, given "`nf`" the number of fields per line, the verb I called `csv` can be rewritten as

```
nq' '+(nf#"C");sep) 0:"file.csv"
```

More, if one knows that the first 2 fields are integers, the third is a string, and the fourth is a floating point, one can write expressions like:

```
+("IICF";sep) 0:"file.csv"
```

and have the data automatically converted to the native types.

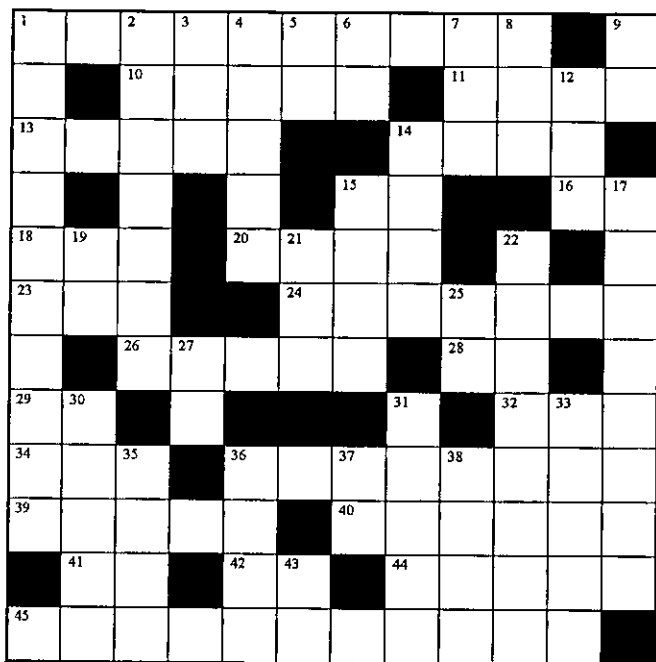
Download your copy of `K` from <http://www.kx.com/> and have fun.

Crossword

Solution to Crossword in 17.2

(	ι	0	)	ρ	A		Δ	F		→	0
ρ	ρ	ρ	F		×	N	-	L		(	
T	R	A	N	S	P	O	S	E		×	B
)	R	E	S	E	T			X	~	D	G
ρ				~	B		(		M	)	-
0	=	0	\	0	ρ	A		1	2	φ	V
	/	Γ		,	M		2	0	ε	Z	1
V	M	+		K		A	×			,	1
ι	^	/	B		5	×	*	N	V	P	
l	N	X		V	-	-	1	↓	0	,	V
/		↓		1	0	1		φ	ι	N	E
V	[	I	]	←	1		→	L	1		C

Crossword



Across

1. The frequency counts of the counting integers (origin 1) in the vector V, up to the biggest number in V
10. Round the numbers in N to the nearest integer (up if .5)
11. Return a vector of counting numbers with one number per element of array A, where $S + \rho A$
13. Are any of the numbers in vector B no bigger than five?
14. The first A elements of array BS
15. The reverse of $\sim T$
16. $N \times \sim 1$
18. $\rho \oslash M$
20. $1 \mid v$
23. $A[\sim 0]$
24. $(\subset \setminus G) = \sim \sim NNV$
26. T distinct random numbers from 1 to 125 (origin 1)
28. Where in the vector A is ...
29. $\div 10 \div 2$
32. $(1 \ 0 \ \forall \ 0 \ 1), 'TWO \ FOUR'[\square IO]$

- ```

34. ~M[; 1] = M[; 2] for two-column M
36. Branch to I if vector V does not contain the number A
39. ($\downarrow$ M) [$\square$ I O]
40. (- U S) + V without using $\uparrow$ or $+$
41. - / $\downarrow$ 2 5
42. (V > 0) - (V < 0)
44. (+ / 1 > NE) , (+ / 2 > NE) , (+ / 3 > NE) , ... in origin 1
45. VECT [T 1 + $\downarrow$ 1 0 0]

```

## Down

1. Number of columns in M, excluding its trailing all-zero columns
2. The lesser of V or B more than MAT
3. Given  $\iota N$  as both arguments, the function that returns an N by N bit matrix with 1s on and above its major diagonal
4. Does V contain the number .55?
5. ' $S = + / V \sim ' S / V$ '
6.  $+ \setminus N \rho 1$
7. Bits to indices ( $B \_\_\_\_ B$ )
8.  $\times / V, [(\square IO + \rho \rho V) - .5] B$  where  $(\rho \rho V) = \rho \rho B$
9. Reverse the dimensions of S
12. ' $1 \ 0 \ 1 / S - 1' \sim '1 \ 1 \ 0'$
14.  $(-A \in VV) / A$
15. The numbers in N5, capped at T
17. The adjective that precedes a number that precedes zero
19. Rank
21.  $0 \sim N2$
22.  $\succ'' C$  (APL\*PLUS) or  $\uparrow'' C$  (APL2), for nonempty items in C
25. The simple elements of nested A, in the order they occur
27. Flip a coin: 1 I win, 2 you lose
30.  $, (\iota 10) \circ . + 0 \ 0$
31.  $v / U = -V$
33.  $\phi^- 1 + NT$
35.  $M - (M > 30) \times M - 30$
36. Go to M if T is true; exit if false
37.  $\pm '\phi' '\phi A' ''$
38.  $E[VS / \iota \rho E]$
43. The name of the first vector

# J-ottings 27: Obverse to Adverse ::: a Trip along the Brailleway

with Norman Thomson

Welcome to the Vector Value excursion from Obverse Central to Adverse! Don't forget you need an inverse for the return journey – personalised if you like – and note that the inverse is also mandatory (a) when the *train* gets *under* (&.) way and (b) when it *powers* (^:) up with a negative argument. For many primitive monadic verbs and a few dyadic ones an inverse function is well defined, and there's a list of these on a billboard further down the line. For example *log* and *power* are inverse operations, and this underlies the use of &. and ^: in the following examples :

|     |             |                                                   |
|-----|-------------|---------------------------------------------------|
|     | log=.^.     |                                                   |
|     | 2+&.log 3   | NB. e to the power ln2 + ln 3                     |
| 6   |             |                                                   |
|     | 2 ^.^:(_1)3 | NB. 2 inverse-log (that is, to the power) 3       |
| 8   |             |                                                   |
|     | 2 ^.^:(_2)3 | NB. 2 inv-log inv-log (i.e. to the power twice) 3 |
| 256 |             |                                                   |

The first of these examples performs a transformation, "log to base e", on both arguments prior to the primary operation *plus*, then performs the "inverse log" transformation on the result. In the second and third examples parentheses are necessary to separate the arguments of the adverb ^: and the adverbially modified verb ^..

Suppose that through some perversity you decided to conclude the log operation by squaring rather than taking anti-logs :

|        |               |                                  |
|--------|---------------|----------------------------------|
|        | log=.^.. :. * | NB. log with "square" as obverse |
|        | 2 +&.log 3    | NB. square of (ln 2 + ln 3)      |
| 3.2104 |               |                                  |

It is hard to see the circumstances in which this would be a sensible thing to do; however when you define your own verbs, J may not be able to define an inverse, and so you are given the opportunity to define your own through *obverse*. The obverse does not need to have a direct connection with the primary verb, nor does it affect its operation, other than in the contexts of *under* and *negative power*. For example suppose you have a primary operation which joins names which are to

have trailing blanks deleted prior to the join, with a final comma to be added after the join :

```

 dtb:={{-i.&' '}:. (&','') NB. delete trailing blanks with obverse
 (dtb 'mickey '),'*' NB. obverse does not affect normal use
mickey*
 'mickey ' ,&.dtb 'rooney '
mickeyrooney,

```

In ordinary usage the obverse side of a coin means the side commonly turned towards you, which in turn usually means "heads" as opposed to "tails". This reflects quite well the idea of obverse in J, because it emphasises (a) that when you use it you are thinking of a verb as a two-sided object, and (b) as with a coin there is no reason why the designs on the two sides should be directly related, although often the relationship is inverse in nature. Another example involves a frequency table verb `frtab` :

```

 x=.0 0 1 1 2 2 3
 frtab=."/1@=
 frtab x
2 2 2 1

```

A natural inverse operation would be to "unwrap" the frequencies back to the original data which is what the verb `convert=.# 1.@#` does. So redefine

```

 frtab=."/1@= :.convert
 frtab ^:_1 frtab x
0 0 1 1 2 2 3

```

The conductor will now give a further demonstration of obverse by converting between percentages and multiples as in, e.g. handling compound interest calculations. Suppose the percentage growth rates of an investment are known for a number of years, and it is desired to compute the average overall growth rate throughout the entire period. The following two verbs convert percentages to and from multiples:

```

 petom=:>:@(*%0.01) NB. %age to multiple, e.g. 5 becomes 1.05
 mutop=:;%0.01@<: NB. multiple to %age, e.g. 1.05 becomes 5
 */petom 5 6 13 _12 0
1.10677

```

Now suppose that this result should be given as percentage :

```

 petom=:>:@(*%0.01) :. mutop NB. %age to multiple with obverse
 */%petom 5 6 13 _12 0
10.6767

```

To compute the *average*, as opposed to *total*, growth over the period introduce *under* :

```
gm=.(+/%#)&.^."_ NB. geometric mean (= arith mean under log)
gm&.petom 5 6 13 _12 0
2.04959
```

It is reasonable to retain the name *petom* for the "obverse-enriched" verb since adding the "flip side" verb *mutop* does not affect its previous function; it is only in the presence of *under* and *negative power* that the enrichment occurs. In this case the obverse is just the inverse, which J is smart enough to figure out for itself, since it has a rule that the inverse of a verb *u@v* is  $(v^{\wedge} _1)@(u^{\wedge} _1)$ , assuming of course that *u* and *v* are among the list of primitive verbs for which inverses can reasonably be computed, as is the case with *petom*. So why bother to write an obverse when all those bright J folks have already given you what you probably wanted anyway, and moreover have provided a conjunction *b.* which by joining a verb to *\_1* returns its obverse :

```
petom=:>:@(*&0.01) NB. back to the original petom
petom b. _1 NB. compute the obverse ..
%&0.01@<: NB. .. which is identical to mutop.
```

Well, let's suppose that on the flip side (converting multiples back to percentages) you decide to display an explicit percent sign. Easy, just do

```
mut=.,&'%'@(":@mutop) NB. enrich mutop with %
mut 1.05
5%
petom=:>:@(*&0.01) :.mut NB. redefine obverse of petom
gm&.petom 5 6 13 _12 0
2.04959%
```

The verb *petom* provides a salutary lesson in the power of parentheses. The verb

```
pet=:>:@*&0.01
```

means  $(>:@*)\&0.01$  and is indistinguishable from *petom* when used directly. However in inverse mode, and bearing in mind that conjunctions have long left scope, the rule for the inverse of *petom* is

```
(*&0.01 ^:_1)@(>: ^:_1)
```

that is  $(\%&0.01 \wedge _1)@(<:)$ , or in plain English subtract one and multiply by 100 (that is, divide by 0.01), which is exactly what *mutop* does. The computed obverse of *pet*, however, is a somewhat complicated verb, which in the case of a

scalar argument reduces to division by 1.01. The programming lesson is that whereas parentheses may be redundant in the normal operation of a verb, they can sometimes have a profound effect on its obverse.

Kirk Iverson recently provided on the J Forum a dazzling example of the use of obverse. The problem is to segregate a vector of integers, say 4 2 2 4 1 4 8, into boxes with empty boxes for those integers in sequence not present. Boxing like items is achieved by using the vector itself as a *key* (/.)

```
k=.4 2 2 4 1 4 8
k</.k
```

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 4 | 4 | 4 | 2 | 2 | 1 | 8 |
|---|---|---|---|---|---|---|

Now augment the vector with say i.10 :

```
aug=.(i.10)&,
aug k
0 1 2 3 4 5 6 7 8 9 4 2 2 4 1 4 8
```

and finally add an obverse which beheads *under* box so that the artificially added i.10 is wiped out item by item each from its own box :

```
aug=.(i.10)&, :. (}.&.>)
```

Finally do the initial box by key *under* aug which achieves both numerical ordering and also empty boxes where appropriate :

```
</.~&.aug k
```

|   |   |   |  |   |   |   |  |  |  |   |
|---|---|---|--|---|---|---|--|--|--|---|
| 1 | 2 | 2 |  | 4 | 4 | 4 |  |  |  | 8 |
|---|---|---|--|---|---|---|--|--|--|---|

Previous paragraphs raise the question of which verbs have "reasonable" inverses, to which the answer is - quite a lot. First there are self-inverse monadic verbs for which an even number of applications amounts to "do nothing". These are

```
arithmetic + - -. (not) % %. (matrix divide)
structural |. (reverse) |: (transpose) /: (grade up) [] and C.
 (see J-ottings 24)
```

Next here is a set of monadic verbs which have inverse partners, that is for each of these verbs  $v \wedge \_1$  is equivalent to its partner :

|            |                                                                 |
|------------|-----------------------------------------------------------------|
| arithmetic | <: and >: (increment / decrement)                               |
|            | + : or +~ and -: (double/halve)                                 |
|            | *: or *~ and %: (square/square root)                            |
|            | ^ and ^ . (power/log)                                           |
|            | #. and #: (base/anti-base = decode/encode in APL)               |
|            | o. and %&(o. 1) (multiply by o / divide by o)                   |
|            | j. and %&0j1 (multiply by j / divide by j)                      |
|            | + . and j. / (vector from/to complex no.)                       |
| structural | < and > (box and open)                                          |
|            | ;~ and >@{ . (self-raze, i.e. box then duplicate/head and open) |
|            | \: and /:@  . (grade down/gradeup following reverse)            |
|            | ,: and { . (itemise/head)                                       |
|            | ". and " : (do and format)                                      |

There are also families of monadic verb pairs formed by "currying" dyadic verbs with a noun:

+&n and -&n (add/subtract constant)  
 \*&n and %&n (multiply by/divide by constant)  
 n&^ and n&^ . (power of n/log to base n)  
 n&o. and \_n&o. (for appropriate n, e.g. sin and arcsin)  
 n&| . and \_n&| . (shift n left/ shift n right)  
 p&|: and (ip&|:) (ip = inverse permutation applied to transpose)

The generic property of the inverse  $i$  of a monadic verb  $v$  is that  $(i@v)$  is equivalent to  $] .$

The "scans" of five arithmetic verbs, namely  $+ * \% =$  and  $\sim$ : can be reversed by  $\wedge \_1$ , that is each of these verbs has the property that  $v/\wedge \_1$  is equivalent to  $] .$

Dyadic verbs have potentially two kinds of inverse, left and right. For example the dyadic  $+$  has a right inverse function  $-$  because  $a+b-b = a$ , that is subtraction on the right "cancels out" addition. However dyadic  $+$  has no left inverse because there is no function  $i$  for which  $a \ i \ a+b = b$  for all  $a$  and  $b$ . By the same reasoning *minus* has a left inverse *minus* and a right inverse *plus*. Inverses for dyadic verbs are rarer than for monadics.

The following is a list of them :

|               | + | - | * | % | ^  | ^, | = | ~: |
|---------------|---|---|---|---|----|----|---|----|
| Left inverse  |   | - |   | % | ^. | ^  | = | ~: |
| Right inverse | - |   |   | * |    |    | = | ~: |

A left inverse *i* is characterised by the fact that the fork [*i* v is equivalent to ] and a right inverse by [*i* v] being equivalent to [ .

"Adverse", like "obverse", is used to define a two-sided verb, but now it provides an *alternative* verb, the one you use in *adversity*, when the first one fails. It is unlikely to be an *inverse*, (although in principle it could be!) If the first verb fails, then the adverse takes over. Here, for example, is a protection device against "index error"

```

3 4{'ABCDE'
DE
3 5{'ABCDE'
index error
from=.{ ::]
3 5 from 'ABCDE'
ABCDE
```

One obvious circumstance in which you might think this could be applied is

```
div=.% :: 1:
```

to prevent attempted division by zero but this doesn't work because division by 0 is not an error in J - the result is infinity ( \_ ). The only circumstance in which 0 would be delivered is on an attempt to divide characters, or two vectors of unequal lengths, both greater than 1.

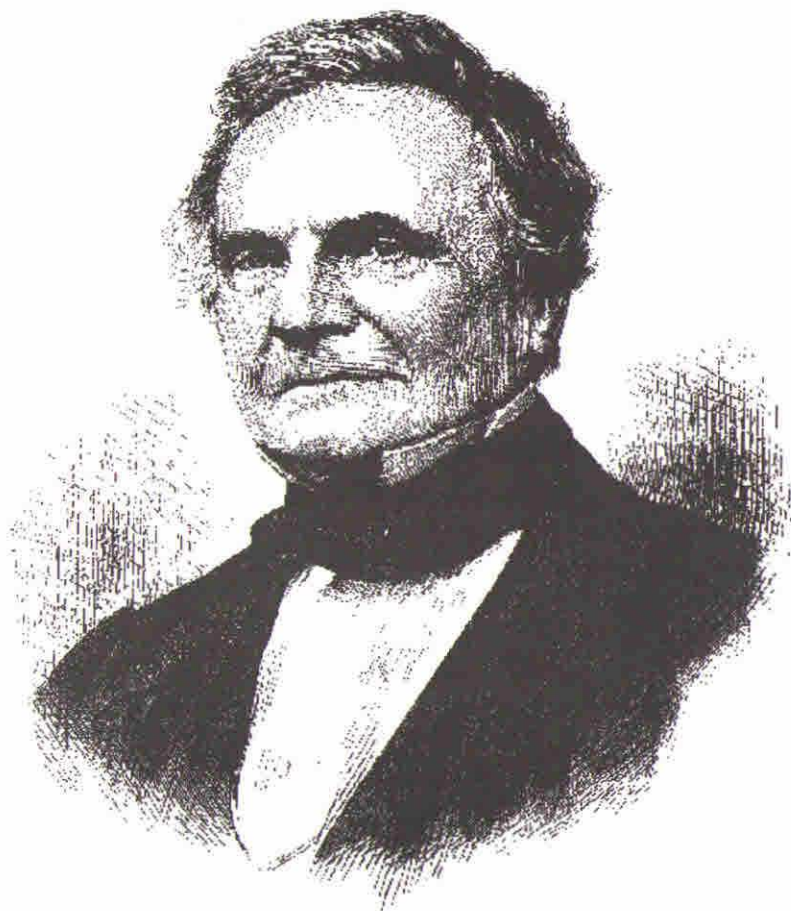
Since they connect verbs, "obverse" and "adverse" are conjunctions, but unlike the more commonly used conjunctions they define "flip-sides", rather than creating fundamentally new composite verbs. To summarise, an obverse is a user-defined inverse, adverse is the emergency exit to forestall error conditions.

And to get things back on the rails, there's a regulator too - if you didn't like the ride, just have a word with the driver's son!

## The Cogwheel Brain by Doron Swade

*reviewed by Anthony Camacho*

The history of computing is full of dead ends and new developments that are, as Clive Sinclair would be glad to hear, a quantum leap on from their predecessors.



Charles Babbage *Illustrated London News* 1871

In the 18th and 19th centuries tables of any kind from logarithms to tide tables were calculated by human computers and then typeset. Many errors crept in as a result. Charles Babbage (1791-1871) was convinced that mechanical devices could be made to do the whole task, including printing out the results or producing a 'stereo' (a printing master) which would allow many copies to be printed of perfect tables.

I have been interested in Charles Babbage since 1960 or 1961 when I saw a replica of part of Difference Engine No 1 in the IBM offices in Wigmore Street. I saw Doron Swade's book advertised and I had to read it. It is partly a biography of Charles Babbage with emphasis on the work on the calculating machines and partly an account of the building of a replica of Difference Engine No 2. Doron Swade was appointed curator of computing at the Science Museum in 1985 and took up a suggestion, made by Allan Bromley of the Basser Department of Computer Science at Sydney, that a replica be built as a celebration of the bicentenary of Babbage's birth (26th December 1791).

In the 1820s Babbage worked on Difference Engine No 1 and succeeded in getting the Treasury to agree to contribute to construction of it with a first payment of £1500 in 1823. [The wages of a labourer in 1825 were the equivalent of 45 to 60 pence a week in today's money!]

Apart from the problem of persuading people that the task was possible and worth the effort, Babbage had to solve some very difficult problems of how to make machinery reliable enough to do the task. No-one before had tried to make a thousand toothed wheels which were so similar as to be interchangeable. If the cylinder of a steam engine of the time was bored so accurately that a sixpenny piece would not pass between piston and cylinder at any point, it was exceptional. Whitworth, who first standardised screw threads, was an apprentice working for Babbage's engineer, Clements, in the 1830s. The first toothed wheels made for Babbage came back with the wrong number of teeth! As Babbage worked he would sometimes find a better way to do something and would backtrack, changing the design to incorporate the improvement. Some marvellous demonstration pieces were made but he never finished any of the machines he designed. He didn't pay enough attention to contracts and in 1833 and 1834 work stopped for sixteen months because the manufacturing engineer claimed ownership of the drawings and parts so far produced.

By the time the dispute was settled the Treasury had paid £17,478 towards the Difference Engine. Babbage, frustrated by having to stop work on No 1, designed No 2. Because no-one ever tried to make No 2 the drawings were complete and

unmarked by use in a workshop and this was a reason for choosing this to build as a replica.

The builders soon discovered that the drawings contained designs that could never work; perhaps as an insurance against the ideas being stolen, but perhaps plain errors. They had to make many amendments before a practical replica was possible.

The early design for No 1 has six columns of 12 wheels (six twelve digit registers). By the time the dispute with Clements was settled it had seven columns of sixteen digits. Difference Engine No 2 was designed with eight columns of 31 wheels. These were both comparatively simple machines which calculated tables by adding successive differences. The Analytical Engine of the 1840s was a multi-function calculator with a program of operations on a pack of punched cards with the capacity to branch between multiple courses of processing according to results so far. It was, in principle, not far from the universality of a modern computer. Babbage thought big. One design had 100 columns of 50 wheels and a store of 1000 fifty digit numbers. If it had ever worked it would have had far greater precision and more digits in its store than the first commercial electronic computer to sell 1000 copies - the IBM 650 - which had three registers and 2000 ten digit numbers on its drum store. But the IBM 650 could transfer a number between registers in 0.96 of a millisecond, add two registers in about 7 milliseconds and the longest it could take to read a number from, or write a number to, the drum was 48 milliseconds. Babbage was aware of the need to operate the machine as fast as possible and was proud of the invention of a carry mechanism that, by 'foreseeing' where carries would be needed, greatly reduced the time it would take - if forty-nine wheels held a nine and were to have a one added to them it would take forty-nine times the carry time to complete the operation. Apart from this it raised great problems in keeping the rest of the machine synchronised. He reckoned that the machine would be able to do sixty additions or subtractions a minute.

Before reading Doron Swade's book I had not realised the extent to which Babbage was engaged in polite society, giving and attending dinner parties, going to theatre and opera, entertaining and being entertained. The list of his friends reads like a who's who of the great and the good of London society. He also, as a Fellow of the Royal Society, was acquainted with the foremost inventors, chemists, engineers of his time - for example Herschel, Sir Humphry Davy, Sir Marc and Isambard Brunel, Stevenson, Maudsley, Laplace, Fourier, Poisson.

On the other hand I did know that Babbage was an arrogant, obstinate and irascible man who grew crankier as he aged and waged a bitter war against barrel

organs and noise. Perhaps, because it is so colourful, this facet of his character is over-emphasised.

When I read *Longitude* (an account of Harrison's chronometers and how he won the prize for an accurate method of navigation with the fourth of them) I was disappointed to be told so little about the differences between the four chronometers and how each improved on its predecessor. Doron Swade is not so disappointing, but still gives too little for me. For example he does not explain how the wonderful carry mechanism works.

The last third of the book is the account of the building of the replica of Difference Engine No 2. It is a fascinating story. Swade, like Babbage, had to persuade people the job could be done, solve all manner of technical problems (with the added *anachronism* problem that no solution was allowed which could not have been used by Babbage), persuade people the effort was worthwhile and raise the money. Swade succeeded where Babbage failed.

The book ends with a 13 page bibliography. Apart from a couple of memoirs by Major General H P Babbage (his son) there is no entry between the 1860s and the 1960s. After his death Babbage and his machines were forgotten. Only *after* modern computers were well on their way did the modern enthusiasm for Babbage arise.

But Babbage did a great many other things. He invented and ran machinery for recording the state of the permanent way (a dozen pens recording on a reel of paper at 44 feet of paper per mile travelled); he invented the system of occulting lights to distinguish between lighthouses; he proposed a system of mechanical notation for describing machinery's moving parts; he did original work on ciphers - and much besides.

In short, I recommend the book; it is well written and full of interesting things.

The Cogwheel Brain by Doron Swade: published by Little, Brown & Co. 2000, ISBN 0 316 64847 7.

# The VikAPL Experience – October 2000

*notes by Adrian Smith, photos by Adrian Smith and Jim Lucas*

## Where and Why

The idea of VikAPL was born early in 2000, rested for a while and was crystallised at Berlin, when it became clear that there was a real demand for a short, workshop-based gathering for the Scandic APL community. In part this was to share some of the Berlin material with a wider APL audience which had been unable to afford to take a full week out in July, and in part it was a desire to explore technical (rather than academic) issues in significant depth.

Accordingly Insight Systems (Helene Boesen did the bulk of the organisation) yielded to the pressure to set something up, and collared Pete Donnelly, Adrian Smith and Conrad Hoesle-Kienzlen who 'volunteered' to do the actual talking bit. The hope was to get around 25 attendees, and the realistic target was 20. Maybe the Millenium Dome could learn something here? Helene had to close the doors on late-comers when the numbers reached 40 and the capacity of the seminar rooms was clearly going to be exceeded.



Gilleleje at Dawn

The venue was excellent, the price very reasonable, the company good (as always at APL gatherings) and the social programme unbeatable (see below!). The

meeting took place at the Gilleleje Holiday Centre which consisted of a central 'hub' housing the restaurant, sauna, bar, and seminar rooms - and a scattered collection of chalets distributed amongst the sand-dunes along the nearby coast. A brisk half-mile walk to breakfast does wonders to ensure the audience is awake for that 9:00am start!

The scheduling allowed for one full day on the Thursday (with an optional late arrival the night before) and most of the day following, after which many of the delegates retreated to the Hotel Hamlet in Helsingør for the Friday and Saturday nights (the reduced cost of the flight more than paid for the extra stopover time). As always, much useful work was done out of hours, especially as we ate together on the Friday night also.

### **Presentations and Seminars**

I was much too involved in preparing and presenting material to be able to take good notes here! I believe I scored a hit with my impression of a flock of birds on the Heathrow runway when faced with a change of wind direction (and hence oncoming Jumbo-jets) - this was actually introducing a serious example of graphics and data-analysis as part of Adrian's Causeway seminar! Pete Donnelly took us through the mechanics of setting up the Dyalog DLL as an in-process OLE server (improving steadily in Dyalog Version 9, and very easy to run from an ASP script in a web-server) and Conrad showed us a strategy for replicating the mainframe interface using TCP/IP and an AP124-style engine.



**Pete Donnelly showing Dyalog 9 as an OLE Server**

The other two Causeway seminars were relocated to the downstairs bar (actually a rather good venue) where the primary focus was on documentation development with the originally-titled 'HelpStuf' application. I have been using this for many years to build help-files and large slabs of the website, as well as the paper manuals. Every time I think I may have finished it (to the point where it is stable enough to sell safely) something unexpected comes along and I start bolting on new stuff. Last year it was the new-format 'compiled HTML' files - myhelp.CHM - and this year it is the near-universal support for CSS-style sheets. I would very much like to do PDFs as a native format, but it is still unclear whether this is a viable long-term option or if HTML will overtake it soon.

This time, I used the seminar as an opportunity to test out the idea of a truly co-operative help-development system which could run locally on a LAN or even over the internet. Richard Smith used part of the time to show a TCP/IP application server which he developed for Adapta DLS in the week before the Berlin conference. Rather than hosting a multi-player production-scheduling application, this was used to run a multi-player tournament of a simple 2-player boardgame (see *Vector* 10.1 for the original design).



**A Coast-to-Coast Tournament  
in Progress**

This was an excellent pressure-test of the server, as well as an interesting exercise in multi-user system design and aeroplane code for the Causeway Company. More usefully, it provoked some very good feedback on the essential elements of a trans-national documentation engine, in particular the requirements for multi-language support. The 'single-user, single-site' product will be for sale by the time you read this, and the working design for the 'multi-user, single site' version was scribbled on the back of one of Pete

Donnelly's handouts in the Hotel Hamlet on the Saturday. Multi-user, multi-site is much harder (until the internet gets so fast that the whole world counts as one site) but no-one else has written this one yet!

We also had time for a couple of short sessions on RainPro and NewLeaf, which were really problem-solving and debugging opportunities for existing users. I think the score was about 50:50 in terms of user-misunderstandings to genuine bugs which I knew about but had been unable to reproduce well enough to fix them. I think everyone went home happy as a result.

## The Viking Challenge

This was the fun-part of the experience, and will doubtless be remembered by the delegates long after the APL code has faded into obscurity. The first lunch break was followed by such traditional winter activities as log-sawing and axe-chucking, all against the clock and with a strongly competitive atmosphere between the four teams. I think Jim took the classic picture here, with both axes in flight ...



**Axes away!**

... and interested readers may attempt the Vector 'Spot the Axe' contest online or by faxing in a copy of the following page with a maximum of 3 clearly marked crosses. The cross nearest to the head of the axe wins!

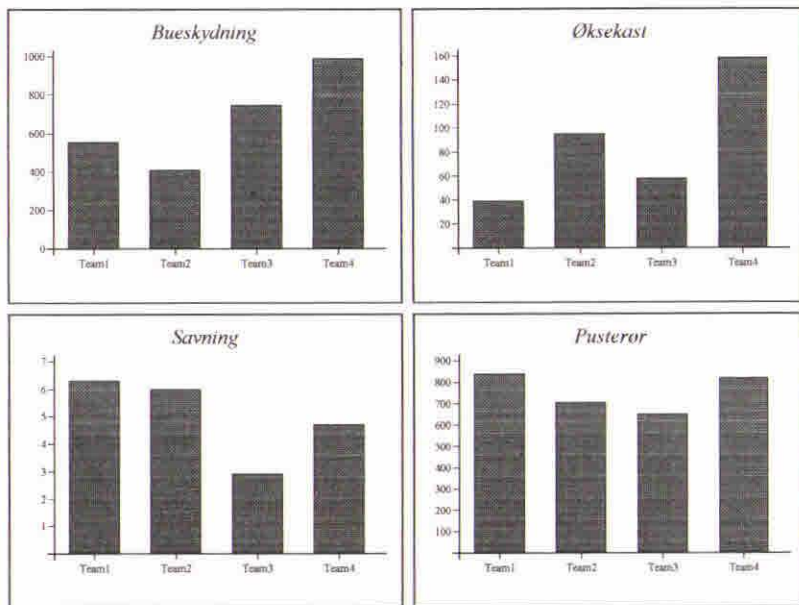
## Spot the Axe...



**Spot the Axe (mark with up to 3 crosses and your name)**

The 2 lucky winners will have the honour of being used as targets in next year's contest. No, seriously, we will think of something, probably Danish, very likely alcoholic, to be presented at the AGM. The booby prize will be a lifetime's supply of *Gammel Dansk* (a very safe offer, as the life of the lucky recipient is unlikely to continue far beyond the first encounter).

For the record, the final scores in the Viking Challenge were as follows:



A decent veil will be drawn over the membership of the 4 teams – we know who we were!

### Should We Do it Again?

Yes, I think we should, but the timing is tricky. This was much too close to the APL2000 meeting, and the content was too strongly Dyalog-biased as a result. However the basic idea and format was very sound, and I hope Insight will have made a small profit from the venture – certainly they deserved to, having taken the risk and done all the hard work.

In many ways it complements the 'Forests in Finland' meeting so should probably stay in Scandinavia, but maybe should be in late summer rather than early winter? Of course winter does have the advantage that hotels are cheap and it is easy to get a provisional reservation for 25 people which can reduce or increase (within limits) as the bookings roll in. But ... someone has to run it which takes time and energy and organisation. Volunteers welcome if Helene can't be talked into setting up VikAPL-2001.

# Dragging and Dropping with Lists, Trees and Graphs

*tutorial notes by Adrian Smith (adrian@causeway.co.uk)*

## Introduction

These notes are designed to accompany the workshop on dragging and dropping which was given as part of the Causeway seminar at VikAPL - the Dyalog APL workspace is available from [www.causeway.co.uk/freestuff/](http://www.causeway.co.uk/freestuff/) if you want to try this at home.

The examples cover the basic mechanics of setting up a drag/drop editor with a simple illustration using the allocation of products into groups. The 'data-processing' part of the application is intentionally kept simple so that you can see clearly what event-handling is possible and sensible here.

The second example shows how you might set up a graphical editor to modify a production schedule, and review the effect on the inventory curve. Again, this is designed to show the event handling rather than be a 'sensible' application.

## Setting up the Basic Data

For the drag-drop example we have a list of groups, products, their descriptions and group codes (here I am assuming that a product is either 'unallocated' or is in one and only one group):

```
Grp+ 'Group One' 'Group Two' 'Group Three'
Alc+2012 2052 2058 2064 2133 2150 2235 2250 2268 2321 2373 2397
Alcdesc+('Product '),''*Alc
Alcg+0 1 2 3 3 2 2 2 2 0 0
```

You can use anything reasonable here as long as the lengths of the three line-data variables match correctly.

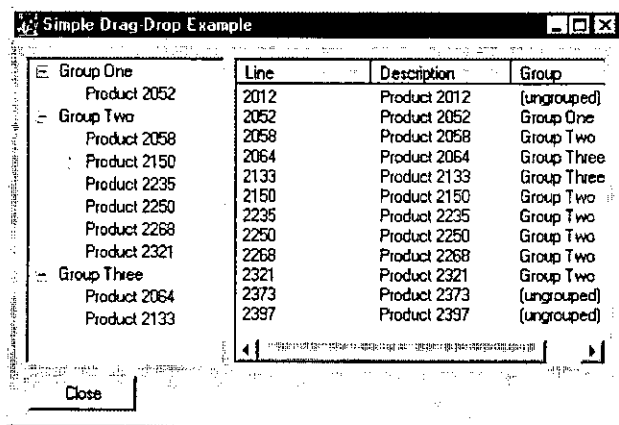
## Sketching the Design

What we would like to do is to allow our user to allocate lines easily to groups, move them from group to group, and de-allocate lines from any group. An obvious visual model for this would be Windows Explorer, with a tree of groups/constituents down the left and a list of products on the right.

The kind of user-actions which seem intuitive would be:

- Dragging a product from the list to the tree to allocate it to the group where it was dropped. Probably we should be able to drag a whole clump of products in one action here.
- Dragging a product code within the tree to move it to another group.
- Dragging a product off the tree into the product list to de-allocate it.

Here is a first-cut design of the application:



... and here are the two simple subfunctions needed to populate the TreeView and ListView respectively:

```

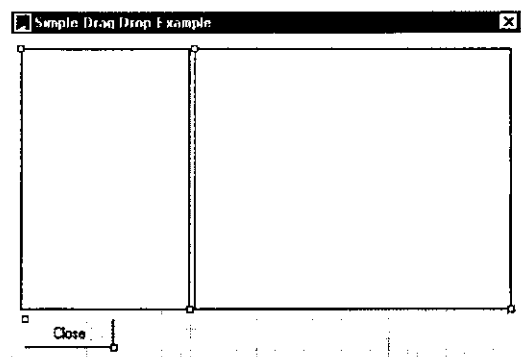
▼tree+MakeTree
[1] A Make tree-structure from base data
[2] A <tree> has items,level,group index,line index
[3] A The tree-control needs cols [:1 2] only
[4]
[5] tree+Δgrp,0,(1pΔgrp),[1.5]0
[6] tree+tree,[1](Δlclg>0)/Δlcldesc,1,Δlclg,[1.5]1pΔlclg
[7] tree+tree[Δtree[:3]];
▼

▼mat+Linedata
[1] A Line-level data as a matrix
[2] A Ungrouped codes have group=0
[3] mat+*Δlcl,Δlcldesc,[1.5]((c='(ungrouped)'),Δgrp)[1+Δlclg]
▼

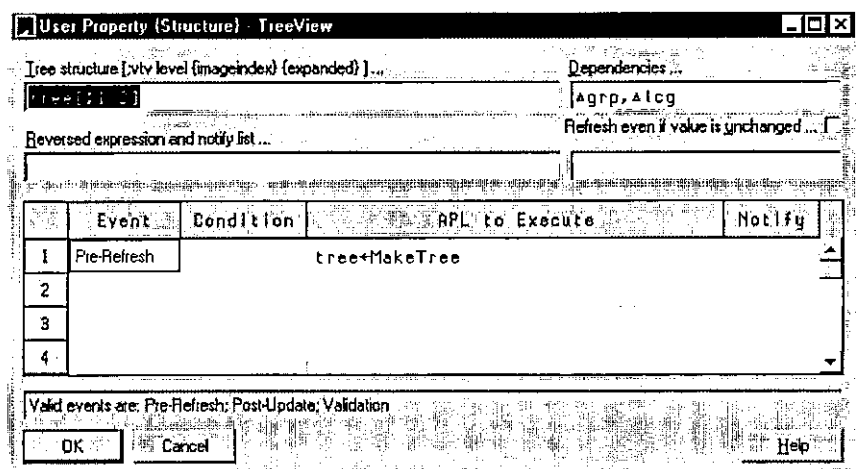
```

Run these from the APL session to make sure the base-data looks reasonable, then we can fire up the Causeway designer to draw the screen and check that it at least shows our starting data correctly:

```
#.Dbx 'dbx.dd'
```



If you set the 'Items' property of the ListView to Linedata, the View to 'Report' (beware - you do need quotes here as this is a runtime property and will be executed) and the ColTitles to something reasonable like 'Line' 'Description' 'Group' you will see the right-hand half of the screen as you want it. For the TreeView, I have been a little sneaky - because I know we will want the tree information for the drag-drop handler I have localised 'tree' on the form and forced it to be recalculated whenever the TreeView is refreshed:



The Gui control should only be passed the first 2 columns (item,depth) here.

You should now find that this 'works' in that you can see the data on screen, but of course you can't drag any items yet!

### Basic Rules of Drag-Drop Processing

Drag-drop editing makes the user's life easy at the expense of making the programmer's life really complicated! You need to:

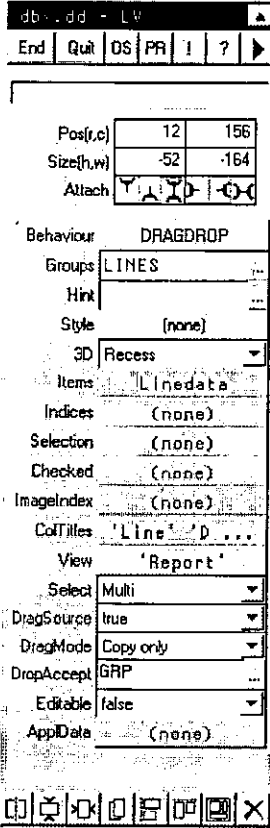
- make sure that the user can only pick up reasonable items from nominated 'drag-source' controls.
- be completely in charge of which controls can act as 'drag-source' and which as 'drag-target' and when something is dropped on a valid target you must be able to check where it came from, as the follow-up action could be very different.
- not all items in the target control may be legitimate targets for the particular item which was picked up! You should only highlight valid target items.
- the user should be able to hit <Escape> at any time to cancel the operation.
- sometimes it is allowed to 'drag-copy' (generally signalled by holding the <Ctrl> key while dragging). If so, you should change the cursor appropriately. If a drag-source is only valid for 'Copy' or 'Move' (but not both) you should show the correct cursor throughout the drag operation. Maybe you want to allow 'right-button drag' operations and put up a menu when the item is dropped.

For our example, we are assuming that the ListView is a drag-source, that it only supports 'Copy' (it shows all the items, not just the ungrouped ones), and that it can target the TreeView but not itself. The TreeView is also a drag-source, but only supports 'Move'; it will accept dropped items from the ListView, or from itself - clearly the two cases are handled very differently.

We can pick up anything from the ListView, but it makes no sense to pick up a group from the TreeView, unless you want to write the code to interpret this as "please re-order the groups" in which case feel free to add this case too. For that matter you could add "please re-order me" handling to the ListView, but for now I will keep it simple as these cases are really just more of the same.

## Naming the Controls and Enabling 'Drag-off'

The first thing we must do is to give our source and target controls sensible names – I have given the ListView the group name 'LINES' and the TreeView the name 'GRP'. Then we need to set the ListView up as a drag-source, for copying only.



You can see that it is now set to act as a drag-source, and that it will accept dropped items only from the 'GRP' control. If you run the dialogue at this point, you will find that you can pick items up, but the cursor shows 'No entry' everywhere except over the ListView itself. It always shows the '+' symbol in the drag-cursor to indicate that we are copying.

I have also left the Select property to the default of 'Multi' as we want to be able to drag a clump of selected items at one go here.

So far so good – the TreeView can be set to something very similar, but its DropAccept entry should be 'LINES,GRP' and its DragMode should be 'Move only'.

Now if you run the dialogue, things will show most of the correct visual behaviours (items highlight as you drag other items over them) but of course nothing changes when you actually let go the mouse. Causeway may help you along the way here, but it can't write your application for you!

We need to add some event-handlers, and these will need to know where the drag-drop originated from, what data was picked up, which item was targeted and perhaps whether we were moving or copying. In this example, we can also pass in the interesting columns of our tree variable as this will be very helpful in determining an item's group or product, given its position in the TreeView control.

## Adding Event Handlers

Rather than describing the message format in detail here, I will just list the code of the two handler functions and let you read the comments. The entries in the event table for the TreeView and ListView should read:

```
DRAGDROP: tree[;3 4] DroponTree w
DRAGDROP: tree[;4] DroponList w
```

The handlers make the necessary change to the global data and 'Notify' the change to Causeway so that the Gui controls are automatically redrawn.

```

▽ tree DroponTree msg;tgt;mc;mb;sel;src;new;linx
[1] A Handle a drop on the tree
[2] A Message gives
[3] A tgt = index of target item
[4] A mc = move/copy flag
[5] A mb = menu required (right-drag)
[6] A sel = incoming data (depends on source control - usually selitems)
[7] A src = group name of source control
[8] A <tree> is the [;group numbers, line index] for each item in the tree
[9] (tgt mc mb sel src)+msg
[10] :Select src
[11] :Case 'LINES' A new line-list data arriving
[12] A Allocate inbound lines to the group that was dropped on
[13] new+tgt>tree[;1] A make new group the same
[14] (sel/Δlcg)+new A Update global data
[15] #.CPro.Notify'Δlcg' A Let the Gui know about it
[16] :Case 'GRP' A move it to a new group within the tree
[17] new+tgt>tree[;1]
[18] sel+{w/1pw}sel A This is the index in the tree
[19] linx+sel>tree[;2] A The index of the original line-item
[20] (linx>Δlcg)+new
[21] #.CPro.Notify'Δlcg' A Let the Gui know about it
[22] :Else
[23] 'unsupported data source - 'src
[24] :End

```

```

▽ trinx DroponList msg;tgt;mc;mb;sel;src;linx
[1] A Handle a drop on the list
[2] A Message gives
[3] A tgt = index of target item
[4] A mc = move/copy flag
[5] A mb = menu required (right-drag)
[6] A sel = incoming data (depends on source control - usually selitems)
[7] A src = group name of source control
[8] A <trinx> is the index in the line-list of each tree item
[9] (tgt mc mb sel src)+msg
[10] :Select src
[11] :Case 'GRP' A Dragged off the tree
[12] A De-allocate dragged item - where we dropped it is unimportant!

```

```

[13] linx+>(sel)/trinx
[14] :If linx>0
[15] (linx>Δlcg)+0 A Update global data
[16] #CPro.Notify'Δlcg' A Let the Gui know about it
[17] :Else A We should avoid this message
[18] 'you can't do that!'
[19] :End
[20] :Else
[21] 'unsupported data source - 'src
[22] :End

```

▼

Try dragging a group-code from the tree over the list, and you should see the "you can't do that" error - not good Windows practice to allow the user to make avoidable errors! Drag a group over the tree and let go and you will crash the handler function for the tree with an INDEX ERROR, which is also not a good idea! As a final refinement let's control the pickup operation from the tree:

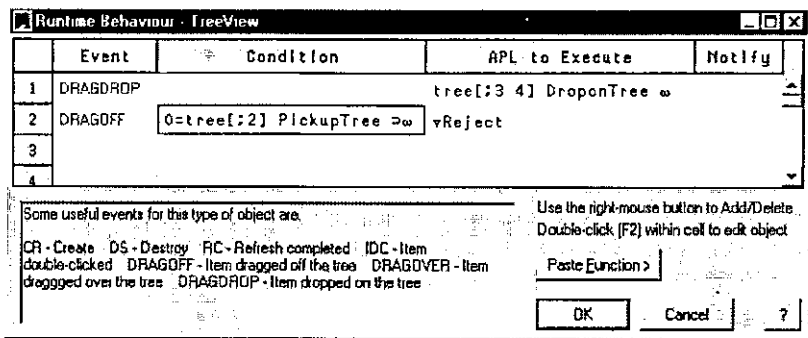
```

vok+lvl PickupTree sel
[1] A We can only pick up level-1 items from the tree!
[2] A If this returns 0 we should reject the pickup from the tree.
[3] ok+>(sel/lvl)>0

```

▼

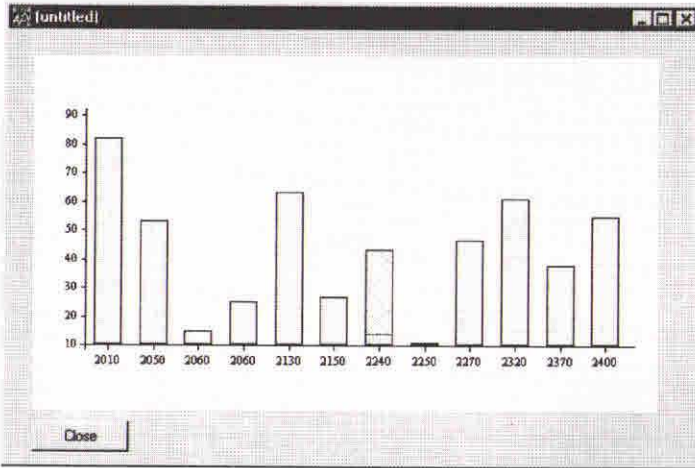
This should be set as the 'DRAGOFF' handler for the tree, but this time it must be set as the 'Condition' - if it fails we will reject the event and the level-0 item will not be picked up:



Now you can test it to destruction, and you should find it very hard to make it do anything wrong. As a bit of light relief, you could add in some extra logic to put up a right-mouse menu when an item is 'right-dropped', or change the data-structure to allow a product to be allocated to more than one group, which is a good excuse to play with the different logic needed for 'Move' and 'Copy' operations.

## Dragging Data with a Barchart

The example shown here is again at the easy end of the scale - it simply allows graphical editing of a single data series using a barchart. A more complex example (developed in conjunction with Mike Baas of Adapta GmbH) is in the workspace - run *DG* to test it and *#.Dbx 'dbx.dg'* to take it apart.



```

VPG-Volbars sz:data
[1] A Make a simple barchart of sales volumes
[2] ch.New 0 0,sz
[3] ch.Name'Volumes'
[4] ch.Set'Xlab'\Delta c
[5] ch.Series'Qty'
[6] ch.Bar \Delta cv
[7] PG+ch.Close
▽

VPFixVol msg
[1] A Update global data to reflect user action
[2] A msg[5 6] give the new (x,y) values for bar index msg[3]
[3] \Delta cv[3=msg]+[0.5+6=msg
[4] #.CPro.Notify'\Delta cv'
▽

```

CDEND FixVol w A Handler for CDEND event on PS

The screen-snap shows the data for Product #2240 being dragged, but you will have to imagine the cursor, or try it out for yourself.

# APL2000 International User's Meeting Orlando, November 2000

*notes and photos by Adrian Smith and Jonathan Manktelow*

## Introduction

This was quite possibly the best APL conference ever. The same was said of last year's APL2000 gathering, and at the end of this one, Eric could be seen fretting over how they could possibly match it next year! I'm sure they will have a good try. The remarkable thing was that it was stuffed with interesting material, and almost none of it came directly from APL 2000 themselves. In fact (in a brilliant piece of stage-management) they kept the best until the very last – a low-key 10 minutes from Bill Rutiser on the future of the interpreter.



**Eric Baelen with Adam Kertesz (who was honoured with the APL2000 award for Outstanding Professional and Technical Achievement)**

The venue was the same as last year, but the Clarion Plaza had tried to fool us by renaming itself the Rosen Plaza in the meantime! Jonathan & I were forewarned and skipped off down the road to the Howard-Johnson 'motel and suites' which was about half the room price for a very much pleasanter environment. We even had a microwave so we could make real tea, and the 15-minute walk up International Drive was surprisingly enjoyable.

Having said this, the actual conference facilities were fine at the Clarion/Rosen and the buffet meals and banquet were up to the same high standard as last year.

### The Social Programme

The two pictures below probably give you a flavour of the evening outing to WonderWorks. Kate Cannon is mastering the JetSki while father Ray has a little local difficulty with an airblast.



**eXtreme APL at the WonderWorks evening**

### The Formal Sessions

These were almost all for the users, by the users. The atmosphere was positive "Hey guys, look what we did with APL ..." but realistic "... but it took us 3 months to get to where we are today when we guessed 3 weeks". There was also some solid training material from the APL2000 regulars, and a genuine 'newbies' day which generated some mixed responses, but at least one convert to the cause. *Vector* is intending to publish several of the more formal papers in full, so I will keep my notes to the plenary sessions with a few stray comments on the others.

### Greg on KREG

KREG is a leading vendor of Healthcare Decision-support tools, and Greg Ferguson had brought along 3 of his development team to enjoy the conference. He gave a fascinating, and to a European quite frightening, talk on the application of decision-support tools in the US healthcare industry. Those of us lucky enough

to have a state-funded hospital system should realise how fortunate we are, and breath a huge sigh of relief that the last Tory administration did not have the time to send it any further down the path to privatisation.

KREG is a tool that is primarily sold by word of mouth "we have never taken the wine and dine approach to marketing" and they reckon that an installation in a large hospital will often return \$1M per year to the customer. Greg noted that hospital information systems are *extremely* poor - people will come into a room and argue about how many patients were treated yesterday - "2 out of 3 hospitals cannot tell you what it costs to treat a patient".

Greg gave as an example a hospital which ran its delivery room as a 'loss leader' in the mistaken belief that the mothers (or children) would remember the experience and return to the same hospital for treatment later in life. Careful examination of the data showed that this simply did not happen, so they closed it, which was "kind of tough for a Catholic hospital".

The KREG system is based on a 10 year investment in APL code and the primary algorithm has not been touched in that time - "it takes 3 days to work out what the code is doing". When they were facing the Windows conversion issue they looked at Dyadic, but (in common with many big systems) they could not read and write the STSC files reliably, so they looked at what SoftMed had done and decided to stay with APL2000 and its (at that time) rather limited Windows functionality. The conversion was begun in spring 1996, and proceeded one module at a time, leaving the data completely alone. It was necessary to overcome the resistance of the Sales and Marketing people (who did not see the need to charge for the update) and of the programmers (who found it hard to accept the need to change from the DOS way of working). In the end there was very little re-use of old interface code "we didn't want to slap a Gui on a DOS system - those look kinda dumb".

#### **Brian Chizever - 101 new tricks to play with the interface**

I really can't add much to Brian's published material here, except to say that a lot of the 'fun' stuff still only works under Windows 2000 so is of academic interest only for a few years yet. Watch the APL2000 website for developments and updates.

#### **Patrick Parks - a new notation for activeX array properties**

As Patrick put it "This stuff is newly hatched and a little unstable right now ...". The ultimate goal is to make it much more possible to take VB documentation and

paste it more or less directly into an APL function as a way of manipulating objects, so *workbooks.count* replaces some funny deeply parenthesised stuff involving *⌈WI* and suddenly we can see our code again. Hopefully we will also see a rapid convergence of APL+Win and Dyalog APL, as they are both heading for the same (VB compatible) goal here.

Patrick did promise Vector that just as soon as the model stabilises, he will send us a full write-up of the new syntax, and from what we saw in his session it is not too far away. Of course this only solves half of the problem – it allows us to access the outside world from APL – the other half is to make it equally easy for the outside world to access APL. This was the bit that Bill Rutiser left until last!

### Rich Krafchin and Barbara on Anemotion

This is a hard paper to write up – here is a bunch of APL enthusiasts with a real flair for marketing doing something that is technologically rather clever, possibly very dangerous, and potentially anti-social. OK, if they don't do it, someone else will, but this is not an argument I can feel comfortable with!

What they have realised is that Microsoft have buried the entire HTML engine (complete with images, sounds and scripting) in that innocent little window we all thought was just an email viewer. This allows you to send emails with a touch of formatting (probably the original intention) but also to include links to pictures and sound clips. The clever bit is that the pictures and sounds can be included in the message as in-line attachments (this is called multi-part MIME) and that the HTML can include JScript or VBScript sections to make the pictures move around and the sounds play at appropriate times.

What the user receives is a single mail message which looks remarkably innocent until you open it (OK, it does have a paperclip, but that is all the warning you get). On being double-clicked it will do pretty well anything you can imagine that involves moving images and other multi-media effects. With careful use of GIF files and timers you can cook up some remarkably snazzy marketing presentations which can be assembled and bulk-emailed to the unsuspecting world. To quote the presenter "This is the most extraordinarily powerful marketing tool that we are now selling to our clients ..."

As with any development, the idea came quickly, the technical discussion went "How long? ... 2-3 weeks .... this is terrific!" Then they learned how a developer (even the APL kind) reckons time. The demo they showed gave us some idea why it had taken them a good part of the year to get from concept to product – the editor for the "Anemotion developer" owes a lot to Macromedia Flash, with

timelines and previews and so on. This is just plain hard Gui stuff, whether in APL or any other language.

So why am I worried? Firstly I am bothered by their view that 300K is an acceptable size for something that will go out as bulk email. With the old-style WinCIM interface to Compuserve I can discard junk mails without ever downloading them; with Outlook and a POP3 mail server I just hit Send+Receive and watch the phone bill clock up. 300K at peak rate costs me rather more than 20p and it will very likely get in the way of some other mails that I actually want to read. If this idea really takes off, it could swamp a lot of mail servers and really get up the noses of us poor country boys with slow modems.

The second concern is that I wonder just how easy it would be to receive one of their mails, patch a virus into it, and send it on its way. It would still appear to come from the original source, it would still play its merry tune and so on. If this did some severe damage, I wonder who would sue whom?

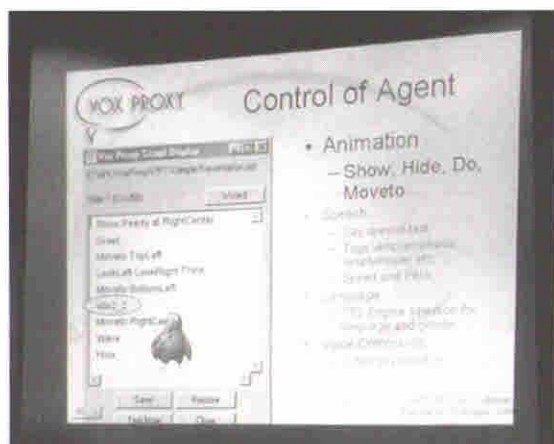
Finally, I wonder why they are using APL? Their software is simply an assembly tool for some graphics and scripts, co-ordinated with a timeline. I don't see where the array-processing comes in, or any real added value from the analytical power of the interpreter. Plain old Gui applications might be better done in a plain old Gui tool like Delphi? If the only reason they used APL is because they are APLers, I am worried.

### The Atkins Family on VoxProxy

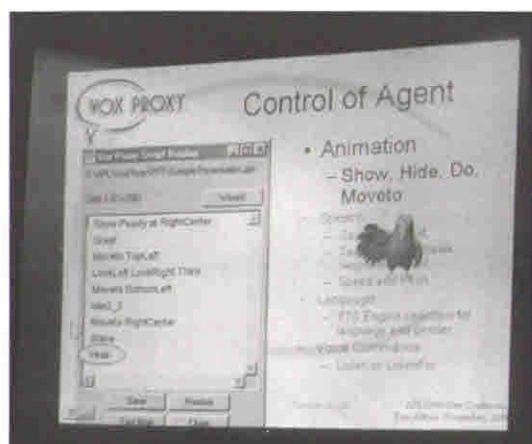
Now this one was superficially similar, but very different in that it really did leverage APL, and that it was both harmless and useful. I thought it was the neatest idea I had seen for years!

This time the target environment is Microsoft PowerPoint, which is great for dumb presentations, but has totally missed out on scripting and animations. Add to this a selection of Microsoft Agents (remember Merlin and Robbie the Robot) with some suitable scripting and - Wow! The entire application runs from a DLL that PowerPoint recognises as an add-in (it shows up on the menubar) and "we're dealing with PowerPoint users here" it must major on ease of use and have excellent defaults. Where is the APL power in this one? Simply that they are using the interpreter directly to run scripts - control structures and all - which are trivially massaged then `⎕FX`ed, wrapped around with some interrupt checking and executed. Temporary variables can be assigned (for example `&fluff` is mapped to `⏟fluff` before execution) and of course macros are just scripts which run other scripts.

Now Tom played his master-stroke - he handed over the entire presentation to Peedy (a very friendly green parrot) who was controlled by a *VoxProxy* script, which itself illustrated all the capabilities of the scripting tool!



I hope you can see enough of the script (and the parrot) to get a flavour of what was going on. Here is the same frame but just after Peedy had executed the 'Wave' command in the script ....



This just gives a small flavour of the power of the idea - agents can request input from the user (possibly even with limited voice recognition) so a collection of such scripts can make a really nice multi-media teaching tool, with multiple-choice questions and selection of material depending on the student's performance.

I suppose I must be one of the few APLers in the world who gave up PowerPoint a long time ago (I think the last time I used it was in 1993), but this almost makes me want to take it up again.

VoxProxy should be available in January 2001, and the guide price was \$200. What's more they really seem to have the business model sorted - keep it low-profile, use targeted marketing where available (the PowerPoint User's magazine), sell it off the website, stay small, get rich slowly and in a sensible controlled way. Every home should have one - see [www.voxproxy.com](http://www.voxproxy.com) and buy while stocks last!

### **Andy Weiss on zero-footprint Blackjack**

It's amazing what you can do in a training exercise! Andy is actually studying stage lighting at college, but his (APL-oriented) father suggested he might like to try computing for a while, so he signed up with APL2000 for some web-development work and this is what came out of it.

He started with the advantage of knowing almost nothing about Windows programming, so he just used the capabilities of the browser (and I suppose an inherited understanding of how timesharing works) to make a very effective multi-player Blackjack.

I think the crucial point on layout was to go with the 'flow' conventions that the browser expects - avoid all attempts at absolute positioning and things just re-organise themselves on the fly, particularly if you use tables sensibly. Any transitions and simple configuration changes are handled entirely locally by the DHTML code, and he uses a timer to make regular round-trips to the server to check on what the other players in the game have done. This necessity to 'poll' back to the server is always regarded as one of the major limitations of HTML in building a genuine multi-user site, but Andy had simply lived with it and made something that appeared to work very well indeed. The confidence of youth, and all that!

### **Bill Rutiser on eXtreme APL**

This time, I can't resist quoting some classic Rutiser quotes:

"I want you to imagine the cartoon on the first slide"

"At least I'm destroying my wife's computer"

"Is it relevant that I can't see anything here ..."

"Do not make any decisions about eXtreme Programming based on this talk – read Kent's book"

"No design discussion between programmers should last longer than 15 minutes without making an experiment"

"If you can't fix it, don't break it!"

Bill's talk was effectively a review of a series of 3 books on "eXtreme Programming" which are apparently the talk of the programming community at the moment. I followed his recommendation and have read 2 of the 3 so far – they are excellent little books and express a lot of what APL developers always were famous for in the past. I think I should dust off the manuscript for "APL – a Design Handbook", retitile it "eXtreme Programming with APL" and retire gracefully to my mansion in the hills! Seriously – I wrote most of this stuff in 1982 without realising that it was anything special. Maybe the 'mainstream' environments have finally caught up sufficiently close to APL that the same techniques are now generally applicable? Should we be worried or just smug?

Anyway, the author is Kent Beck and the York branch of Borders had all 3 on the shelves, so I'm sure you can find them. Start with "eXtreme Programming Explained" and look out for a decent review of all three in the next Vector.

### David O'Neill on Stats in British Columbia

This was a very enjoyable and entertaining potted history of the demographics department in the province of British Columbia, where about 1/3 of the staff (around 30 people) do some APL in the production of demographic trends and classic GIS summaries.

"The Census data-extraction system – every time we had a census we re-wrote the system!"

My notes make sense to me, but it would be much better if David could find the time to write this up for publication himself, as it has a lot of valuable lessons for anyone trying to maintain an APL operation in the face of political ignorance and the march of technology.

### The Closing Plenary

These get more like religious revival meetings every year! There is no doubt that the APL2000 community feels itself to be moving forward again, that the internet paradigm gets more like timesharing as every day goes by, and that APLers understand how to do timesharing! Suddenly you can build websites that do

interesting analytical stuff, that run at a decent speed and where you can fix things when they break. Yes, there is a big learning curve for us all to climb, but the attendance at Fred Waid's JavaScript day (see later) was a sign that a fair number of us old dogs are actually getting motivated to learn some of the new tricks.

The format was quite neatly stage-managed, with a warm-up from Eric followed by each developer giving a very brief presentation of what they are working on today, and what they intend to be in APL+Win Vn4. The Linux interpreter achieved a 6-space prompt and successfully executed 112 to general acclaim. In fact we were all given a copy of this to take home - mine is currently taking up a very small part of the memory card in the digital camera!

The final talk was a 5-minute summary from Bill Rutiser on his work with the core of the interpreter. I did my best to get verbatim notes here, as this was probably the most important session of the entire conference:

"The APL interpreter that we're using today we started writing in 1982. 1Mb memory, 20Mb Hard drive, 6 developers using terminals. Some of the assumptions built into the interpreter are no longer valid.

One of the things people have wanted is to run as an in-process server - the internal architecture is not suitable for that.

What I have been working on for the past year is a re-factoring of the interpreter so it can run as a COM server in IE. It's like re-wiring a 20-year old office building. The former owner re-organised many times and what infrastructure we had got lost! I am slowly rebuilding the infrastructure and when I am done it will be a wonderful thing.

Currently I have APL/360 with nested arrays and text in /out as a COM object that can be used as an invisible engine."

Question: "Will it become the mainline interpreter?"

"Not really known. I am putting code back into the mainline codebase as convenient. It's to everybody's advantage to combine them as soon as possible, and the main APL development machine could be based around this COM object."

"Please don't take this as an announcement!"

As an observer, my instinct is to trust Bill on this one, and I think he is being given the time and thinking space to get it right. We should end up with a seriously good array engine for the 21st century, which will not need another rewiring for a good many years, if ever.

## Tutorial Day - Fred Waid on JavaScript

As you would expect, this drew a good crowd of serious developers. The message was simple - over the next 2 years Microsoft will have completed the remote User Interface so you can run the entire visible part of your application in the browser. They are getting there on time, this time around - IE5.5 is the crucial update for developers, and it has been quietly slipped out on cover disks to make sure enough of the users will already have it when the time comes.

Essentially there will be *WinForms* (which replace the current Windows API) and *WebForms* - you will need to code for both if you want to deploy the same application locally and remotely. The current support for *WTI* is around 3000 lines of C code which will obviously need rewriting - and maybe there will be enforced changes in the APL code that calls it. Either way, you should focus on building a CALC engine and separate the Gui code into one or the other.

Q: "How close is JScript to JavaScript?"

A: "They are the same thing - the only difference is a lawsuit!"

Java may already have run its course - Microsoft are shipping an old JVM with IE5.5 in an intentional move to stabilise it. The chance of getting a major Java project approved is now at the "snowball in hell" level.

Fred noted that Visual InterDev is just a very thin wrapper on DHTML, and that most of what you need is supplied with Andy Weiss's BlackJack workspace, which could save you \$100 if you don't mind doing a little tidying up. However for the rest of the talk he used InterDev as the most convenient tool to show what was going on.



Fred in expository mode during a break

Some useful analogies helped to make the APL crowd at home. Scripts go in the <head> section and are your functions and global variables. When the browser has finished parsing the script, it runs the 'onload' event in the <body> section which is the same as your `⌈LX`. Think of the semi-colon as a diamond, but if you use these at all you must use them everywhere! Localisation is by script, or you can have globals which are accessible to all scripts. Yes, there are nested arrays ...

```
var t1 = new Array()
t1[0] = 's1'
t1[100] = 5
```

But note that `⌈IO` is set at zero, and that all those elements in the middle are nulls, which is a specific data type that you can test for. Unlike APL, the array length is automatically extended, and you also have to get used to many things we think of as functions as being intrinsic methods of the object:

```
Alert "length is " + t1.length
length is 101
```

There is a huge repository of idioms on the internet - just try asking [www.google.com](http://www.google.com) a question like 'JavaScript + "delete trailing blanks"' if you need a code fragment for your own project. Note that there are actually very few reserved words, because of the redefinition of functions as properties, so it is quite legal to say:

```
length = t1.length
```

The frustrating thing is that there *is* an immediate execution mode, but it is regarded as part of the debugger, so the only way to get to it is to make your code blow up! Shades of the APL habit of introducing lines of dominos as the quick and easy way to stop a function.

Fred talked briefly about the design issue of where you split the application between the client and the server - OK you put the engine in the server but where do you draw the line? Changing a text field to ALL CAPS on input clearly goes in the client (even 3270 did that) and probably numeric verification and other basic validation, but what about simple arithmetic? Do you work out your tax rates in the browser - how about the discount options which may depend on a prior purchase? Probably not! Would you run something like Excel solver in the browser - not on today's hardware with interpretive JavaScript!

However there is one up-coming technology which makes a very significant difference here, it is called HTC and it is one of these little invisible changes which has slipped in with IE5.5.

Currently, when you submit a web form, the server must replace the entire form with new content when it returns it. OK, you can use frames, but in the future we will be able to have the server send back an HTC page which is an HTML stream with *no displayable components* - what's more the arrival of this is registered as an event in the currently loaded JavaScript which can extract values from it. Suddenly we can have a page which round-trips to the server in the background, and updates fields in the browser having transmitted only the minimum few bytes needed for this particular job. This actually pushes the 'engine-room' boundary some way back to the server - why code all this stuff about tax rates down into JavaScript (and have a substantial number of bytes of program code uploaded with the page) when we can make a trivial round-trip with a few dozen bytes and have it run in the server by the program we already have? If the rules change, we fix it in one place, and if we need to integrate it with our customer database (which should know which state/country the customer is coming from) we can do it all in one simple in-line operation!

The message is becoming clearer by the day, and the internet is getting to be much more like the mainframe of old, just substitute your 'browser' for the old 3270 terminal and 'webforms' for AP124 and things start to make sense again. Fred went on to show a lot more stuff with XML namespaces and COM objects used as embedded display components, but I will stop where my notes stop and leave him to continue the saga next year when the technology will have advanced one more notch and a decent fraction of the world population of browsers may actually be supporting this stuff for real.



A gratuitous commercial for the HoJo off International Drive

## A Grid Object for APL+Win

by Eric Lescasse ([eric@lescasse.com](mailto:eric@lescasse.com))

Every APL developer knows that it is very difficult to find an application where using a Grid object of some kind is not required. Among all the applications I have been writing for the last 10 years, I don't remember any of them not using a Grid.

Before APL+Win was out, we had to write our own Grid in APL+PC or APL+Dos and though a somewhat difficult and time-consuming task, this was still feasible. So, many APL+ users developed their own Grid tools. With Windows, developing a Grid object becomes a very difficult and arduous task: it proves to be just impossible for most APL developers.

Dyalog APL has had a Grid object for several years now, but APL+Win was still lacking one. So APL+Win developers have been using third party Grid tools, starting with the Formula One VBX years ago, and more recently various ActiveX Grids such as Formula One from TideStone, Spread from FarPoint, FlexGrid from VideoSoft, etc.

These ActiveX Grids are fine: they are the tools that Visual Basic and C application developers frequently use, and I have used several of them with complete success in various Dyalog APL and APL+Win applications. However none of them are designed to be used in an array oriented language like APL and therefore they are sometimes a bit cumbersome to work with. In the worst case, you have to deal with individual cells, not with matrices, and that makes them a little slow for some operations.

So finally APL2000 has decided to add a new Grid object to APL+Win and to make it from start to finish an array-aware Grid that is simple to use and very efficient.

An Alpha test of this Grid object was presented by Patrick Parks at the APL2000 User's Conference in November in Orlando (USA), and I have had a chance to be able to use this Grid object extensively since its early development stages in spring 2000.

Although the APL2000 Conference Manual clearly states *"This is an ALPHA TEST version of the control and should not yet be deployed in your production applications. All features discussed here are subject to change"*, I have indeed been using it in 2 large applications which are being developed, at my own risk.

## A Quick Overview of the APL+Win Grid

First let's have a look at it ...

|     | C1 | C2                                        | C3     | C4     | C5     | C6                                                                                                     | Locale |
|-----|----|-------------------------------------------|--------|--------|--------|--------------------------------------------------------------------------------------------------------|--------|
| R1  |    | (1,2)                                     | (1,3)  | (1,4)  | (1,5)  | (1,6)                                                                                                  | 1027   |
| R2  |    | (2,2)                                     | (2,3)  | (2,4)  | (2,5)  | 17/07/2000                                                                                             | 1030   |
| R3  |    | (3,2)                                     | (3,3)  | (3,4)  | (3,5)  | 18:00:00                                                                                               | 1031   |
| R4  |    | (4,2)                                     | (4,3)  | (4,4)  | (4,5)  | 18/07/2000                                                                                             | 1033   |
| R5  |    | (5,2)                                     | (5,3)  | (5,4)  | (5,5)  | This is a tracker window... it displays to the right of the cell specified by onCellMouseEvent handler | 1034   |
| R6  |    | (6,2)                                     | (6,3)  | (6,4)  | (6,5)  |                                                                                                        | 1035   |
| R7  |    | <none>                                    | (7,3)  | (7,4)  | (7,5)  |                                                                                                        | 1036   |
| R8  |    | <input checked="" type="checkbox"/> Check | (8,3)  | (8,4)  | (8,5)  |                                                                                                        | 1039   |
| R9  |    | >                                         | (9,3)  | (9,4)  | (9,5)  |                                                                                                        | 1040   |
| R10 |    | <                                         | (10,3) | (10,4) | (10,5) |                                                                                                        | 1043   |
| R11 |    | >>                                        | (11,3) | (11,4) | (11,5) |                                                                                                        | 1044   |
| R12 |    | <<                                        |        |        |        |                                                                                                        | 1046   |
| R13 |    | Button                                    |        |        |        |                                                                                                        | 1053   |
| R14 |    | (14,2)                                    |        |        |        | (14,5)                                                                                                 | 1057   |
| R15 |    | (15,2)                                    | (15,3) | (15,4) | (15,5) | (15,6)                                                                                                 | 1069   |

This shows what is already available.

- Populating the grid with APL nested arrays
- Colors, Borders, Alignments
- Hierarchical multi-line/columns headings
- Formatting
- Various data types (text, number, Boolean, date, currency)
- Merge cells
- Cell objects (button, check box, dropdown list, arrows)
- Multiple sheets
- Selections
- Tracking Window
- Support for locales
- Fixed rows and/or columns
- Formatting patterns for cells
- Images in cell corners
- Support for XML

The Grid is currently delivered as a single DLL (APLGRID.DLL) and must be registered before it can be used, with:

```
Regsvr32 c:\aplwin36\aplgrid.dll
```

Note that this DLL may be installed in any directory on your hard drive, as long as it is properly registered. The Grid can be used with versions of APL+Win other than the latest 3.6.09 (or the 3.7 Beta). I have tried it with success under APL+Win 3.5 and even 3.0.

To uninstall the Grid simply enter:

```
Regsvr32 -u c:\aplwin36\aplgrid.dll
```

provided you had installed the grid in c:\aplwin36

Note that the APLGRID.DLL uses other Windows system DLLs which must be present on your machine:

```
ADVAPI32.DLL
COMCTL32.DLL
GDI32.DLL
KERNEL32.DLL
MFC42.DLL
MSVCRT.DLL
MSVCP60.DLL
OLE32.DLL
OLEAUT32.DLL
OLEPRO32.DLL
RPCRT4.DLL
USER32.DLL
```

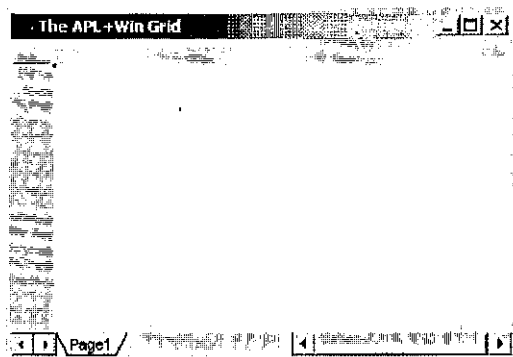
Under a 32-bit Windows operating system you should find all these DLLs under your Windows System directory, except maybe MSVCP60.DLL.

## Exploring the Grid

### Creating a Grid

Once APLGRID.DLL has been installed, start APL+Win and you're ready to use the grid.

```
Ⓜself+ff'Ⓜi>Create' 'Form'('caption' 'The APL+Win Grid')
Ⓜself+ff.ss'Ⓜi>Create' 'APL.Grid'('where'o 0,'ff'Ⓜi'size')
```



By default the grid appears with 1 page, no rows and no columns.

Even in its Alpha Test stage, the Grid already has a large number of properties, methods and events hooked in:

```

p"wi" 'properties' 'methods' 'events'
100 27 38

70 TELPRINT(aaa[;1]='x')+aaa->wi'properties'
xActiveCell xFixedRows xSelection
xAlign xFontName xSelectionCol
xArrayObjects xFontSize xSelectionCols
xAutoEditStart xFontStyle xSelectionCount
xBorderStyle xFormat xSelectionMode
xCellType xGuidelines xSelectionRow
xCellTypeEx xHeadCols xSelectionRows
xCol xHeadRows xStayInSelection
xColSize xImage xTabWidth
xColorBack xImageFile xText
xColorGrid xImageSize xTrackingWindow
xColorSelBack xLocale xValue
xColorSelText xNumber xValueDigits
xColorText xPage xValueEx
xCols xPageName xValueType
xConversionErrorValue xPages xXML
xCurrency xRow xXMLChanges
xDate xRowSize xXMLPage
xFixedCols xRows xXMLRange

70 TELPRINT(aaa[;1]='X')+aaa->wi'methods'
XAddSelection XJoin XSetSelection
XEditCancel XMoveTrackingWindow XWhereIs
XEditStart Xredraw

```

```

70 TELPRINT(aaa[;1]='X')#aaa->[]wi'events'
XCanColSize XCellMouseLeave XEditStart XRowSized
XCanRowSize XCellMouseMove XEditVerify XRowSizing
XCellChange XCellMouseUp XKeyMove XSelectionChange
XCellClick XColSized XPageChanged XViewChange
XCellMouseDown XColSizing XPageChanging XPageOpen
XCellMouseEnter XEditEnd XPageOpen

```

The syntax for a given property, method or event may be obtained using a question mark:

```

[]wi'?Text'
xText : property Text
Ref: Variant <- xText(Variant Rows, Variant Cols)
Set: xText(Variant Rows, Variant Cols) <- Variant

```

### Properties and methods

Rows and columns are added with the Rows and Cols properties:

```

[]wi'Rows'100
[]wi'Cols'20

```

It is also possible to add more sheets:

```

[]wi'Pages'3

```

Now let's populate the grid with some APL data:

```

data+3 3p'APL' 15000 36760 'C' 4950 36765 'Visual Basic'
16000 36768 'Delphi' 2950 36810

```

```

data
APL 15000 36760
C 4950 36765
Visual Basic 16000 36768
Delphi 2950 36810

```

```

[]wi'Text'1 1 data
[]wi'XRedraw'

```

| APL          | 15000 | 36760 |
|--------------|-------|-------|
| C            | 4950  | 36765 |
| Visual Basic | 16000 | 36768 |
| Delphi       | 2950  | 36810 |

Note that the whole nested array is installed in the Grid with one matrix operation with the Text property. When the array is contiguous only the starting row and column is required. You may however specify the row and column numbers for each row and column of the array. This allows us to do things such as installing data on alternate lines:

```
Ⓜwi'Text'(1 3 5 7)(13)data
Ⓜwi'XRedraw'
```

| APL          | 15000 | 36760 |
|--------------|-------|-------|
| C            | 4950  | 36765 |
| Visual Basic | 16000 | 36768 |
| Delphi       | 2950  | 36810 |

Also note that the XRedraw operation is necessary to display the data set with the Text property. The principle is that you may want to make several changes to the Grid, before you want to display all of them at once in one XRedraw operation. This not only reduces any flickering to a minimum but also speeds up things considerably.

Example:

```

wi'Text'(14)(13)data
wi'ColorBack'(1 3)(120)(256 192 192)
wi'ColSize'(1 2 3)(120 60 80)
wi'Align'(14)(2)2
wi'XRedraw'

```

| APL          | 15000 | 36760 |
|--------------|-------|-------|
| C            | 4950  | 36765 |
| Visual Basic | 16000 | 36768 |
| Delphi       | 2950  | 36810 |

To add data to another page, be sure the grid has several pages (with the Pages property), shift to another page with the Page property, add the necessary rows and columns, and use the Text property again:

```

wi'Pages'2
wi'Page'2
wi'Rows'(1>pdata)
wi'Cols'(2>pdata)
'ff.ss'wi'Text'1 1 data
'ff.ss'wi'Redraw'

```

Note that although you installed text in each cell with the Text property, cell content may be read as numbers using the Number property:

```

dr''+,wi'Text'(14)3
36760 36765 36768 36810
82 82 82 82
dr''+,wi'Number'(14)3
36760 36765 36768 36810
323 323 323 323

```

This is a very useful feature.

Row and column headings are easily added using negative row and column numbers:

```

wi'HeadRows'2
wi'HeadCols'2
wi'ColSize'(-2 -1)(60) a note: sizes are expressed in
pixels
wi'Join'2 -2 3 1
wi'Text'(-14)(-2 -1)('Array' 'Cell' ''
'',[1.5]4pc'Oriented')
wi'Join'2 2 1 2
wi'Text'(-2 -1)(13)('Character' 'Numeric'
'',[.5]3pc'Cells')
wi'XRedraw'

```

The following small utility function can help immediately see the result of a command, without having to type `wi'XRedraw'`:

```

v ss A
[1] wi'XRedraw'
v

```

Example:

```

ss wi'Join'2 2 1 2
ss wi'Text'(-2 -1)(13)('Character' 'Numeric'
'',[.5]3pc'Cells')

```

|       |          | Character    | Numeric |       |
|-------|----------|--------------|---------|-------|
|       |          | Cells        | Cells   | Cells |
| Array | Oriented | APL          | 15000   | 36760 |
|       | Oriented | C            | 4950    | 36765 |
| Cell  | Oriented | Visual Basic | 16000   | 36768 |
|       | Oriented | Delphi       | 2950    | 36810 |

Page 1

The Join method above merges 1 line, starting at line -2 and merges 2 columns starting at column 2 on this line, hence the "Numeric" heading spanning columns 2 and 3.

Rows and columns may of course be restricted to the minimum necessary to hold all the data, and colour can be set for the background of the grid:

```

wi'Rows'4
wi'Cols'3
wi'ColorGrid'0

```

|       |          | Character    | Numeric     |
|-------|----------|--------------|-------------|
|       |          | Cells        | Cells       |
| Array | Oriented | APL          | 15000 36760 |
|       | Oriented | C            | 4950 36765  |
| Cell  | Oriented | Visual Basic | 16000 36768 |
|       | Oriented | Delphi       | 2950 36810  |

We may convert the numbers in column 3 to dates:

```
ss wi'Date'(14)3(wi'Number'(14)3)
```

Dates are automatically formatted according to your Windows International Settings unless you specify another locale (see later).

|       |          | Character    | Numeric          |
|-------|----------|--------------|------------------|
|       |          | Cells        | Cells            |
| Array | Oriented | APL          | 15000 22/08/2000 |
|       | Oriented | C            | 4950 27/08/2000  |
| Cell  | Oriented | Visual Basic | 16000 30/08/2000 |
|       | Oriented | Delphi       | 2950 11/10/2000  |

```
ss wi'Rows'10
```

Now let's define cell 5 1 as a Combo Box:

```

ss wi'CellType'5 1 1
ss wi'Text'5 1 '(none)'
ss wi'ValueEx'5
1('(none)',⎕tclf,'APL',⎕tclf,'C',⎕tclf,'Visual
Basic',⎕tclf,'Delphi',⎕tclf)

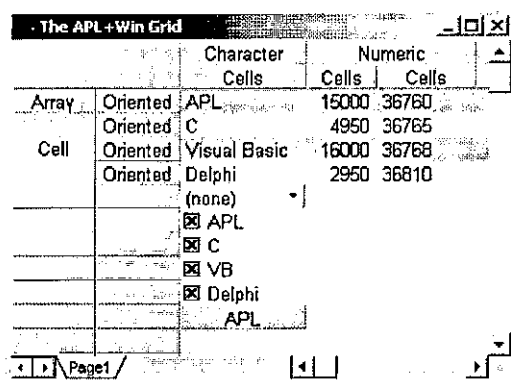
```

and cell 6 1 to 9 1 as Check Boxes

```
ss [wi'CellType'7 1 2 0 ss [wi'ValueEx'6 1'APL'
ss [wi'CellType'7 1 2 0 ss [wi'ValueEx'7 1'C'
ss [wi'CellType'8 1 2 0 ss [wi'ValueEx'8 1'VB'
ss [wi'CellType'9 1 2 0 ss [wi'ValueEx'9 1'Delphi'
ss [wi'Align'(6 7 8 9)1 1 a note: 1=Left Align
```

and cell 10 1 as a Button:

```
[wi'CellType'10 1 4
[wi'ColorBack'10 1(256192 192 192)
[wi'Text'10 1'APL'
ss [wi'Align'10 1 4 a note: 4=Center
```



Note that it would make things even easier to define another small utility called ee as follows:

```
v A←A ee B
[1] A←[wi(1+B),A,1+B
v
```

and the last 4 instructions could be written:

```
ss(<10 1)ee''('CellType'4)('ColorBack'(256192 192))('Text'
'APL')('Align'4)
```

Check boxes could be checked by program with such instructions as:

```
[wi'Value'(6 7 8 9)1 1
```

and unchecked with:

```
[wi'Value'(6 7 8 9)1 0
```

You may also display images in the Grid using the **image** property. Images are specified as a rank 3 cube where the first coordinate represents the placement axis and the rows and columns represent the rows and cols of the grid. First plane is TopLEFT, Second is TopRIGHT, Third is BottomLEFT, Fourth is BottomRIGHT.

One use of the Image property is to duplicate the comment symbol from Excel, but because you have more control over images and placement, they have lots of other uses.

```
images←1 2 3 4 0.+ 4×3 2p0 1 2 3 4 5
Ⓜw'Image' (14) (5 7 9) (3 5) images
Ⓜw'ImageFile' 'd:\aplwin\grid66.bmp'
```

| The APL+Win Grid |          | Character                                  | Numeric |       |
|------------------|----------|--------------------------------------------|---------|-------|
|                  |          | Cells                                      | Cells   | Cells |
| Array            | Oriented | APL                                        | 15000   | 36780 |
|                  | Oriented | C                                          | 4950    | 36765 |
| Cell             | Oriented | Visual Basic                               | 18000   | 36768 |
|                  | Oriented | Delphi                                     | 2950    | 36810 |
|                  |          | (none)                                     |         |       |
|                  |          | <input checked="" type="checkbox"/> APL    |         |       |
|                  |          | <input checked="" type="checkbox"/> C      |         |       |
|                  |          | <input checked="" type="checkbox"/> VB     |         |       |
|                  |          | <input checked="" type="checkbox"/> Delphi |         |       |
|                  |          | APL                                        |         |       |

One nice feature of the future APL Grid is that it understands "locales", i.e. whatever country you live in, you'll be able to display numbers and dates in the local standard format for your country.

In order to see this, let's create a simple Grid example:

```
Ⓜw'Rows'100
Ⓜw'Cols'20
loc←10 1p1027 1030 1031 1033 1034 1035 1036 1039 1040 1043
Ⓜw'value'1(1 2)('Locale' 'Number' 'Date' 'Country')
Ⓜw'locale'(110)(2 3)(loc,loc)
Ⓜw'number'(110)1 loc
Ⓜw'number'(110)2(10 1p1234.56)
Ⓜw'date'(110)3 36793 0 Ⓜw'align'(110)3 2
Ⓜw'text'(110)4(10 1p'Catalan' 'Danish' 'German' 'English(US)' 'Spain'
'Finland' 'France' 'Iceland' 'Italy' 'Netherlands')
Ⓜw'XRedraw'
```

| Locale | Number  | Date       | Country     |
|--------|---------|------------|-------------|
| 1027   | 1234.56 | 26/07/2000 | Catalan     |
| 1030   | 1234.56 | 26-07-2000 | Danish      |
| 1031   | 1234.56 | 26.07.2000 | German      |
| 1033   | 1234.56 | 7/26/2000  | English(US) |
| 1034   | 1234.56 | 26/07/2000 | Spain       |
| 1035   | 1234.56 | 26.7.2000  | Finland     |
| 1036   | 1234.56 | 26/07/2000 | France      |
| 1039   | 1234.56 | 26.7.2000  | Iceland     |
| 1040   | 1234.56 | 26/07/2000 | Italy       |
| 1043   | 1234.56 | 26-7-2000  | Netherlands |

Notice how dates are differently formatted depending on the countries and notice that European use a comma as the decimal separator while USA uses a dot.

There's also a Tracking Window feature which automatically displays a window as your mouse cursor enters some cells.

I'll stop here describing the Grid methods and properties. A complete book would be necessary to describe and show them all in detail, even with the Alpha Test version of the Grid. At least you've got a flavour of what this object will be.

## Events

The Grid already contains 22 events hooked in. Here they are again:

|                        |                        |                      |                         |
|------------------------|------------------------|----------------------|-------------------------|
| <i>XCanColSize</i>     | <i>XCellMouseLeave</i> | <i>XEditStart</i>    | <i>XRowSized</i>        |
| <i>XCanRowSize</i>     | <i>XCellMouseMove</i>  | <i>XEditVerify</i>   | <i>XRowSizing</i>       |
| <i>XCellChange</i>     | <i>XCellMouseUp</i>    | <i>XKeyMove</i>      | <i>XSelectionChange</i> |
| <i>XCellClick</i>      | <i>XColSized</i>       | <i>XPageChanged</i>  | <i>XViewChange</i>      |
| <i>XCellMouseDown</i>  | <i>XColSizing</i>      | <i>XPageChanging</i> |                         |
| <i>XCellMouseEnter</i> | <i>XEditEnd</i>        | <i>XPageOpen</i>     |                         |

The ones which have been put in bold are the ones which fired in the simple example shown below.

One way to see some of them in action is to define a simple handler for all of them at once with an expression like:

```
Ⓜwi''(c''('on'),''Ⓜwi'events'),'c''c'Ⓜwself Ⓜwevent Ⓜwarg'
```

Then move the cursor to cell 6 2 and start typing APL, then press Enter, then type C in cell 7 2 and press the down arrow key.

You should see the following flow of events displayed in the APL session:

```

...
ff.ss XCellMouseLeave 5 2 6 2
ff.ss XCellMouseEnter 6 2 5 2
ff.ss XCellChange 6 2 7 2
ff.ss XSelectionChange 6 2 1 1 1
ff.ss XCellMouseDown 6 2 1 1 0
ff.ss Focus
ff.ss Focus
ff.ss XCellMouseUp 6 2 1 0 0
ff.ss XEditStart 0 6 2 394204
ff.ss XEditVerify 65 1 6 2 394204
ff.ss XCellMouseLeave 6 2 0 0
ff.ss XCellMouseEnter 0 0 6 2
ff.ss XEditVerify 80 1 6 2 394204
ff.ss XEditVerify 76 1 6 2 394204
ff.ss XKeyMove 7 2 6 2 13 0
ff.ss XEditEnd 0 6 2 394204 APL
ff.ss XCellChange 7 2 6 2
ff.ss XSelectionChange 7 2 1 1 1
ff.ss XCellMouseEnter 6 2 0 0
ff.ss XCellMouseEnter 6 2 0 0
ff.ss XEditStart 0 7 2 394204
ff.ss XEditVerify 67 1 7 2 394204
ff.ss XKeyMove 8 2 7 2 40 0
ff.ss XEditEnd 0 7 2 394204 C
ff.ss XCellChange 8 2 7 2
ff.ss XSelectionChange 8 2 1 1 1
ff.ss XCellMouseLeave 6 2 6 1
ff.ss XCellMouseEnter 6 1 6 2
...
ff.ss Unfocus

```

Every time an event occurs Dwarg provides you with important information you need to know to properly handle this event, like the current cell coordinate, the value of the key pressed, the cell content, etc.

## Current State of the Grid

In my opinion, in its current state (Alpha Test) the APL+Win Grid is far from being finished. There will probably be quite a number of additional methods, properties and events added and improvements made before APL+Win Release 4.0 is out.

Some properties are incomplete or do not work yet.

Even like that, this object is secure enough and we have been able to use it with success in 2 different applications, though sometimes we encounter difficulties.

Here is a list of the things we have been missing most:

- The **modified** property which is common to all APL+Win objects and tells if the object has been changed, is not yet hooked in: we have had to write our own modified property for the Grid
- The APL+Win **def** property which stores an image of a form in an APL variable currently ignores the Grid, so we can't use it to recreate forms containing grids
- The **FontName**, **FontSize**, **FontStyle** do not yet work and the Grid can only display one font in one size. Happily this font and size is fine. Of course this should be fixed for Release 4.0
- Some properties like the **BorderStyle** property are not easy to use and APL2000 is aware of that
- Probably the most annoying problem is the fact that we have had to fight a lot to get the grid display correctly in our real life applications. In a complex form with a Selector and many Pages containing Grid objects, the Grid would not display at all when opening a Page: we would have to click where the Grid should have appeared to make it display itself. Or the Grid would display without its scroll bars and only display its scroll bars when we click in it. In other cases the only way we found to get the grid displayed correctly was to change a column width by one pixel and change it back immediately. The **XRedraw** method did not always work the way we expected (i.e. forcing the Grid to display itself immediately). The good news is that we have been able to solve ALL of these problems by changing the way we were displaying the grid's parents or grandparents. But still there should be work done to improve this area, since in its current state it is time consuming to solve these problems. Of course we knew we were using an Alpha Test version of the Grid and that these kinds of problems are more than expected.
- The **onEditVerify** event fires when pressing alphanumeric keys but does not fire for some other keyboard keys (usually handled by the **onKeyDown** event). This is a serious problem which is difficult to get around when you need to react to **KeyDown** keys pressed in the Grid. I hope there will be a solution to trap **KeyDown** keys some way or another in the final release.
- There is no way to change grid line colors
- The horizontal scroll bar does not go away when all the columns fit within the visible part of the Grid object. There is no such problem with the vertical scroll bar.

## Conclusion

Despite these little early Alpha Test problems, this APL+Win Grid object is a real pleasure to use and is already quite secure: in 6 months use, I don't remember having crashed APL+Win because of it.

There is no doubt for me that the APL+Win Grid will soon become the most important object most APL+Win developers will use, as soon as Release 4.0 is out.

There are a lot many ways in which one can use a good Grid object and a lot of tricks that can be made with such an object. It can greatly improves GUI interfaces and at the same time greatly eases the application development process.

It is extremely useful in almost all applications.

The APL+Win Grid is very promising because it has the following qualities:

- It is completely array oriented and designed for APL-minded people
- It is extremely fast
- It is very easy to use
- It is very consistent in its design (it always uses Row and Column vectors in the data arguments)
- It announces itself to be very complete
- It seems to be based on a solid product with a small memory FootPrint and 100% based on MFC
- It seems very secure
- It can be called from Javascript, so you can use the same object in a Web based front end to your APL application

My only regret is that such an object has waited so long to find its way within APL+Win but it just might be worth the wait.

# Hyperbolic Symmetry is a Breeze

by Cliff Reiter (*reiterc@lafayette.edu*)

## Prologue

During the past couple semesters I have enjoyed an occasional practice with Lafayette's Ultimate (Frisbee<sup>1</sup>) Club. An annual club tradition is the creation of a design that is hot stamped onto discs that are used for fund raising, practice, and games. The students create designs that are attractive to college students (since they need to sell them) and often their designs give interesting insights into college age values. I have not been involved in designing the club's discs. However, I am extremely jealous. I wanted to create a design for a disc. Many people, including me, find hyperbolic designs are quite interesting. Indeed, Escher was stunned and delighted when he saw some of Coxeter's hyperbolic tilings and this led to his beautiful Circle Limit III woodcut. See [7] or [11] for the image, [5] for a mathematical discussion of the image, and [11] for discussion of the symmetry in Escher's drawings.

This note is the story of a hyperbolic J disc design. We will see that creating hyperbolic symmetry is a breeze. Basically we construct a polygon, which happens to be in the shape of a J. We then multiply the polygon by a bunch of hyperbolic matrices and display the results in a graphics window. Creating images with hyperbolic symmetry really is that easy. However, the story here is more interesting. We want to use a two-colour design. Thus we need to separate "levels". This requires careful order of operations and attention to numeric precision. Furthermore, the company that does the hot stamping requires the colours to be separated at least  $1/32$  of an inch, but hyperbolic tilings get small and intertwined near the edge. Hence, we used some image processing techniques to keep the colours apart. Lastly, we wanted to add some text around the edge. That was easy using our annular insertion utility AnIns that is described in [10] and available from [9].

---

<sup>1</sup> Frisbee is a trademark of Wham-o mfg. co. The official disc of the Ultimate Players Association is a Discraft 175g Sport Disc.

## Hyperbolic Symmetry

There are two convenient models for hyperbolic geometry. These are the Poincare model in which the hyperbolic plane appears in the unit disk and the Weierstrass model which is on the surface of the upper half hyperboloid  $z = \sqrt{1+x^2+y^2}$ . Plotting images in the Poincare model is easiest, while the symmetries are easiest to describe in the Weierstrass model. Fortunately, it is easy to convert between these models. We can convert points in the Weierstrass model to the Poincare model by the projection  $W2P$  which is given by the following:

$$W2P \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \frac{1}{1+z} \begin{pmatrix} x \\ y \end{pmatrix}$$

The inverse is:

$$P2W \begin{pmatrix} x \\ y \end{pmatrix} = \frac{1}{1-x^2-y^2} \begin{pmatrix} 2x \\ 2y \\ 1+x^2+y^2 \end{pmatrix}$$

It is straightforward to implement these in J.

```
W2P=: (}: % >: @{:)"1

W2P 2 _2 3
0.5 _0.5

P2W=: (] (-.@] %~ +:@[, >:@]) (+/)%:*)"1

P2W 0.5 _0.5
2 _2 3
```

In [1] and [8] hyperbolic symmetries are used to create probabilistic iterated function systems, and our implementation of the symmetries is similar to the ones appearing there. However, we will be applying the symmetries to polygons, so our graphics construction is quite different.

It is known that there is a tiling of the hyperbolic plane with  $p$ -sided polygons meeting  $q$  at a vertex if and only if  $(p-2)(q-2) > 4$ , see [4]. The "edges" of these tilings are arcs and it looks like the polygons are getting smaller near the edge, but in the hyperbolic sense, the tiling is legitimate. The symmetries of the tiling are generated by reflections through the sides. All of them can be described in

terms of the following reflections; see [1], [3], or [6] for further description of the geometry.

- The reflection  $A$  across one edge of the  $p$ -sided polygon;
- The reflection  $B$  across the  $x$ -axis; and
- The reflection  $C$  across a line from the origin to a vertex of the polygon.

The transformations are represented with matrices  $A$ ,  $B$ , and  $C$  (acting on the Weierstrass model) as follows:

$$A = \begin{pmatrix} -\cosh(2b) & 0 & \sinh(2b) \\ 0 & 1 & 0 \\ -\sinh(2b) & 0 & \cosh(2b) \end{pmatrix}, \text{ where } b = \cosh^{-1} \left( \frac{\cos\left(\frac{\pi}{q}\right)}{\sin\left(\frac{\pi}{p}\right)} \right),$$

$$B = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad \text{and} \quad C = \begin{pmatrix} \cos\left(\frac{2\pi}{p}\right) & \sin\left(\frac{2\pi}{p}\right) & 0 \\ \sin\left(\frac{2\pi}{p}\right) & -\cos\left(\frac{2\pi}{p}\right) & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

It is useful to define two more matrices,  $S=CB$ , which gives a counter-clockwise rotation by  $2\pi/p$  around the origin, and  $T=AC$ , which gives a counter-clockwise rotation by  $2\pi/q$  around a vertex of the central polygon of the tiling.

These symmetries can be implemented in J. We give  $p$  as the left argument and  $q$  as the right argument.

```
load 'trig'

A=: 4 : 0
b=.arccosh (cos 1p1%y.)%sin 1p1%x.
((-@cosh,0:,:sinh),(0:,:1:,:0:),: -@sinh,0:,:cosh)2*b
)

B=: (1 _1 1*=:i.3)"_

C=: 4 : 0
(((cos , sin),: sin , -@cos),0 0 1"_) 2p1%x.
)
```

mp=: +/ . \*

S=: C mp B

T=: A mp C

ABCST=: A,B,C,S,:T

Here we construct the matrices associated with  $p=5$  and  $q=6$ .

'a b c s t'=: 5 ABCST 6

$s$   
 0.309017 \_0.951057 0  
 0.951057 \_0.309017 0  
 0 0 1

$t$   
 \_1.03262 \_3.17809 3.1885  
 0.951057 \_0.309017 0  
 \_0.985302 \_3.03245 3.34164

There are three classical families of discrete symmetry groups of the hyperbolic plane but we are interested in the family involving just rotations. It is generated by  $S$  and  $T$ . The above matrices  $s$  and  $t$  are from the case when there are 5-fold and 6-fold rotations.

## Creating a J

We think of a  $J$  as a tee shaped polygon atop two semicircles. We pick a convenient scale (where the semicircles are about the origin) and then transform the  $J$  into a polygon in the Poincare disk. We begin by recentering the polygon (subtracting the average of the minimum and the maximum in each coordinate) and then dividing by the maximum norm.

$\$cj=: | : 2 \ 1 \ 0. / \ 2r200p1 * -i. \ 101$   
 101 2

$\$J=: 2 \ 0,2 \ 8, _1 \ 8, _1 \ 10,6 \ 10,6 \ 8,4 \ 8,4 \ 0, (4*cj), |.2*cj$   
 210 2

$(<./, :>./)J$   
 $-\frac{4}{6} \ \frac{4}{10}$

```

norm=: +/&.(*: "_)"1
J=: (%>./@:norm) J-"1]1 3
(<./,:>./) J
_0.581238 _0.813733
0.581238 0.813733

```

Since we will apply many transformations, we actually want to start with a smaller version to avoid overlap. We scale the J-polygon by 0.29 and add 0.24 0.23 to each coordinate using the following affine transformation.

```

]af=: 0.29 0 0,0 0.29 0,:0.24 0.23 1
0.29 0 0
0 0.29 0
0.24 0.23 1

$aJ=: }:"1 (J,.1)mp af
210 2

```

The next three commands use utilities from script *dwin.ijs* to plot the polygon *aJ*. The script is available at [9]. We use *dwin* to open a graphics window and *dpoly* to draw the polygon.

```

load 'fvj2\dwin'
_1 _1 1 1 dwin 'J'
0 0 0 dpoly J

```

The result is shown in Figure 1.

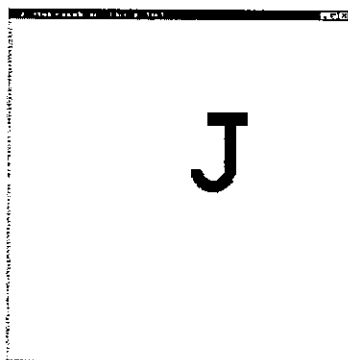


Figure 1. A J-Polygon

## Making Levels

Now we turn to making the hyperbolic transformations we need, taking increasing care to organize them into "levels". First, the function `clean` multiplies the absolute value times signum (which is zero very near 0). This makes entries very close to zero equal to zero, reducing roundoff error. Then `S1` is the matrix powers of `s`; that is, these give the five 5-fold rotations about the origin (including the identity). Note we speak of the transformations as though they are in the Poincare disk, even though they are really in three space. Second, `S2` is a rotation about the other point, followed by symmetric rotation around the origin. It is constructed using `mpt` which gives a table of all possible matrix products. This yields 25 transformations (it turns out they are all new, although that is not obvious). Next, we get `S3` by applying the same technique. However, this time we will get duplicates unless we remove the matrices from the previous levels via `(S1,S2)-.` as seen below.

```
clean=: * * |

mpt=: mp"2 _" _ 2

$S1=: clean s mp^(i.5) s
5 3 3

$S2=: clean ,/S1 mpt t mpt S1
25 3 3

$S3=: (S1,S2)-.~clean,/S1 mpt t mpt S2
100 3 3
```

In our disk design, one hot stamp die will be used for the colour used for levels 1 and 3. Hence we open a graphics window (using *dwin.ijs* again) and plot the result of applying those transformations to the polygon `aJ`. We must be careful to switch back and forth between the Poincare and Weierstrass models at the correct times using `W2P` and `P2W`.

```
wd 'reset;'

WINSHAPE=: 1000 1000

(1.25*_1 _1 1 1) dwin 'lev13'

0 0 0 dpoly W2P S1 mp"2 1"2 _ P2W aJ

0 0 0 dpoly W2P S3 mp"2 1"2 _ P2W aJ
```

Can you count that the number of polygons is correct? If you haven't made this plot yourself, you can peek ahead to Figure 2.

If we are trying to create an image with hyperbolic symmetry, we are essentially done. You could add a plot of level 2 (and 4 and 5) and have a very nice image. This is a good time for readers who want to create their own hyperbolic images to try their hand with polygons of their own choosing and different hyperbolic symmetries (other than the  $p=5, q=6$  we used).

Next we save the image as a bitmap.

```
glfile 'lev13.bmp'
```

```
glsavebmp 1000 1000
```

For our application, we need registration marks (crossing lines) in each corner for the printer to use to align the two colours. This was accomplished as follows and then the file *lev13r.bmp* was saved using commands like those above. It contains the level 1 and 3 polygons and the registration marks. The file created that way is shown in Figure 2.

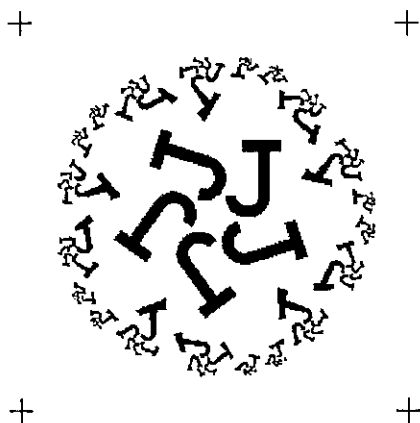


Figure 2. Levels 1 and 3 with Registration Marks

```

]corn=: 1.175*(#:i.4){_1 1
_1.175 _1.175
_1.175 1.175
1.175 _1.175
1.175 1.175

```

```
ha=: 0.075*_1 0,:1 0
```

```
va=: 1|."1 ha
```

```
0 0 0 dline corn+"1"1 2 ha
```

```
0 0 0 dline corn+"1"1 2 va
```

Next we continue to two further levels in much the same manner. However, for S4 we additionally use nub ~. to remove duplicate entries at the same level. For S5 we need to use both nub and remove with larger tolerance (~.!.1e\_14 and ~.!.1e\_14) in order to compensate for increased roundoff error.

```

$S4=: ~.(S1,S2,S3)-.~clean,/S1 mpt t mpt S3
375 3 3

```

```
S1234=: S1,S2,S3,S4
```

```

$S5=: ~.!.1e_11 S1234 -.!.1e_11~clean,/S1 mpt t mpt S4
1400 3 3

```

The result of plotting levels 2, 4, and 5 is created in a manner like that above; however, we used a variant of aJ with fewer vertices for level 5 to keep the graphics buffer from overflowing. We also added the registration marks and then saved the result as the file *lev245r.bmp*.

## Image Processing a Gap

If we used the *lev13r.bmp* and *lev245r.bmp* images to create the actual dies, the resulting images would be too close together in some places. You can test that by plotting levels 1-5 together. We can handle this by first "fattening" the polygons shown in Figure 2 (aiming for something like 1/32 of an inch expansion). We can accomplish this using minus 3 cut (which is a very handy facility for image processing). Then we can replace the image from *lev245r.bmp* by only keeping the points that don't meet the fattened polygons. It is convenient to change the images to 8 bit graphics and use functions from *raster4.ijs* (a script for use with [8] available from [9] for the image file reading and writing. The details are fairly

brief, and the impact subtle, but important. Further commentary appears after the computation below.

```
load 'fvj2\raster4'
lev13=: 0={."1 readbmp24 'lev13.bmp'
plev13=: 1008 1008{._1004 _1004{.lev13
onein=: 1&e.@:
flev13=: 9 9 onein;._3 plev13
wb=: 255,: 0 0 0
(wb;flev13)writebmp8 'flev13.bmp'
lev245=: 0={."1 readbmp24 'lev245r.bmp'
die2=: lev245*~.flev13
(wb;die2)writebmp8 'die2.bmp'
```

Note the entries of `lev13` are 0 or 1 depending on whether the corresponding pixel is white or black. The array `plev13` adds blank padding around the edges. Figure 3 shows `flev13` which are the fattened polygons obtained by seeking ones in a 9 by 9 tessellation of the padded array.

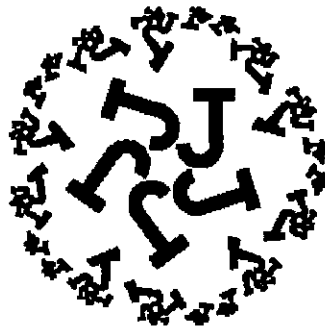


Figure 3. Fattened Level 1 and 3

Notice `die2` is the parts of levels 2, 4 and 5 that are not part of the fattened polygons. We save the image as an 8-bit image for convenience even though it only has the white and black palette (`wb`).

## Adding Text

Our last step is to add text along the outer rim in the same colour as levels 1 and 3. First we put the text into a rectangular bitmap.

```

wd 'reset;'
WINSHAPE=: 4510 100
dwin 'textrect'
glfont '"Comic Sans MS" 960 bold'
glrgb 0 0 0
gltextcolor ''
gltextxy 1 990
text=: 'Hyperbolic Symmetry is a Breeze with ... etc'
glshow ''
glfile 'txtr.bmp'
glsavebmp 4510 100

```

That creates a 4510 by 100 pixel image with the desired text. We put it onto the rim of the image from levels 1 and 3 using AnIns from *anins.ijs* as described in [10] and shown below.

```

txtr=: 0={."1 readbmp24 'txtr.bmp'
lev13r=: 0={."1 readbmp24 'lev13r.bmp'
die1=: lev13r (i00 111%125) AnIns 1p1 _1p1 txtr
(wb;die1) writebmp8 'die1.bmp'

```

### A Colour Image

Lastly, the colour image can be created with the following commands. The palette is white (for background), blue (for die 1), green (for die 2) and black (for the registration marks).

```

pal=: 255,0 0 255,0 255 0,:0
(pal;die1+2*die2)writebmp8 'hy_fly_di.bmp'

```

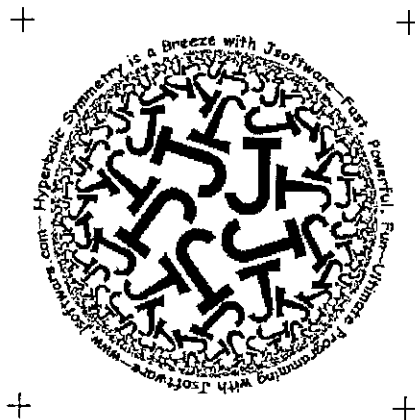


Figure 4. The Disc Design

Figure 4 shows the resulting image. Can you see the separation of *die1* from *die2*?

The image from Figure 4 can then be transferred onto a flying disc. Figure 5 shows Ken Iverson at Kiln farm trying out his prototype of a flying disc with this image (photo by Jana Michaud).

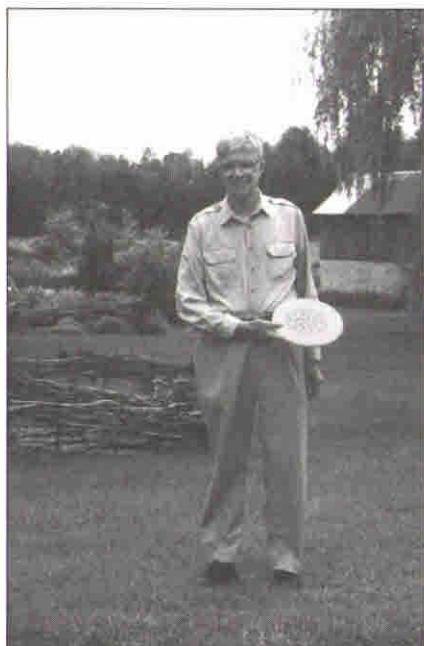


Figure 5. Ken Iverson with a Flying Disc at Kiln Farm

## References

- [1] B. Adcock, K. C. Jones, C. A. Reiter, and L. M. Vislocky, Iterated Function Systems with Hyperbolic Symmetry, *Computers & Graphics*, **24** (2000) 791-796.
- [2] N. Carter, S. Grimes, and C. Reiter, Frieze and Wallpaper Chaotic Attractors with a Polar Spin, *Computers & Graphics*, **22** 6 (1998) 765-779.
- [3] K.W. Chung, H.S. Chan and B.N.Wang, Hyperbolic symmetries from dynamics. *Computers & Mathematics With Applications*, 1996, **31**, 33-47.

- [4] H.S.M.Coxeter, *Generators and Relations for Discrete Groups*, 4th ed. Springer-Verlag, 1980.
  - [5] H.S.M.Coxeter, The Trigonometry of Escher's Woodcut "Circle Limit III", *The Mathematical Intelligencer*, 18 4 (1996) 42-46 and at [http://www.ams.org/new-in-math/circle\\_limit\\_iii.pdf](http://www.ams.org/new-in-math/circle_limit_iii.pdf).
  - [6] D. Dunham, Hyperbolic symmetry. *Computers & Mathematics With Applications*, 1986, 12B, 139-153.
  - [7] M. C. Escher, Woodcut Circle Limit III, <http://www.ams.org/new-in-math/escher-fig1.html>.
  - [8] C. A. Reiter, *Fractals, Visualization and J*, 2nd Edition, Jsoftware, Toronto, 2000.
  - [9] C. A. Reiter, J materials, <http://www.lafayette.edu/~reiterc/j/index.html>.
  - [10] C. A. Reiter, CD Labels and More, *Vector Vol.17 No.2* 98-105.
  - [11] D. Schattschneider, *M.C. Escher: Visions of Symmetry*. W.H. Freeman and Company, 1990.
-

# Working with Existing Ascii Files Using J Memory Mapping

by Donald B. Pittenger ([dbpitt@demlab.com](mailto:dbpitt@demlab.com))

This article deals with memory-mapped files using J from the perspective of a pretty average J programmer who is a bit baffled by the terse J documentation and more that a little frustrated by the lack of information about 'what this memory mapping is really like'. This is my attempt to close the information gap, albeit from the limited perspective of my own attempts to modify an existing system to incorporate memory mapping technology.

Documentation on J memory mapping is mostly in the form of two tutorial labs included with J (the Help index in J4.04c brings up nothing on memory mapping if you search on 'memory' or 'mapping' or 'files'). Otherwise, Chris Burke has an introductory article in *Vector* 15.4 that offers useful background information. I urge you to read Burke's article and to print and read the lab *Mapped Files*, as most of the important information there is not covered here.

The sources just cited mostly deal with files that are created within J; little is said about pre-existing files other than that they too can be mapped. Since I tend to work with files created by spreadsheets or that were packaged on CDs by government agencies such as the U.S. Census Bureau, I was keenly interested in how memory mapping might make life easier for me. I got some tips via the J forum at [jsoftware.com](http://jsoftware.com), but ultimately had to experiment for myself. This article reports what I have discovered so far.

## J and Files: Problems Encountered

Even though modern APLs are quite capable of dealing with 'native' files, J somehow feels more comfortable doing so. For that subjective reason and because scripts can be easily modified using Notepad or a similar simple editor, I've followed the common path of writing more and more of my systems in J.

Aside from large-scale data extraction from CDs, I tend to work with data created by spreadsheets and saved as \*.txt files (tab delimited, with end-of-line characters). Data from such files can be converted to boxed matrices using the J utility `clipunfmt`. When my systems create new data, these are usually text-formatted using the utility `clipfmt` and written to disc as text files. The advantage of text files is that they can include row and column labels (or even

more elaborate self-documentation) and therefore are understood by humans. Further, if imported by a spreadsheet program, data are easily modified and the results re-saved as a text file. Also, spreadsheets can be used for data entry and \*.txt versions can be saved for later use by J. If there are more convenient means of data keeping for small or moderate sized data sets, I don't know what they are.

However, spreadsheet-based files become impractical once they grow beyond a megabyte or so in size. (I've dealt with one such beast of about 20MB, but the resulting text file nearly brought J to its knees, a subject that will be discussed below.) Here is an example of the sort of file I often deal with. The United States contains around 3,150 counties or county-equivalents plus 50 states and the District of Columbia. The sum of those areas, plus the nation itself, comes to about 3,200. I am a demographer and need data for populations by sex and age group. Typically, ages can be in five-year ranges starting with ages 0-4 and continuing 5-9, 10-14, etc. up usually to range 85+. This means three data dimensions—area, sex, and age—that need to be flattened into a two-dimensional table. I happen to work with age as the columns and stack sex into two rows for each area. Adding a column of sex totals plus a few columns needed to contain area names and geographical codes, I wind up with a table of 22 columns and around 6,400 rows. An Excel file with summation coding, to allow data entry validation, runs about 1,500KB and the text file version 655KB.

Let's say I have a system written in J that fetches and uses the text file just described. The character string is read into J and `clipunfmt` transforms the string into an array of boxed strings. Let's further say that I want to extract all the counties and place them into a rank-three area-sex-age array before getting on with whatever the system really does with the data. Normally, I'll have the system strip off the area information columns and remove the duplications (remember, there are two rows per area—one per sex) so that I can set aside a boxed index table for use in extracting particular counties from the data array that is being built. I might also strip off some leading rows containing header information for further use. What's left is a matrix containing only boxed character-based numbers.

Here trouble begins. The characters in those boxes need to become unboxed numbers. If the matrix was small, you might write `vec=. ;". each mat` to get a numerical vector which then could be shaped as desired. Unfortunately, if the matrix is large, J will churn, your hard drive light will assume a steady green state, and it will take a bit of frantic work to forestall a system lockup. The problem is, boxed arrays seem to chew up RAM. One computer I work with is a 300Mhz Compaq with 64MB RAM running under Windows 98 with a LAN in the

background. Converting a boxed array of 60,000 cells will crash J in that environment. That's maybe 500K of data with more than 100 times that amount of (potential) memory, and it crashes! (One problem with J is that its quirks are not as well documented as APL's. Perhaps Roger Hui or some other knowledgeable Jster would write something for *Vector* paralleling the APL optimization pieces that have appeared over the years.)

My solution is a utility that converts the matrix into a vector and then executes and un-boxes in 10K or 20K box chunks. There are other ways of skinning this cat, no doubt, but all this is merely an annoyance. The *real problem* is when you have finished the conversion to numerical arrays. It happens that the system I'm discussing reads in data from seven censuses (1930 through 1990). The pre-1970 data are not complete; many counties have empty boxes and the conversion creates zeroes where no data exist. The conversion process (in which matrix chunks are erased the instant they are no longer needed) yields arrays which do not seem to take up more than three or four megabytes of RAM. Yet the result is that the computer paints Windows objects slowly and the hard drive is usually madly paging memory. Once J has been terminated, these symptoms persist, though in weakened form. On the computer I describe, the clock will begin to lose three or four minutes a day. The only cure is a reboot.

The other computer I use has 128MB RAM and runs under Windows NT with no network, and it doesn't get as woozy as the 64MB machine. Still, no matter how much memory is on-board, a comparatively modest amount of boxing and un-boxing seems likely to create problems. Is there a solution? Why not try memory mapping?

## Playing with Memory Mapping

As was mentioned above, I didn't find enough documentation to confidently plunge into experimentation: I had to make the plunge with trepidation. Happily, it seems that it is a simple matter and, so far, I've broken nothing.

The main thing with pre-existing files (as opposed to the sort of J-created files demonstrated in the labs) is that you apparently have to do the header file work on your own; the lab is sketchy on this point. Basically, you need to know the file structure and then create a descriptor table, program to the structure, or both. I hope to clarify this below.

First, here are some simple experiments you can try which let you see what happens when an existing file is mapped.

Start by fetching the mapped files script:

```
load 'jmf'
```

You also should have created a small Ascii file to use as the experimentation target. It is a good idea to create a table of four or five rows and half a dozen columns in a spreadsheet and save it as a tab-delimited or space-delimited or perhaps comma-delimited file. Say it is called `test.txt` and its path is `c:\directory`.

To map the file, simply type

```
JCHAR map_jmf_ 'dat'; 'c:\directory\test.txt'
```

where `dat` is a J variable (or 'noun' for those of you who insist on using J-speak) that represents the content of the mapped file. JCHAR is a byte-count cover supplied by J; others are JB01, JINT, JFL and JCMPX. Their meanings should be apparent, but you might find it interesting to replace JCHAR with one or another of them when mapping and then inspect the first few items using, say `3{.dat`. JCHAR should produce recognizable parts of the table you created; the others should yield various numbers whose form is related to the number of bytes specified by the cover variable used.

To clear the mapping, type

```
unmapall_jmf_ ''
```

and you are ready to map or re-map a file—J will protest if you try to map a currently-mapped file.

*Lab Mapped Files* indicates that you can give the rank and shape by changing the left argument of the mapping function. For instance,

```
(JCHAR;5) map_jmf_ 'dat'; 'c:\directory\test.txt'
```

will put `dat` into five columns, according to the lab. If you try this with your test data, you will probably find that each column is one character wide. I haven't experimented with this much, so probably this feature is more useful than it seems.

The important thing is that, so long as you know the underlying structure of the file, you can almost instantaneously access any part of it using a simple command such as

```
a=. ((linecnt*80)+i.80){dat
```

where `linecnt` is the sequence number of a line you want to access, lines happen to be 80 characters each, and you want to assign the characters in that line to variable `a`. Selecting characters within the line will take a little more elaborate offset-specification, of course, but the principle is the same.

You can bog the system down or perhaps even crash it if you try to pass through the entire file at once. For example, I did `+/dat=TAB` on a 600K file, causing all of it to be handled in RAM or virtual-RAM. As a result, interactive work suffered from slow painting of characters to the session screen (remember the joys of Sharp APL/PC on the original 8088 IBM PC?: that's how slow J looked). Moral: extract fairly small chunks of a file at one time. ('Fairly small' can be a hundred kilobytes or more. It depends on how much RAM you have that J can use without paging. Still, smaller is generally better.)

This might look about the same as simply opening a file and reading byte segments. One difference is that `jmf` uses calls to the operating system which is able to do file access in some sort of optimal fashion by bypassing normal data loading operations. But this goes far beyond my competence: read the references mentioned in the lab to understand what's happening behind the memory mapping curtain. From a program-length standpoint, little is gained because byte segments have to be specified in both instances. (Remember we are dealing with character string files here; mapped files created from J can be integer, floating point, etc., so access can be in terms of, say, the first ten numbers which would be simpler than extracting characters and then converting.)

## How Memory-Mapped Files are Structured

It is interesting to write a small test memory-mapped file in J, following the instructions in the lab, and then examine the file once it has been unmapped. My early experimentation was with APL+Win, which lets me read files and file segments as integer, character, floating point, and so forth. (In J, you have to read the file as character data, then use an appropriate conversion routine for integer, float, or whatever. Score for APL.)

A J memory-mapped file has two parts: a header and the balance of the file. This was mentioned above. What the documentation does not tell you is that the header—presently 284 bytes long—is *integer* data. The balance of the file is whatever type it is. Mixed up in this is whatever junk the file has overwritten; when inspecting either the header area or the balance area you are likely to see scraps of HTML code or other stuff that might confuse you at first. The header tells you how large it is itself, how large the file balance is (this can be greater than the number of bytes populated with data), the number of bytes per item, the

total number of items, the rank of the data and the shape of the data. There are a couple of other numbers whose meaning I'm not sure of, but they don't matter so long as you already know the data type for the file body.

This means it is theoretically possible to (1) add a J-style header to an existing file or (2) construct a J-style memory-mapped file in APL or some other language. If correctly done (and I've tried the second option, but using J instead of APL), you can map the constructed file and manipulate it just as if it were a memory-mapped file created using the J tools.

A word of warning: The folks at Jsoftware, Inc. do not guarantee that the present (September, 2000) structure of J memory-mapped files will remain unchanged. This means 'rolling your own' file headers or other tricks might not work in the future. Of course, little in computing is really permanent. Not many years ago, the J Windows interface was drastically changed, and most programs dealing with event handlers had to be revised.

## Memory Mapping Existing Data Files

Text files pose a memory-mapping problem because delimited strings are not the same length. That is because the whole point of delimited files is to cut file size by eliminating unneeded blank characters. The only solution is to create some sort of mapping index table that specifies the sequential file position of each end-of-line character and each tab (or comma, for CSV files) within each line. I wrote a program that did this for the census data tables mentioned above, and found that the index was roughly a tenth the size of the target file. This explicit indexing with its requisite custom examination of each file—a task that would have to be repeated each time the target file was altered in any substantive way—persuaded me that text files and memory-mapping go together only if alternatives are even more costly or impractical. But it would be practical if I had to deal with a text file on a CD-ROM, which by definition is unalterable. Then I could pass the file through an indexing program and save the resulting index table with the knowledge that no further upkeep would be needed.

Fixed-field character-based files seem to be a better fit with memory mapping. Each line—or record—is the same length, so all you need to read the desired bytes is an index or data dictionary of the field width and perhaps the column subject plus maybe a rows index. This would be far smaller than the text table index mentioned above. Furthermore, so long as field widths are unchanged, there is no need for re-indexing the file after making changes.

So what I did was write a program converting the text files into fixed-column files. I had one row per area, but strung the sex data into a vector of males by age group followed by females by age group. Each row also started with a short column containing the area's geographical identification code. The resulting files were around 1040KB, up from about 655KB.

A defect of fixed-format files is that I can't simply read the data into a spreadsheet for easy modification. I might be able to modify data from a spreadsheet or word-processing program, but I would have to be very careful not to end up with too few or too many characters per line. Operationally, the best strategy is to maintain the text file and rebuild the fixed-column file each time the text file is changed. But this is nearly the same as the re-indexing of text files noted above except that the index file would be tiny and extraction index specification might run a little faster. (Of course you can use memory mapping to change characters in-place. This should work fine in a data processing environment. But small numbers of *ad hoc* changes are probably more suited to the spreadsheet option described above.) The reader should now see that selecting an optimal solution for his needs might require some head scratching or maybe even coin flipping.

## Memory Mapping in Action

Considerations of optimization of disc storage and operational hassle aside, how well does memory-mapping work in practice?

To find out, I rewrote a J script that reads in the census data and calculates estimates of intercensal net migration, displaying age-sex-specific net migration rates on a Windows form, one graph per census interval. The original programs created arrays containing data for all states and counties and caused noticeable slowing of screen painting and other signs of memory paging. There was really no alternative if I used the text files in my database.

The revised system uses fixed-field files, reading in county data for only one state at a time (a convenient way to organize the data in any case). This drastically reduced the size of data arrays while eliminating the need to create boxed tables and then unpack them in the process of creating arrays. The result was a system that did not have an extended 'hourglass' period for initial data assembly and there were no memory paging after-effects. There were pauses after a different state was selected, while data were fetched and migration rates calculated, but the wait was only a few seconds on the relatively slow 300MHz machine. I expect this process would be nearly imperceptible on a 1GHz computer, except perhaps when dealing with states with more than 100 counties.

The verdict? In this test case, the result was much better performance paid for by the relatively small cost of creating alternative files 60 percent larger than the original text files.

## Conclusion

Memory mapping existing character-based files takes some support work, but with a little planning it can eliminate some nasty J side effects related to working with large files.

---

---

# TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

## Contents

|                                  |                  |     |
|----------------------------------|------------------|-----|
| Technical Correspondence         |                  | 112 |
| At Play with J: Four Cubes Redux | Gene McDonnell   | 113 |
| Reflexive Data Domains           | Phil Chastney    | 121 |
| Translate your Dyalog GUIs       | Kimmo Kekäläinen | 131 |
| APL Unicode Font - Extended      | Phil Chastney    | 139 |

---

# TECHNICAL CORRESPONDENCE

From: Eugene McDonnell

September 2000

Readers of Vector might be interested in being told about the web site of Dr. Charles Brenner, a long-time and present user of APL, who has built a profession as an expert in dna analysis. He was recently interviewed by the New York Times and the story is one of many interesting things available at his web site, <http://www.dna-view.com>.

Brenner was the writer of the 1130 APL system in 1967 and the designer and implementer of APL's dyadic and monadic transpose, both projects he was commissioned to do by Larry Breed while he was working at IBM's Watson Research Laboratory in Yorktown Heights, NY. Brenner spent six years in London, earning a living by playing bridge. On returning to California he earned his Ph. D. in mathematics, in the field of number theory. He also created the beautiful boolean transpose software for STSC's APL, and did a number of contract APL jobs at I. P. Sharp in the 80's, including some work for me in speeding up the C code I had written for SAX, Sharp's APL for Unix that was based on STSC's Unix APL.

His current work in DNA analysis uses software written, sold, and maintained by him in STSC APL.

He has received a lot of publicity recently in helping solve a case involving the man who created and ran the DHL express mail empire, but who also sired a number of children on very young south Pacific women — when he died in a plane crash these women hoped their children would obtain a share of his estate. The story of how Brenner ironed out all the details of the claimant's kinship with their father is fascinating.

---

# At Play With J: Four Cubes Redux

by Eugene McDonnell

*A Festschrift for Kenneth Iverson  
on his 80th birthday,  
2000 December 17*

I recently cleaned out the chest of drawers in my bedroom, in the course of which I got rid of many frayed and yellowed handkerchiefs, never-worn T-shirts, paper thin undergarments, and, much to my pleasant surprise, I excavated a set of four coloured cubes, an inch and a quarter to the side, a modern version of a puzzle dating back at least a century, under various names; the set I had is marketed under the name *Instant Insanity* in the United States by Parker Brothers, the purveyor of many other games, most notably Monopoly. The cubes' faces are coloured with one of the four colours blue, green, red, or white, in some mixture. You are asked to stack the cubes one above the other in such a way that each side of the stack contains a face with each of the four colours, in some order.

In 1981 I had written a pamphlet on the puzzle called *The Four Cube Problem*,<sup>1</sup> subtitled "a case study in Basic, APL, and functional programming." In the pamphlet a prize-winning Basic solution and an APL solution written by me were compared.

The Basic program had 91 non-comment lines, and the APL had nine; the average length of a Basic line was 31 characters; of the APL line 21 characters; Basic used 18 variables, APL none; Basic had 21 loops, APL none. The Basic program was written for and executed on the Sorcerer computer, which I suspect was a fairly early desktop computer, and so its execution time of 3 minutes and 5 seconds can't be fairly compared with the APL program, which took 0.7 seconds to execute on one of the largest and fastest commercial computers available in 1981, the Amdahl V8 computer in the I. P. Sharp machine room in Toronto.

The APL solution emphasized the *functional programming* approach introduced by John Backus<sup>2</sup>, employing the direct definition form by K. E. Iverson as a way of facilitating the definition of functions and their use in exposition. The solution used one constant function, one dyad, and seven monads.

The APL solution from my 1981 pamphlet could, I believe, be easily translated into Dyalog APL:

```
I:C S (Bw[1;A])O S(Bw[2;A])O S(Bw[3;A])O G Bw[4;X]
X:3 4 p 3 5 4 6 1 3 2 4 1 6 2 5
G:2 1 3 4 Q(1,ρw)ρw
B:(Rw)∘.= 'BGRW'
R:(v/<w)∧.=Qw)≠w
A:(124)ϕ4≠X,[1]ϕX
O:((1+ρw)≠α),[2](((1+ρα),3p1)×ρw)ρw
S:(∧/∧/∧/ω=<[2]w)≠w
C:'BGRW'[w+.×14]
```

This solution took as argument a 4 by 6 character matrix representing the cubes, with the initial letters of the face colours given in the order top, bottom, left, right, front, and back; the colours were blue, green, red, and white, abbreviated by 'bgrw'. The result yielded was a three-dimensional array of shape n, 4, 4, where n is the number of solutions for the given set of cubes. A belt in a cube consists of a circuit of four faces; there are three basic belts in a cube: left, front, right, back; top, left, bottom, right; and top, back, bottom, front. The last cube is taken as the base of the stack, and only its three basic belts are considered, since rotating or reversing these belts will not provide any essentially different solutions. The face numbers of the basic belts are given by the constant X. For the other cubes, a total of 24 belts are considered, obtained by rotation and reversal of the basic belts in all possible ways. These are given by the function A.

The faces are represented by a boolean vector of length four containing a single 1 in the position corresponding to the colour:

```
Blue: 1 0 0 0
Green: 0 1 0 0
Red: 0 0 1 0
White: 0 0 0 1
```

A boolean representation was chosen in order to economize on space. This conversion is done by B, and the inverse by C. In the display below you can see the sizes of the intermediate arrays formed during execution. The largest needs space for  $*/720$  4 4 4 or 46,080 items. That is how many bytes are needed for a character representation. If bits are substituted, only one-eighth, or 5,760 bytes are needed. Integers would require four times, or 184,320 bytes: a prohibitive size for many systems of that day.

The development of the result, and the shapes of the intermediate arrays are shown below:

```

I: C S (Bw[1;A]) O S(Bw[2;A]) O S(Bw[3;A]) O G Bw[4;X]
 +2 4 4→ +2 4 4→ +2 4 4→ +3 4→ 1
 +16 4 4→ +24 4 4→ +24 4 4→ +3 4 4→ 2
 +3 1 4 4→ 3
 +-----72 2 4 4-----> 4
 +-----28 2 4 4-----> 5
 +-----672 3 4 4-----> 6
 +-----45 3 4 4-----> 7
 +-----720 4 4 4-----> 8
 +-----1 4 4 4-----> 9
 +-----1 4 4-----> 10

```

At the right of line 1 the shape of the belts of the bottom cube is 3 4. In line 2 the boolean conversion has been made, and the shape is 3 4 4. In line 3 the function *G* has added a dimension of length 1; this is needed so that each belt from the next cube can be stacked on each existing stack.

Function *O* does this stacking; function *S* removes stacks containing duplicate face colours. This process continues with the remaining cubes, and finally *C* converts the boolean vector forms into colour letters.

This brief synopsis is a necessary prelude to the description of the J solution.

The function *I* in APL has this equivalent in J:

```
i =: (0&{) (s@o) (1&{) (s@o) (2&{) (s@o x) (3&{)
```

J's *insert (/)* adverb is defined this way in the J dictionary on gerund left arguments:

*m/y inserts successive verbs from the gerund m between items of y, extending m cyclically as required. Thus, +`\*/i.6 is 0+1\*2+3\*4+5.*

This suggests that *i* could be rewritten like this:

```
g =: (s@o)^(s@o)^(s@o x)/
```

Whereas in both APL's *I* function and J's (irrelevant) *i* function it is necessary to specify index values to select rows of the argument, the gerund form of *g*, using J's *insert* primitive (*/*) depends on the nature of insertion to go through the rows of the argument in the proper order, from the bottom up.

The remaining functions are detailed as follows:

```

b =: 0 2 1 3 0 4 1 5 2 4 3 5
x =: (3 1 4$ b)&{
o =: [: ,/ ([: ~. [: a [) , "1 2/]
a =: (i.24)"_ |."_1 [: [4: # [: (] , |."_1) (3 4$b)"_ {]
s =:] #~ 2: > [: >./"1 [: , "2 [: +/"3] =/ [: ~. ,

```

The constant *b* is used here in order to keep the length of displays within the confines of the page width. The *b*'s in functions *x* and *a* should be replaced by the value of *b*. Thus there are actually only five functions needed: *g*, *x*, *o*, *a*, and *s*, and, indeed, *g* could be replaced by its body, reducing it to only four functions. The function *R* of the APL version is replaced by the J primitive *nub* (*~.*). The functions *B* and *C* of the APL solution convert between the character and the boolean list representations used. These are not needed in the J solution because it is designed to be generic, permitting any form of representing the cubes. The nine APL functions use 151 tokens. The five J functions use 106 tokens.

The function *x* applies to the bottom row of the argument, and produces the three basic belts in the form of 1 by 4 tables. Because it is used with the insert primitive, there is no need to specify the index value 3; this comes about because of the nature of insertion. This is the seed needed to start the building of the candidate piles. The result of each step is a three-dimensional array of shape *m,n,4*, where *m* is the number of successful piles so far, initially 3; and *n* is the height of each pile, starting at 1, and ending at 4.

The effect of *s@o* is to insert the dyad *o* between two arrays *c* and *d*, where *c* is shape *k* by 24, where *k* is at most 24, and at least 1; and *d* is the result of the previous step, with shape *m,n,4* as described above. Each of the *k* rows of *a* is placed on each of the *n* by 4 tables, and then these are joined to form a single table of *k* times *m* items. As noted above, the winnowing of the 24 belts selected by function *a* is accomplished by the J *nub* primitive (*~.*) within *o*, ensuring that there are no duplicate solutions.

The result of *o* is passed on to *s*. I rather like *s*; notice the nice way the rank operator gets used: first rank 3, then rank 2, then rank 1. It uses the phrase *] =/ [: ~.* , to form the outer product equal of the argument with the *nub* of its ravel. This gives a four-dimensional array of *k* boolean tables, with a table for each of the rows in each solution. It sums the 3-dimensional arrays, which effectively counts the number of appearances in each column of each colour. The tables are raveled, and a mask is formed with a 1 for each row which contains no item greater than 1, and applied to the argument, which gets rid of all of the candidate solutions containing duplicate colours, leaving only those which

remain as candidates for final solutions. Here is a sample of how `s` works, using `t`, containing two 3-high stacks:

```

 $t
 2 3 4
 t
 brwg
 wwgr
 rgbg

 brwg
 wwgr
 rbbw
 $t0=: t = / ~. , t
 $t0
 2 2 2 4

 $t1=: +/"3 t0
 2 4 4
 t1
 1 1 1 0
 0 1 1 1
 1 0 1 1
 0 1 0 2

 1 1 1 0
 1 1 1 0
 1 0 1 1
 0 1 1 1

] t2=: ,"2 t1
 1 1 1 0 0 1 1 1 1 0 1 1 0 1 0 2
 1 1 1 0 1 1 1 0 1 0 1 1 0 1 1 1

 $t3=: >./"1 t2
 2 1
] t4=: 2>t3
 0 1

```

This solution can be improved a bit. Suppose I replace the function `x` with a similar function `y` which has the property that it is executed only if its argument has shape `,6,`. This can be done using J's function power primitive. Then we could do without the `gerund` form altogether. Here is function `y`:

```
y=: (3 1 4$b)&{^:(,6)"_ -: $)
```

Now we can solve the problem with:

```
(s@o y)/
```

That's all there is to it. J has helped to give a solution significantly shorter than the APL solution.

Now that I had a generic solution, I was able to test it on four different representations: a single character, a symbol, an integer, and a boxed name. All of these are scalars. Here are the four representations of the one-solution set of cubes:

```
nu
3 1 0 0 2 2
3 2 2 1 3 0
3 3 1 1 2 0
3 0 1 1 3 2
```

```
ch
```

```
wgbrrr
```

```
wrrgwb
```

```
wgggrb
```

```
wbggwr
```

```
sy
```

```
`white `green `blue `blue `red `red
`white `red `red `green `white `blue
`white `white `green `green `red `blue
`white `blue `green `green `white `red
```

```
bn
```

|       |       |       |       |       |      |
|-------|-------|-------|-------|-------|------|
| white | green | blue  | blue  | red   | red  |
| white | red   | red   | green | white | blue |
| white | white | green | green | white | blue |
| white | blue  | green | green | white | red  |

And the solutions for each:

```
(s@o y)/ nu
1 0 3 0
2 2 1 3
0 1 2 1
3 3 0 2
```

```

 (s@o y)/ ch
gbwb
rrgw
bgrg
wwbr
 (s@o y)/ sy
`green `blue `white `blue
`red  `red `green `white
`blue `green `red  `green
`white `white `blue `red
 (s@o y)/ bn

```

|       |       |       |       |
|-------|-------|-------|-------|
| green | blue  | white | blue  |
| red   | red   | green | white |
| blue  | green | red   | green |
| white | white | blue  | red   |

A set of cubes which gives 22 solutions is:

```

`white `blue `red  `blue `white `green
`red  `blue `red  `blue `green `white
`red  `blue `red  `green `green `white
`green `white `white `red `blue `red

```

I measured the time needed to produce solutions for each of these representations using two different cube sets, one which had only one solution, and one which had 22 solutions.

Here is a tabulation of the times taken using each of the two test cubes:

|                  | one solution | 23 solutions |
|------------------|--------------|--------------|
| integer          | 0.012        | 0.047        |
| single character | 0.010        | 0.036        |
| symbol           | 0.012        | 0.047        |
| boxed name       | 0.058        | 0.254        |

The table shows that the symbol datatype is competitive in time with the single character and integer data types, which is what we hoped would happen. Symbols are a much more efficient datatype to work with than are boxed names.

It also shows, which should not be a surprise to anyone, that I have on my desk a computer that is seventy times faster and has much larger memory and disk storage than did the large roomful of computer and disk packs that had seemed so very powerful twenty years ago.

---

<sup>1</sup> McDonnell, E. E., *The Four Cube Problem*, APL Press, Palo Alto 1981

<sup>2</sup> Backus, John, "Can Programming Be Liberated from the von Neumann Style? A Functional Style and its Algebra of Programs", *Communications of the ACM* 21, 8, August 1978

---

# Reflexive Data Domains

*by Phil Chastney*

*email: phil @ amadeus-info.freemove.co.uk*

## 1. Nested Arrays

**1.1** The idea this time around is to show how the notation defined previously can be used to describe first lists, and then nested arrays in general.

If we define a typical first generation APL data space as

$$\text{SIMPLE} \leftarrow (\mathbb{H}^*N^*N) \oplus (\mathbb{R}^*N^*N)$$

then, as in K,  $(A; B; C)$  being a three element vector, each of whose elements is a simple array, we can make the assertion

$$(A; B; C) \in \text{SIMPLE}^*3.$$

This can be extended, of course, so that lists are elements of the domain  $\text{SIMPLE}^*N$ , while the domain of rectangular arrays of rectangular arrays is  $\text{SIMPLE}^*N^*N$ .

**1.2** This is OK as far as it goes, but we have defined one depth of nesting only:

$$\text{DEPTH}_1 = \text{SIMPLE}^*N^*N.$$

We can also construct depth-2 nested arrays from depth-1 nested arrays, or, more succinctly,

$$\text{DEPTH}_2 = \text{DEPTH}_1^*N^*N$$

and, in general,

$$\text{DEPTH}_{n+1} = \text{DEPTH}_n^*N^*N$$

and so on, and so on, ad infinitum.

Observe that, just as  $\mathbb{R}^*N^*N$  included all scalar values from  $\mathbb{R}$ , so also

the domain of arrays of arrays defined by  $\text{SIMPLE}^*N^*N$

includes, in particular,  $\text{SIMPLE}^*N^*0$

which reduces to  $\text{SIMPLE}^*0$

which reduces to  $\text{SIMPLE}$

and, by induction, the domain definition at each depth of nesting properly includes all its predecessors.

1.3 The task now is to find an expression which defines the domain of nested arrays, in all its generality. Consider the following:

$$\begin{aligned} \text{DEPTH}_1 &= \text{SIMPLE} * N * N \\ \text{DEPTH}_2 &= \text{DEPTH}_1 * N * N \\ &= (\text{SIMPLE} * N * N) * N * N \\ \text{DEPTH}_3 &= \text{DEPTH}_2 * N * N \\ &= ((\text{SIMPLE} * N * N) * N * N) * N * N \end{aligned}$$

This way lies madness: either we have an infinite number of domain definitions, or we have a domain definition of infinite length. What we really need is some way of expressing this infinite recursion.

1.4 We first define a function **ENCLOSE**, acting on a domain  $D$ , such that

$$\text{ENCLOSE}(D) = D * N * N$$

which can then be used to rewrite the above as a recurrence relation

$$\begin{aligned} \text{DEPTH}_0 &= \text{SIMPLE} \\ \text{DEPTH}_{n+1} &= \text{ENCLOSE}(\text{DEPTH}_n) . \end{aligned}$$

This is classic tail recursion, and can equally well be represented by repeated function application:

$$\text{DEPTH}_n = \text{ENCLOSE}(\text{ENCLOSE} \dots \text{ENCLOSE}(\text{SIMPLE}) \dots)$$

What we want, the set of all nested arrays, is the *closure* of **SIMPLE**, with respect to the **ENCLOSE** function, and this we can write as

$$(\text{ENCLOSE} * N) \text{SIMPLE} .$$

This may require some justification. (If you find all this intuitively obvious, then feel free to skip to the next section — the next bit is detailed but not difficult.)

1.5 A set  $D$  is **closed with respect to a function  $f$** , if

for all  $x$ ,  $x \in D$  implies that  $f(x) \in D$  as well.

For example, taking the square of a natural number will always give us a natural number, so we say that "the natural numbers are closed with respect to the squaring operation". They are not closed with respect to the square-root operation, however: 2 is a natural number, but  $\sqrt{2}$  is not.

Obviously, if  $f(x) \in D$ , then  $f(f(x)) \in D$  by definition.

And so is  $f \dots f(f(f(x))) \dots$ .

Instead of using repeated function application, we could write the same thing using function composition. Using the conventional notation of  $(f \circ g)$  to denote the composition of  $f$  with  $g$ , then  $f(f(x))$  can be rewritten as  $(f \circ f)(x)$ .

For instance, if  $f$  is the squaring function, then the derived function  $(f \circ f)$  raises its argument to the fourth power.

If we denote the set of all functions by  $\_$ , then  $f \in \_$  and  $(f \circ f) \in (\_ \otimes \_)$ .

1.6 Repeated function composition is frequently written using exponentiation<sup>1</sup>, thus:

$$\begin{aligned} f^0(x) &= x \\ f^1(x) &= f(x) \\ f^2(x) &= (f \circ f)(x) \\ f^3(x) &= (f \circ f \circ f)(x) \\ &\vdots \\ f^n(x) &= (f \circ \dots \circ f \circ f)(x) \end{aligned}$$

and, following this convention, we can state that

$$\begin{aligned} f^0 &= \emptyset \\ f^1 &\in \_ \\ f^2 &\in \_ \otimes \_ \\ f^3 &\in \_ \otimes \_ \otimes \_ \\ &\vdots \\ f^n &\in \_ * n, \text{ etc. etc.} \end{aligned}$$

This is starting to look familiar: just as  $\mathbb{R}^*N$  is the set of all finite-length vectors of real-valued numbers, so  $\_ * N$  is the set of all finite-length vectors of function definitions.

To put it another way,  $\_ * N$  is the closure of the set  $\_$  under composition and, in particular,  $f^*N$  (where  $f \in \_$ , and so  $f^*N$  is a subset of  $\_ * N$ ) is the closure of the set  $\{f\}$  under composition. What we want is something slightly different: an expression for the closure of the set  $\{x\}$  under the function  $f$ .

1.7 If a vector of function definitions is applied to an argument, then (conceptually) the serial composition of those functions is first constructed, and the resultant derived function is then applied to the argument in question.

For example, if  $COMPO \in \_ * N$ , then we know what is meant by  $COMPO(x)$ , where  $x$  is a single argument. (That is,  $COMPO(x)$  means (i) the composition of the functions in  $COMPO$ , followed by (ii) the application of the derived function to  $x$  - although, in practice, this would probably be achieved through successive application of the individual functions. The difference is a matter of pragmatics, not semantics.)

<sup>1</sup>) ... and so is repeated function application, as it happens - the notation is used ambiguously

We now adopt the convention that a set of functions applied to the same single argument produces a set of results.

With that convention, the expression  $(\text{ENCLOSE}^*N) \text{ SIMPLE}$  denotes the domain sum of the series obtained by the application of each element of  $\text{ENCLOSE}^*N$  to the argument  $\text{SIMPLE}$ , i.e:

$$\begin{aligned}
 \text{DEPTH}_0 &= (\text{ENCLOSE}^*0) \text{ SIMPLE} \\
 &= \text{SIMPLE} \\
 \text{DEPTH}_1 &= (\text{ENCLOSE}^*1) \text{ SIMPLE} \\
 &= \text{ENCLOSE}(\text{SIMPLE}) \\
 \text{DEPTH}_2 &= (\text{ENCLOSE}^*2) \text{ SIMPLE} \\
 &= (\text{ENCLOSE} \circ \text{ENCLOSE}) \text{ SIMPLE} \\
 \text{DEPTH}_3 &= (\text{ENCLOSE}^*3) \text{ SIMPLE} \\
 &= (\text{ENCLOSE} \circ \text{ENCLOSE} \circ \text{ENCLOSE}) \text{ SIMPLE} \\
 &: \\
 \text{DEPTH}_n &= (\text{ENCLOSE}^*n) \text{ SIMPLE} \\
 &= (\text{ENCLOSE} \circ \text{ENCLOSE} \circ \dots \circ \text{ENCLOSE}) \text{ SIMPLE} \\
 &:
 \end{aligned}$$

and finally:

$$\text{NESTED} = (\text{ENCLOSE}^*N) \text{ SIMPLE}.$$

Normally, the notation would imply that we take the domain sum of all earlier values in the series,

$$\text{NESTED} = \sum_{i \in N} \text{DEPTH}_i,$$

but we already know that each element in the series includes all arrays of lesser depth, so the domain sum of all items in the series is the same as the last item generated by the series, so you are free to think of it as

$$\begin{aligned}
 \text{NESTED} &= \bigcup_{n \rightarrow \infty} \text{DEPTH}_n \\
 &= \bigcup_{n \rightarrow \infty} (\text{ENCLOSE}^*n) \text{ SIMPLE}
 \end{aligned}$$

should that prove more congenial or convenient.

## 2. Other Approaches

2.1 Even after that mind-numbing display of pedantry, the notation may prove a trifle awkward later.

The convention that "a set of functions applied to the same single argument produces a set of results" implies that we can always distinguish the cases where

this convention applies. Functions are themselves sets, so we are claiming the ability to recognise a certain type of set of sets. This may not always be possible — when — is an operand, for example — but we can worry about that later. There are no difficulties with the syntax defined so far.

2.2 Secondly, there is the need to name the ENCLOSE function.

Bear in mind that the *ability* to name an entity and the *need* to name an entity are different things. In “Stack Operations”, we commented on the interesting question of what entities *could* be given names. The other side of the coin is equally interesting: what entities *must* be given names. Assembler programming, for instance, can be quite irritating because of the recurrent need to name all sorts of transient entities. K, Lisp and Perl allow the definition of anonymous functions<sup>2</sup>: APL requires a function definition to be given a name. Lisp and Perl take their inspiration from the lambda calculus, and so shall we, but without going into the subject in any depth.

If we examine a simple sample definition, like

$$\text{ARRAY\_OF} \leftarrow \lambda D. (D^*N^*N)$$

we find

- i)  $\lambda$ , the lambda itself, introducing a list of the names of the formal parameters;
- ii)  $D$ , the list (a very short list, in this case) of names of formal parameters;
- iii) the dot, marking the end of the list of names;
- iv)  $(D^*N^*N)$ , the “function body”.

This definition can be assigned, thus:

$$\text{ARRAY\_OF} \leftarrow \lambda D. (D^*N^*N)$$

and applied by name, by prefixing it to its actual parameter, thus:

$$\text{ARRAY\_OF } R$$

or applied anonymously, thus:

$$(\lambda D. (D^*N^*N)) R.$$

The result of applying the function is obtained by substituting the actual parameter for the formal parameter throughout the function body, thus:

$$(R^*N^*N).$$


---

<sup>2</sup>) Postscript, Forth and Joy go further, and dispense with names for formal parameters. During its execution, a function takes the actual values it needs from the run-time stack. Perl does a similar thing, by taking its actual values from a list with a reserved name.

And that, for the time being, is all there is to it. Fans of one-liners can now rewrite our earlier definition as

$$\text{NESTED} \leftarrow ((\lambda D. (D^*N))^*N) (H^*N) \oplus (R^*N).$$

2.3 Another device often used is to name the domain, and include a reference to that name within its own definition, so that the definition becomes self-referencing, or recursive. Defining the domain of simple arrays in the normal way:

$$\text{SIMPLE} \leftarrow (H^*N^*N) \oplus (C^*N^*N)$$

we could then use that definition in our named definition of nested arrays:

$$\text{NESTED} = \text{SIMPLE} \oplus (\text{NESTED}^*N^*N).$$

Note that the domain definition for simple arrays could be assigned. The "equals" sign in the definition of NESTED, however, is more problematic. It is not — it cannot be — assignment, and it is most certainly not the boolean test for equality, but something closer to "equals by definition", for which we lack a defined symbol.

2.4 What we want here is some equivalent to the "self" reference, making it clear that a domain definition is contemplating itself, and the obvious candidate is  $\odot$ , the "circle-jot" or "navel" symbol.

We can now write an anonymous, recursive, data domain definition for nested arrays, as follows:

$$\text{SIMPLE} \oplus (\odot^*N^*N)$$

which can therefore be assigned, thus:

$$\text{NESTED} \leftarrow \text{SIMPLE} \odot (\odot^*N^*N).$$

2.5 Problem solved? Not yet. There are problems with left recursion vs. right recursion, very like those encountered in the lexical phase of compiler writing.

The problem arises from the fact that infinite domains are not representable. Tests of domain membership can, of course, still be evaluated, by rewriting within the interpreter. An expression like  $X \in H \oplus C$ , for example, would be converted into  $(X \in H) \vee (X \in C)$ , which can then be interpreted using built-in tests for membership of the primitive data domains.

Within a compiler, a typical finite state recogniser, proceeding from left to right, can handle right recursion without difficulty. Left recursion, however, will send the recogniser into an infinite loop, and must first be removed (before the recogniser is written, usually), using special techniques to convert left recursion into right recursion.

Here in APL-land, the other side of the Looking Glass, the rewriting system will be able to handle left recursion without difficulty, but, given an expression like  $X \in \text{NESTED}$ , it will need to convert the definition of  $\text{NESTED}$  from right recursion to left recursion. We can save the machine that difficulty, by re-arranging our definition, as follows:

$$\text{NESTED} \leftarrow (\odot^* \text{N}^* \text{N}) \oplus \text{SIMPLE}.$$

2.6 The universal data domain for a typical 2nd-generation APL, then, could be written:

$$\mathbb{E} = (\odot^* \text{N}^* \text{N}) \oplus (\text{C}^* \text{N}^* \text{N}) \oplus (\mathbb{H}^* \text{N}^* \text{N}).$$

This still suffers from the drawback that the "navel" symbol is not well-defined. It is not difficult to construct a compound expression where the referent is less than obvious, unless some sort of scope rules are added to its definition.

### 3. Some Sample Descriptions

#### 3.1 K

$$\begin{aligned} \text{LIST\_OF} &\leftarrow \lambda D. (D^* \text{N}) \\ \text{SIMPLE}^3 &\leftarrow (\mathbb{H}^* \text{N}^* y) \oplus (\mathbb{R}^* \text{N}^* y) \\ \text{NESTED} &\leftarrow (\text{LIST\_OF}^* \text{N}) \text{SIMPLE} \end{aligned}$$

K is lean and mean. It recognises only atoms, drawn from the primitive data domains  $\mathbb{H}$  and  $\mathbb{R}$ , and lists, whose elements may be atoms or lists. These lists are precisely the "vectors" of APL, where the term includes all vectors of vectors, etc. Within the K lexicon, the word "matrix" is applied to lists meeting certain conditions of "rectilinearity". K does distinguish homogeneous arrays, and if this homogeneity is lost (following an indexed assignment, for instance), the array is automatically converted to a list of lists.

Syntactically, it uses pure parenthesis-and-semicolon notation to define these lists, and there is never any need for *enclose* or *disclose* operations.

#### 3.2 mainframe APL

Ideally, we would hope to find a similar simplicity in the definition of APL. Something like this, for instance:

$$\begin{aligned} \text{ARRAY\_OF} &\leftarrow \lambda D. (D^* \text{N}^* \text{N}) \\ \text{SIMPLE} &\leftarrow \text{ARRAY\_OF}(\mathbb{H} \oplus \mathbb{R}) \\ \text{NESTED} &\leftarrow (\text{ARRAY\_OF}^* \text{N}) \text{SIMPLE} \end{aligned}$$

<sup>3)</sup>  $y$  is the set  $\{0, 1\}$ , so that  $(\text{N}^* y)$  contains the empty set, and all single-element sets drawn from  $\text{N}$ , and  $(\mathbb{R}^* \text{N}^* y)$  is the set containing all scalars and all vectors drawn from  $\mathbb{R}$ .

but there are problems. The simple data space may be variously defined as

```
SIMPLE ← ARRAY_OF (M⊕R)
SIMPLE ← (ARRAY_OF M)⊕(ARRAY_OF R)
SIMPLE ← (ARRAY_OF M)⊕(ARRAY_OF C)
```

but that is not the real problem. The main difficulty is the definition of nested arrays. Mainframe implementations typically define an *enclose* function, which converts an array to a *boxed item* (of rank 0), and nested arrays can then be built up from boxed items. The interesting thing is that these boxed items can exist independently. The following (failed) attempt illustrates the difficulty:

```
ARRAY_OF ← λD. (D*N*N)
SIMPLE ← (ARRAY_OF M)⊕(ARRAY_OF R)
ENCLOSE ← λA. (A)
BOXED = ENCLOSE (SIMPLE⊕NESTED)
NESTED = ARRAY_OF (BOXED) .
```

The last couple of lines do not deliver assignable results: we need that "equals by definition" again, for which we still lack a defined symbol. Also, the BOXED and NESTED domains are indirectly recursive, which is fine for declaratives, but we do not have any defined semantics for a declarative environment. The way out is to compose ARRAY\_OF and ENCLOSE, and apply the function thus derived, allowing unlimited depth of recursion. Finally, we define the domain of boxed items separately - a necessary step only because boxed items are first class objects within the target language.

```
ARRAY_OF ← λD. (D*N*N)
SIMPLE ← (ARRAY_OF M)⊕(ARRAY_OF R)
ENCLOSE ← λA. (A)
NESTED ← ((ARRAY_OF∘ENCLOSE)*N)SIMPLE
BOXED ← ENCLOSE (SIMPLE⊕NESTED)
```

Notice that the definition given for ENCLOSE fails to capture the differences in definition between the Sharp APL and the APL2 implementations.

#### 4. Enclose, Disclose, etc.

4.1 Recall that, for a domain  $R^*J^*K$ ,  $K$  defines the rank of the array, and  $J^*K$  defines the shape, thus when  $K=0$ , and  $J^*K$  is empty, it is intuitively obvious (that is, after sufficiently long exposure to APL conventions, it becomes intuitively obvious) that the domain consists of the scalar reals. Similarly, for any other domain,  $A$ , when  $K=0$ , the domain  $A^*J^*K$  is identically equal to  $A$ .

To put it another way, if a non-empty array  $X$  is built from elements of the domain  $A$ , then the result of  $(10)\rho X$  must necessarily be an element of  $A$ . Likewise, the result of  $LIST[I]$  for integer  $I$ . There is no need for a separate *disclose* function.

4.2 Going the other way is more tricky, however. The only time an automatic *enclose* is triggered is when a semicolon is encountered in the code. A corollary of this is that the list thus in progress of definition has a minimum of two elements. Single-element lists and zero-element lists are permitted, it is just that there is no immediate way of defining them — they must be derived.

A separate *enclose* would be a convenience. The precise syntax is not important as this stage, but one promising candidate is a  $\$$  function, monadic and dyadic, which applies to depth of nesting in a manner analogous to the  $\rho$  function. Something like  $1\$SIMPLE$  to construct a single-element vector, perhaps.

4.3 Monadic  $\rho$  would (continue to) give the shape of the array at the top level of nesting. Analogous to the monadic  $\rho$ , monadic  $\$$  could be used to give the depth of the deepest object. (Such a definition has the nice feature that both monadic functions are interrogative in mood, while the dyadic definitions are imperative in mood.)

$K$  provides a  $\#$  function which gives the length of the topmost list, followed recursively by the length of the lists at the next level — provided they are all of the same length. It therefore functions as an indicator of rectangularity, providing useful information which is not otherwise easily obtainable.

4.4 Other members of the group of “convenient but not indispensable” functions, monadic head and tail functions could also be defined (co-incidentally, bringing APL a little closer to Lisp).

For a vector drawn from domain  $D$ , the domain of *head* would be  $D \leftarrow (D^*N)$ , and its definition would be

$$\uparrow V \leftrightarrow (10)\rho V$$

while the domain of *tail* would be  $(D^*N) \leftarrow (D^*N)$ , with definition

$$\downarrow V \leftrightarrow 1\downarrow V.$$

Note that *head* will fail if  $V$  is empty, and that if  $V$  is a vector of enclosed items, the scalarisation process will automatically disclose the first item, after the fashion of the *first* function defined in APL2 and Dyalog APL.

## 5. Summary

5.1 By consistently following the existing syntax, and consistently applying our semantic rules, we have seen how nested arrays can be deemed to be a part of the language, with the minimum impact on the syntax and semantics of 1<sup>st</sup>-generation APL.

That this gives us a neat and concise definition of a large and useful domain is due to in part to the power and elegance of the syntax for domain definition, and in part to the regularity and consistency of APL data structures.

5.2 We have only considered the transient data space so far: the persistent data space is a little different. This needs to be explored briefly, before we proceed with the study of how nested arrays can be included within the domain of existing primitives.

## 6. References

6.1 R D Tennent (1981), *Principles of Programming Languages*, Prentice Hall.

Once again, the idea presented above is a very close copy of Tennent's Chapter 3, except that here it has been applied to APL rather than Lisp, and uses (a slight extension of) the notation introduced for simple data domains.

6.2 David Gries (1971), *Compiler Construction for Digital Computers*, Wiley.

The particular problem of direct left recursion in regular grammars is covered in pp 93-94, and almost all other books on compiler writing, as well. Indirect left recursion is more difficult.

6.3 Manfred von Thun

<http://www.latrobe.edu.au/www/philosophy/phimvt/f00syn.html>

References on most other languages are easily available, and in any case, are not essential for an understanding of APL domains, but Joy may be less well-known than most, so the above reference is given for those open-minded persons who like to take a different world view occasionally.

# Translate your Dyalog GUIs without touching MAKE\_-code

*by Kimmo Kekäläinen (Kimmo.Kekalainen@pp.htv.fi)*

I have done it sometimes before in some special cases: provided a menu-driven possibility to change language on the fly from English to Finnish and vice versa. This far, applications have been small autonomous tools with the kind of interface that you normally start building from scratch.

As a Finn, if you even dream of your applications ending up as products and being of interest beyond domestic circles, you must either write them in English from the start, or prepare to support the language of the interface as a user configurable parameter.

How easy it's going to be to support different languages depends very much on the structure of the application. People like Adrian who went off and developed their own GUI builder tool (Causeway) having text labels, button captions and so on in arrays separated from the code, may find it quite flexible just to provide some kind of GUI to maintain relevant text strings. Whichever the maintaining method is, it has to be something that can be given to someone else (most likely not a programmer) familiar with the language to support.

I started using Dyalog APL in Winter '92, taking part in the beta test program and since then I have lived on quite happily still carrying along many programming habits that date back those early days. Even today, aside from the general dialogue forms, all the other forms in my applications are created using MAKE\_{MyApp}-like calls, keeping them in a separate layer in the structure of the application. This convention is well supported by WDesign, which is a fine tool, although I have used it quite occasionally. Even when I do use it, I always make final refinements by hand.

What to do with MAKE\_ when you face the need to support different languages?

In this approach all the text strings which appear on the form (except for the items driven directly from data) are hard-encoded in the APL functions. Am I prepared to change programming habits, adopt a different way of defining application structure to get some better language support?

What is the source of your productivity? I know a lot of APLers, Finns and foreigners, many older, a few younger – most having some kind of inclination to

ultra-individualism, very anxious to create their own styles, find their own ways to face logistic challenges. In a world as rich as APL spiced up with Dyalog fancy additions to the language like namespaces, there are endless possibilities. Do these discoveries produce a big increase in your productivity? After all, I think it is not so relevant how you do your things – what counts is how consistent you are in your way of doing them.

Well, what to do then with MAKE\_ functions that are part of your adopted structure?

Version 9 of the Dyalog APL interpreter made it possible to manipulate GUI properties directly by simple assignment. This felt good and natural, and despite the fact that it was mainly said to be based on the need to make the syntax closer to the one used in other programming languages, such as Visual Basic, known for their perfect support of third party OLE components, and to prepare for the future .NET development, you were almost tempted to indulge in inadmissible thoughts like “shouldn’t they have implemented it this way from the very beginning? This is APL, after all.”

The new supported syntax makes the old

```
'MyForm' □ws 'Caption' 'This is mine'
```

be reborn as

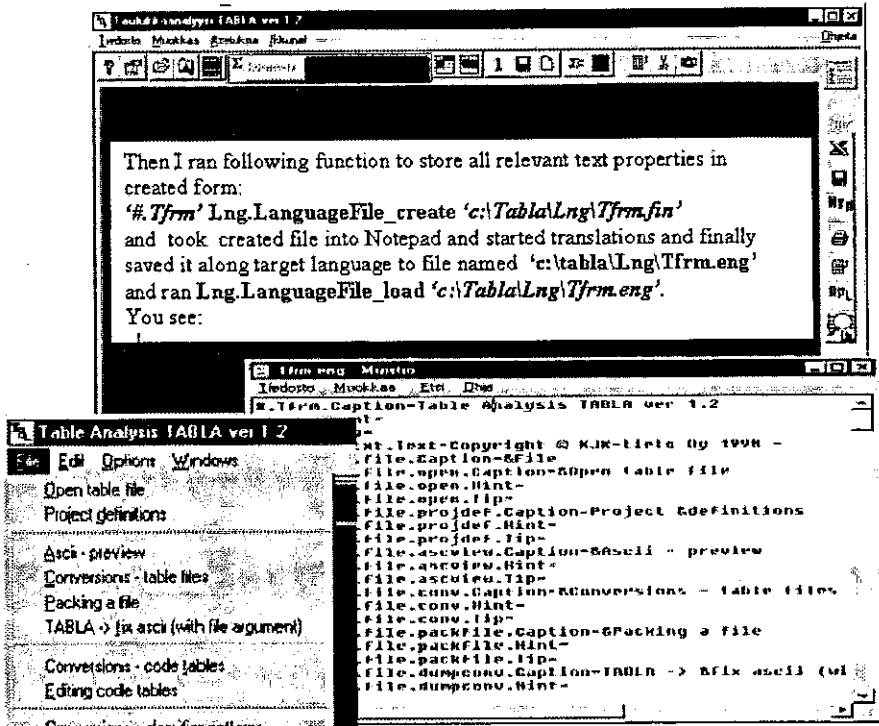
```
MyForm.Caption←'This is mine'
```

certainly more straightforward and less convoluted thanks to the reduction in the number of quotes needed. Would this be of help in our translatability problem? I started my experiments by creating a little toolset that can be run from the root, or the session namespace or any other namespace. I prefer to run it from namespace Lng.

The simple idea is to read all the language-dependent text to appear in a form directly from an active instance and store it in an ASCII file. To test my tool, I took a form written years ago and certainly not with the purpose of trying out such a tool.

To allow the new syntactic constructs, I must make sure that □wx+1 before I load the form.

## MAKE\_Tfrm



The principle on which the tool is based is that the text file with all the strings can be given to your expert in the target language. The form instance is still created with the MAKE\_ function and all the GUI strings are substituted at runtime with the ones read from the ASCII file after the form is created. With the speed of execution of today's workstations, nobody notices any speed delays.

The default language to be used is a system parameter and so are all the supported languages. They can be read from an Applini file or from the Windows registry at application start-up.

The language to be used is stored somewhere safe as a global variable or DATA property: my choice is to use

```
USE._lang+'fin' or 'eng' or 'swe'...
```

The naming conventions then are quite consistent and clear. Let's say the application root directory is C:\Tabla. Then the place for the text source files would generally always be .\Lng. If you want it to run independently of the working directory, you must use the full path. I also associate the file extensions to the language identifier. The function that associates the file name with a particular combination of form and language is *Form\_texts* (see functions).

So, to update all the captions, labels and other texts after creating the form, just execute:

```
MAKE_Tfrm ◊ Lng.LanguageFile_load Form_texts 'Tfrm'
```

Yes, I am aware that there is a world of things to consider beside MAKE\_ when you seriously aim for a language-independent solution. Still, for me this looks like a promising start and saves me from the need to re-invent everything at the same time. At least this is of great help in solving the problem associated with the strings hard-encoded in the source code, which I consider the trickiest aspect of the general problem. Anyway, I think the best benefit comes from the fact that you don't have to touch existing MAKE\_-code at all. To get the effects, just add a line after each MAKE\_ call, and magically the form appears fully translated.

Captions and contents for general modal dialog forms like OK-, WARN- and similar messages which indeed are a relevant part of any application, are most likely to be stored into some tables organised by language version with identifying hooks and read from external files, supposed to be easily editable.. Help files are in general already separate from application code and can be translated using the tools they were created with in the first place.

I include the code I use. Make sure to check the comments in the code for the rules concerning running the functions under different namespace conditions. I expect it to be flexible enough to handle most obvious situations. If you are interested in trying this yourself and want to avoid to have to retype all the code, just contact me (Kimmo.Kekalainen@pp.htv.fi) and ask for a working workspace.

## Sample Functions

```
v frm LanguageFile_create file;exp
[1] a frm = Dialog gui form
[2] a file = name for ascii file to store text properties
[3] a 'Caption' 'Text' 'Hint' 'Tip'
[4] a 'RowTitles' 'ColTitles' 'Items'
[5] a to be loaded by LanguageFile_load file
[6] exp+GetLanguageExpressions frm
[7] exp write_file file
```

[8]     a Copyright KJK-tieto Oy/Kimmo Kekäläinen

v

```

v z←GetLanguageExpressions obj;i;obs;ob;props;search;j;
 tob;prop;val;add

[1] obs←ObjectChildren obj
[2] obs←obs[;1]
[3] search←'Caption' 'Text' 'Hint' 'Tip' a these props
[4] search,←'RowTitles' 'ColTitles' 'Items' a of interest
[5] a vector vectors and text matrices (Tip)
[6] a -> ; -delimited simple text vectors
[7] z←0p<' ' a empty result
[8] :For i :In iobs
[9] ob←i>obs a one object at time
[10] props←ob [WG'PropList'] a get props
[11] props←(propssearch)/props a those of interest
[12] :If 0=pprops a if none
[13] :Continue a go next
[14] :EndIf
[15] :For j :In jprops
[16] prop←j>props a one property at time
[17] val←ob [WG prop] a get prop value
[18] tob←ob, '.', prop a join object.property
[19] :If 2=pval a if value matrix
[20] val←~i+DLTB,val, ','; 'a -> simple ; -segm vec
[21] :EndIf
[22] :If 2=|val a if nested
[23] val←~i+ΔENLIST val, ','; 'a -> simple; -segm vec
[24] :EndIf
[25] add←tob, '=', val a obj.prop=text value
[26] z←z, cadd a add it to result
[27] :EndFor
[28] :EndFor
[29] a Copyright KJK-tieto Oy/Kimmo Kekäläinen

```

v

```

v z←ObjectChildren ob;[ML];depth

[1] a returns z as two-column nested
[2] a matrix[;1]=names of children
[3] a matrix[;2]=depth from parent ob=first row
[4] a ob is parent Dialog GUI Object or [se or # or .
[5] z←, c' ' o ob, ob a pessimist default
[6] ~(←([CNC ob)ε~1 9)p0 a must exist to hope for kids
[7] z←~i+SS2VV' ', Obj_Child ob a this actually does
[8] ~(←(, '#')= '' [NS''])pdepth a if not run in root OK
[9] ~(←('#.'=2+ob)pdepth
[10] a border line case seems to be if
[11] a this fn is run in root space #
[12] a and ob name is given with full #-path
[13] a #. will not be set to children
[14] a so, for sake of consistency
[15] ~(←/(←ob)=""((pob)+''z))pdepth a if start alike OK
[16] (1>z)+2+1>z a then first has, next don't
[17] z←(c' #. '), ''z a -> change alike

```

```

[18] depth:
[19] depth+='/.'='+z
[20] :If ob=',.'
[21] :OrIf ob=',#'
[22] (1+depth)+1
[23] depth[1]+0
[24] :EndIf
[25] z+z,[1.1]depth
[26] a Copyright KJK-tieto Oy/Kimmo Kekäläinen

```

v

v z+Obj\_Child ob;obs;[]ML;t;root

```

[1] z+ob,' '
[2] obs+[]WN ob
[3] :If 0<pobs
[4] z+z,Obj_Child"obs
[5] :EndIf
[6] z+ΔENLIST z

```

v

v {parent}LanguageFile\_load file;mat;exp;i;t;ob;txt;prop;tob

```

[1] a file = ascii-file created by in LanguageFile_create
[2] a running level is guided by {parent} option. If not given
[3] a object name is taken from file and assumes names are
[4] a with full ns path = starting with #. or []se.
[5] a
[6] a If parent is given, it replaces that in file
[7] a so #. or []se. -> parent.
[8] a if parent is given as empty ''
[9] a it defaults ##. which means that object
[10] a it supposed to be found one level above
[11] a The limitation with this is that if you run
[12] a Lng-utilities under []se this does't work.
[13] a Safest way and most common case to set parent is
[14] a parent +' []ns '' which means calling level as parent
[15] a so '' Lng.LanguaFile_load 'c:\tmp\CalcBarFrm.eng'
[16] a (' []ns '') Lng.LanguaFile_load 'c:\tmp\CalcBarFrm.eng'
[17] a should work similarly
[18] mat+read_file file o +(0=1+pmat)p0
[19] mat+mat TBF'-'
[20] mat+(-mat^.= ' ')/mat o +(0=1+pmat)p0
[21] :For i :In 1+pmat
[22] exp+mat[i;] o t+exp'='
[23] ob+(t-1)+exp o txt+DTB t+exp
[24] :If 0=[]NC'parent'
[25] parent+,parent a make sure vector
[26] :If '#.'='2+ob
[27] ob+2+ob a remove root-path
[28] :EndIf
[29] :If '[]'='1+ob
[30] ob+3+ob a remove []se-path
[31] :EndIf
[32] :If 0=pparent
[33] parent+'###' a empty parent means

```

```

[34] :EndIf a look one level above
[35] ob+parent,',' ,ob a set parent
[36] :EndIf
[37] prop+(t+~1+WHERE ob=',')+ob a get property
[38] tob+(-1+pprop)+ob a obj without prop
[39]
[40] :If 9=NC tob a make sure object
[41] :Continue a really exists
[42] :EndIf a before settings
[43]
[44] :If ','etxt a if delimited (';')
[45] txt+SS2VV',' ,txt a - > vec vec
[46] :If prop='Tip' a unless Tip
[47] txt++txt a - > char mtx
[48] :EndIf
[49] :EndIf
[50] *ob,'+txt' a set object property
[51] a then with Dyalog ver < 9 you could run it
[52] a with expression below, and when stripping of
[53] a ns-spagetti, even down to 6.x
[54] a ((-1+pprop)+ob)WS prop txt
[55] :EndFor
[56] a Copyright KJK-tieto Oy/Kimmo Kekäläinen 2000 -
v

v z+Form_texts frm
[1] :If '#,'=2+frm o frm+2+frm o :EndIf
[2] z+'.\Lng\' ,frm,',' ,lang_test
v

v z+lang_test
[1] a fin / eng
[2] a supported by Gui-utils
[3] :If 0=NC'ase,._lang'
[4] SE._lang+'fin'
[5] :EndIf
[6] z+SE._lang
v

```

## Utility code

```

v vv write_file file;tn;ML;TRAP
[1] a writes vector vector to ascii file
[2] a killing old if exists
[3] ML+0
[4] TRAP+o 'F' '-_kill'
[5] _create:
[6] a if create fails go and kill
[7] file NC_CREATE tn+~1+|/0,NNUMS
[8] ML+3 o vv+vv,"cTC[2 3]
[9] vv NAPPEND tn o NUNTIE tn o +0
[10]
[11] _kill:file NNTIE tn+~1+|/0,NNUMS
[12] file NERASE tn o +_create
v

```

```

v z←read_file file;tn;⊞ML;⊞TRAP
[1] a returns ascii-file as char mtx
[2] ⊞TRAP←0 'E' '→Δerr'
[3] ⊞ML←0 ◊ file ⊞NTIE tn←⌊1+⌊0,⊞NNUMS
[4] z←⊞NREAD tn,82,⊞NSIZE tn ◊ ⊞NUNTIE tn
[5] z←⊞TC[2],z←⊞TC[3]
[6] z←↑⌊1+↑⌊'⌊⊞TC[2]=z⌋<z ◊ →0
[7] Δerr:z←0 0p'
v

v Z←DTB S
[1] a deletes trailing blancs from
[2] a text vector or matrix S
[3] →(1=ppS)ρΔV
[4] Z←(∼ϕ^⌊ϕ^⌊S=' '⌋)/S ◊ →0
[5] ΔV:Z←(ϕ∼^⌊ϕS=' '⌋)/S
v

v Z←M TBF C;R;B
[1] a vZ←M ToBlancsFrom C -- BY KJK
[2] a blanc rows after char/s C
[3] R←ρM ◊ B←,v\MeC ◊ M←,M ◊ M[B/⌊ρM]←' ' ◊ Z←RρM
v

```

Kimmo Kekäläinen  
<http://www.kjk-tieto.com>

# APL Unicode Font – Extended

*by Phil Chastney (phil @ amadeus-info.freemove.co.uk)*

## A New Release

A new version of SimPL.ttf is now available from the Vector website.

## What's Changed

The objective remains the same: to provide the full set of APL glyphs, plus enough characters to display text in all modern European languages, and anything else that looks like it might be useful. The principal differences from the earlier versions of the font are those required to bring it in line with Unicode version 3.0, plus some of the elements of the proposed extensions to Unicode version 3.2.

First among these is that the APL Quad now has its own codepoint at U+2395, which makes uniformity of appearance much easier to achieve in a large font.

The domain names **H**, **N**, **Z**, **Q**, **R**, **C** and **P** are already part of the Unicode standard (although **H** and **P** are not given any semantics), so it seems sensible to continue to use double-struck characters for domain names. A double-struck zero, **0**, is such an obvious choice for the empty domain, that it has been retained but moved from the "empty set" codepoint (U+2205) to the Private Use Area, where it keeps company with **2**, **D**, **F**, **S**, **U** and some other characters whose semantics we have yet to define.

The move is necessary because the Unicode Consortium have mandated (or, more exactly, appear to be about to mandate) that the empty set shall be represented by the circle-slash. This is a difficult area for Unicode, whose stated objective is to concentrate on semantics, not appearance. In practice, mathematic symbology works the other way, appropriating whatever symbols are available: attaching semantics to these symbols is a second, separate, stage. Maybe the Unicode consortium should just define the appearance, and leave the rest of us to interpret them as we choose? (Also, what do they mean by "slash"? Most slashes are inclined at an angle of about 24°, whereas every "empty set" symbol I have seen uses a diagonal slash, at an angle of 45°. And if I think the result looks ugly, and totally out of sympathy with the surrounding notation, that is best dismissed as just a prejudiced personal reaction.)

## What's New

Glyphs (some better than others) are provided for all the Latin and Latin-based characters in Unicode 3.0 – that is the whole of Latin 1, Latin Extended A, Latin Extended B, and Latin Extended Additional. The resultant character set covers not only the modern European languages, as per our objective, but also covers many African, North American and Asian languages as well.

- Bearing in mind that this is intended as a programmer's font, an attempt has been made to better distinguish upper-case letter "O" from the digit zero, and lower-case letter "l" from the digit "1".
- The double accents used in Vietnamese are not well done, particularly the circumflex-tilde combination on uppercase letters. This is the result of a wrong decision early in the design stage, and backing out of that mistake is going to be a long job, so the font is being released *as is*, warts and all.
- The presence of the long-tailed "r" and the addition of the Tironian Et, mean that most recent renderings of Irish are now possible, provided you can accept the "gaelic g" as simply an alternative glyph for the lower-case Latin letter "g".
- The Latin character set may not yet be complete: I believe proposals for Sami<sup>1</sup> are still outstanding, and these will be included in the font when they have been included in the Unicode standard.
- The characters necessary for Romanised Sanskrit and Pin-Yin Chinese are now all available within Unicode 3.0, and so some of the stuff previously in the Private Use Area has been shifted out.

There are a few additional characters in the Greek area – nothing major.

Coverage of Cyrillic is much improved. Coverage of Church Slavonic is incomplete, but (I believe that) all the glyphs currently in non-liturgical use are

---

<sup>1</sup> "Sami" is now the preferred term for Lappish, the language of the Laplanders. "Perhaps 30,000 speakers", according to the Dictionary of Languages. The Ethnologue site at <http://www.sil.org/ethnologue/ethnologue.html> lists eight dialects, with a total of over 40,000 speakers. This should serve as an indicator of how uncertain our knowledge is – this is no place for dogmatism: the two references do not even agree on how to spell the name of the language. One dialect, Kildin, uses the Cyrillic script.

provided, giving 100% coverage of those languages formerly known as "Soviet minority languages", with the exception of Komi<sup>2</sup>.

Unicode never promised to include pre-composed forms for all accented characters, so that they cannot really be criticised for most of the omissions listed<sup>3</sup> at

<http://www.eki.ee/letter/chardata.cgi?ucode=e000-f8ff>

but some of these characters cannot be formed from the existing repertoire of base characters and non-spacing modifiers, so I have included the lot. This should give complete coverage for Tagalog<sup>4</sup> and mediaeval Luxemburgish, for those who need it. If not, let me know what is missing.

The Unicode 3.2 proposals include extended (broken, multi-line) parentheses, brackets and braces in one of the Mathematical Symbols areas. Their proposals for multi-line braces look unworkable, so that the characters previously included in SimPL.ttf remain in the Private Use Area. Whether these characters connect vertically appears to vary from printer to printer (or does it vary from driver to driver?).

## Using the Font

Apart from that unfortunate exception, the font appears to print OK at 12pt, 24pt and 36 pt, on lasers and inkjets.

The font is still based on the Adobe standard of an M-square of 1000 units. This has caused problems in the past, with early versions of Word 2000, but no

---

<sup>2</sup> "Komi is spoken in the Komi republic in north-eastern European Russia along river valleys that drain into the Barents Sea". (The Dictionary of Languages, again.) Somewhat surprisingly, given the distance between them, their language is closely related to Sami. The Soviet administration appears to have taken a liberal view of minority languages that would shame most Western governments.

<sup>3</sup> This list is maintained by the Estonian Standards Bureau, and provides language-related information not available from the Unicode site.

<sup>4</sup> Now officially renamed "Pilipino", Tagalog, with 10,500,000 speakers, is just one of eight major languages of the Philippines, but now appears to be the dominant one. Its Latin rendition is unique in its use of double-width diacritical markings – something of a problem for those brave souls trying to implement Unicode text interpreters. The double-width tie and tilde are provided in SimPL.ttf – laying out the page is your problem. The Constitution of the Philippines provides for a new, synthetic, language to be known as Filipino.

problems have been encountered so far with either Word 2000 or Word 97, even though Microsoft still "strongly recommend" an M-square of 2048 units.

By maintaining one version of the font for development, and one for font generation, it has been possible to produce a font with a better screen display, which is particularly noticeable in those areas where NT's screen rendering has always been a bit off-hand: in Font Preview, CharMap and Insert/Symbol. (Hinting is still done automatically.)

There are 101 small things to clear up, mostly in the area of Extended Cyrillic. As these things improve, they will be made available via the Vector website, but new releases will not be announced, unless something major changes.

---

## Index to Advertisers

|                     |   |
|---------------------|---|
| Dyadic Systems Ltd  | 4 |
| Strand Software     | 2 |
| Vector Back Numbers | 5 |

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: [apl385@compuserve.com](mailto:apl385@compuserve.com).

### Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith  
Brook House  
Gilling East  
YORK, YO62 4JJ  
Tel: +44 (0) 1439-788385  
Email: [apl385@compuserve.com](mailto:apl385@compuserve.com)

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at [www.vector.org.uk](http://www.vector.org.uk)

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

## Subscribing to Vector

Your Vector subscription includes membership of the British APL Association, which is open to anyone interested in APL or related languages. The membership year normally runs from 1st May to 30th April. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: \_\_\_\_\_

Address: \_\_\_\_\_

Postcode / Country: \_\_\_\_\_

Telephone Number: \_\_\_\_\_

Email Address: \_\_\_\_\_

Category (please tick box) to run from: 1st May ☐ August ☐ Nov ☐ Feb ☐

UK private membership ..... £12 ☐

Overseas private membership ..... £14 ☐

Airmail supplement (not needed for Europe) ..... £4 ☐

UK Corporate membership ..... £100 ☐

Corporate membership overseas ..... £135 ☐

Sustaining membership ..... £430 ☐

Non-voting UK member (student/OAP/unemployed only) £6 ☐

### PAYMENT - in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: \_\_\_\_\_ Expiry date: \_\_\_\_ | \_\_\_\_

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice  
☐ one year's subscription only

Signature: \_\_\_\_\_

**Data Protection Act:**  
 The information supplied may  
 be stored on computer and  
 processed in accordance with  
 the registration of the British  
 Computer Society.

Send the completed form to:

BAA, c/o Rowena Small, 12 Cambridge Road, Waterbeach, CAMBRIDGE CB5 9NJ, UK

Fax: +44 (0) 1653 697719

## The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

### 2000/2001 Committee

|                           |                                                                                                      |                                                                                                |
|---------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Chairman                  | Adrian Smith<br>01439-788385<br>apl385@compuserve.com                                                | Brook House<br>Gilling East<br>YORK YO62 4JJ                                                   |
| Secretary                 | Anthony Camacho<br>0117-973 0036<br>acam@tesco.net                                                   | 11 Auburn Road<br>Redland<br>BRISTOL, BS6 6LS                                                  |
| Treasurer                 | Nicholas Small<br>01223-570850<br>treas.apl@bcs.org.uk                                               | 12 Cambridge Road, Waterbeach,<br>Cambridge CB5 9NJ                                            |
| Journal Editor            | Stefano Lanzavecchia<br>stf@apl.it                                                                   | c/o APL Italiana<br>Corso Vercelli 54<br>20145 - Milano<br>Italy                               |
| Projects and<br>Publicity | Dr Alan Mayer<br>01792-205678x4274<br>a.d.mayer@swansea.ac.uk                                        | European Business Management School,<br>Swansea University,<br>Singleton Park, SWANSEA SA2 8PP |
| Webmaster                 | Ray Cannon<br>01252-874697<br>100430.740@compuserve.com                                              | 21 Woodbridge Road,<br>Blackwater, Camberley,<br>Surrey GU17 0BS                               |
| Activities                | Jon Sandles<br>01904-612882<br>jon_sandles@csi.com                                                   | 138 Burton Stone Lane,<br>YORK YO30 6DF                                                        |
| Education                 | Dr Ian Clark<br>07931 370304<br>earthspot@aol.com<br><br>Dave Phillips<br>100117.2254@compuserve.com | c/o Vector Production                                                                          |
| Administration            | Rowena Small<br>01223-570850<br>treas.apl@bcs.org.uk                                                 | 12 Cambridge Road, Waterbeach,<br>Cambridge CB5 9NJ                                            |

### Journal Working Group

|               |                                                                                                                                                                    |                                                |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Editor:       | Stefano Lanzavecchia                                                                                                                                               | see above                                      |
| Production:   | Adrian & Gill Smith                                                                                                                                                | Brook House, Gilling East, YORK (01439-788385) |
| Advertising:  | Gill Smith                                                                                                                                                         | Brook House, Gilling East, YORK (01439-788385) |
| Support Team: | Jonathan Barman (01488-648575), Anthony & Sylvia Camacho,<br>Ray Cannon (01252-874697), Marc Griffiths,<br>Bob Hoekstra (01483-771028), Jon Sandles (01904-612882) |                                                |

## VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" – an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

## SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd  
The Maltings, Castlegate,  
MALTON, North Yorks YO17 7DP  
Tel: 01653-696760  
Fax: 01653-697719  
Email: sales@causeway.co.uk  
Web: www.causeway.co.uk

Compass Ltd  
Compass House  
60 Priestley Road  
GUILDFORD, Surrey GU2 5YU  
Tel: 01483-514500

Dyadic Systems Ltd  
Riverside View, Basing Road,  
Old Basing, BASINGSTOKE,  
Hants, RG24 0AL  
Tel: 01256-811125  
Fax: 01256-811130  
Email: sales@dyadic.com  
Web: www.dyadic.com

HMW Trading Systems Ltd  
Hamilton House,  
1 Temple Avenue,  
LONDON EC4Y 0HA  
Tel: 0870-1010-469  
Email: HMW@4extra.com

Insight Systems ApS  
Nordre Strandvej 119G  
DK-3150 Hellebæk  
Denmark  
Tel: +45 70 26 13 26  
Fax: +45 70 26 13 25  
Email: info@insight.dk  
Web: www.insight.dk

APL2000  
Rapid Application Development  
6610 Rockledge Drive  
Suite 502  
Bethesda MD 20817  
USA  
Email: sales@apl2000.com  
Web: www.apl2000.com

Soliton Associates Ltd  
Groot Blankenberg 53  
1082 AC Amsterdam  
Netherlands  
Tel: +31 20 646 4475  
Fax: +31 20 644 1206  
Email: sales@soliton.com

Dutch APL Association  
Postbus 1341  
3430BH Nieuwegein  
Netherlands  
Tel: +31 347 342 337